

0.1 Introduction

What is a Monte Carlo method ?

According to Nicolaus Metropolis (1953):

"A Monte Carlo method is any computational method that uses random numbers as a crucial part of the algorithm".

In general, a Monte Carlo algorithm is designed to explore a multidimensional space using stochastic methods, such as a *Markov chain*. We will see that in physics, these Markov chains often rely on the *Metropolis scheme*. It is clear from Metropolis' sentence that the first problem to face in designing a good Monte Carlo algorithm is to handle the non-trivial **problem of generating a good sequence of random numbers**.

0.2 The problem of generating random numbers

How can we generate random numbers?

It is important to notice that a truly random sequence of numbers can be generated only by physical processes that are intrinsically random. Examples could be

- The radiation decay, see for instance [1] where the authors used a radiative α -particle source and a high-resolution counter turned on for periods of 10 ms. The average number of decays in this time interval is 24.315. When the count was odd, they recorded a zero-bit; if it was even, the bit was set equal to one. From a sequence of 31 bits, they obtained a random number. By performing the sampling several times they obtained 2.5×10^6 31-bit truly random numbers;
- Thermal fluctuations in electronic devices;
- The arrival times of cosmic rays on the surface of the earth;
- Etc.

All these methods are highly impractical, especially for numerical simulations with algorithms implemented on a computer.

However, for an algorithm that has to run on a computer, the natural procedure consists of generating a sequence of random numbers through a specific algorithm. Unfortunately, by definition, each algorithm is **deterministic** and cannot generate genuinely random numbers. One then refers to these algorithms as generators of **pseudo-random numbers**.

The original idea in the '40s (still valid) was to produce a sequence of integers $I_0, I_1, I_2, \dots, I_n$ with $0 \leq I_n < 2^w$, where w is the word size of the computer, by using simple arithmetic rules of the form

$$I_{n+1} = f(I_n) \quad (1)$$

and then produce a random number ξ_n with $0 \leq \xi_n \leq 1$ by the simple division

$$\xi_n = I_n/m, \quad m = 2^w \quad (2)$$

Clearly, a sequence generated by such a deterministic rule cannot be random! In this respect, it is interesting to cite **J. von Neumann**

“Anyone who considers arithmetical methods to produce random numbers is in a state of sin”.

Despite these numbers not being random, one can still make them look as random as possible. Moreover, we must use an extensive sequence of random numbers in simulations.

A sequence of pseudo-random numbers is good if, for example,

- The cycle of the sequence (i.e., the period after which the sequence repeats itself) is very large
- The sequence is made by uncorrelated numbers (so they do not look too close to a deterministic sequence of numbers)

Many algorithms that generate pseudo-random numbers have the following common characteristics:

- The use of large prime numbers
- The use of the modulus function.
- The numbers x generated follow the uniform distribution in the unit interval i.e. $[0, 1]$.

At present, the most used random number generators of integer numbers in the interval $[0, M]$ are based on the so-called **linear congruential method** i.e. with the rule

$$I_n = (aI_{n-1} + c) \mod (m), \quad (3)$$

where m is called the **modulus**, $0 \leq a < m$ the **multiplier**, $0 \leq c < m$ the **increment** and I_0 is the initial integer (initial condition) The modulus is in principle an arbitrary number but it is convenient to consider $m = 2^w$ where w is the number of bits of a word of the computer. Given $I_n \in [0, M]$, a number $\xi_n \in [0, 1]$ is simply given by $\xi_n = I_n/M$. The advantage of the algorithm (3) is that it is **very fast**. A problem is that if the choice of the constants is inaccurate, the **cycle of the algorithm is very short**. Indeed, the same sequence will occur in general after a period not bigger than m (in general, $m/4$).

Examples

- In the old C compilers, this algorithm was implemented in the routine called **RAN_MAX** with $m = 32767$. Other old algorithms are **RAN** supplied by Digital on its computers with $m = 2^{32}$, $a = 69069$ and $c = 1$;
- **RAND** was the standard Unix generator with $m = 2^{31} - 1$, $a = 110355245$ and $c = 12345$;
- **DRAND48** was provided by IBM on its RISC machines with $m = 2^{48}$, $a = 5DEECE66D_{16}$ and $c = B_{16}$.
- The random number generator that is often used nowadays is **drand48()** that furnishes a pseudo-number in the interval $[0, 1]$ by using the double-precision extension. This is a quite good routine that can be easily used in different software.

It is possible to improve the algorithm (3) by using the following two points extension

$$I_n = (aI_{n-1} + bI_{n-2}) \mod (m). \quad (4)$$

The above algorithm needs two seeds (initial conditions) and may have a period bigger than m . A more complex example is the random number generator called **RANMAR** in the CERN library. It is an extension

0.2 The problem of generating random numbers

of a Fibonacci sequence, i.e. a sequence that is defined as $S_n = S_{n-1} + S_{n-2}$. In this case, the extension is to consider sequences as $S_n = S_{n-j} \cdot S_{n-k} \mod m$ where $\cdot$ represents any binary operation (sum, product, ..). RANMAR requires 103 seeds and has a period around 10^{43} .

The RNG proposed in our C program is called **RngStreams**. It is based on the **combined multiple recursive generator (CMRG)** introduced in [2]. At a generic **order k** a **multiple recursive generator (MRG)** is defined a k points extension of a linear congruential method

$$I_n = (a_1 I_{n-1} + \dots + a_k I_{n-k}) \mod m \quad (5)$$

where m and k are positive integers and each $a_i \in \mathbb{Z}_m = \{0, 1, \dots, m-1\}$. The maximal period length of the above recurrence is $m^k - 1$. The idea behind (CMRG) is to **combine** J copies of (5)

$$I_{j,n} = (a_{j,1} I_{j,n-1} + \dots + a_{j,k} I_{j,n-k}) \mod m_j, \quad j = 1, \dots, J \quad (6)$$

where m_j are distinct primes and the j -esim recurrence (of order k) has period $m_j^k - 1$. Given this set of recurrence, the sequence of random numbers is defined, for instance, by the following combination

$$w_n = \left(\sum_{j=1}^J \delta_j \frac{I_{j,n}}{m_j} \right) \mod 1, \quad (7)$$

The algorithm that can produce pseudo-random sequences of length $\approx 2^{191}$ is actually slightly more complex than this and if someone is interested in deepening this argument please refer to the original work [2].

Note. The RNG used by the Python function **random.py** is based on the Mersenne Twister method. It has a period of the order of $2^{19937} - 1$, is very fast, and has passed many of the statistical tests required by a good random number generator.

0.2.1 Properties of a RNG

Let us summarize some properties of the pseudo-number generators:

1. From eq. (3) and (4) it is clear that the generation of a number at time n depends on the values of the previous numbers. In this respect, the sequence of numbers obtained is **highly correlated**.
2. These algorithms generate **periodic sequences** (i.e. not random). This happens for example when the number generated coincides with the initial seed I_0 . Hence the sequence of finite random numbers at disposal is finite. This could be a problem if a given simulation requires a very large sequence of random numbers.

The restriction to a finite set of numbers implies additional **problems in high-dimensional spaces**. For example, some algorithms generate points that lie on low-dimensional subspaces (see for instance the random number generator **RANDU** (IBM) that in 3D distributes the random points along planes. This problem is related to the concept of **potency of a random number generator**

Potency of the generator

One of the known problems of a congruential generator if one is interested in sampling points in d -dimension is that the set of d -tuples $(\xi_0, \xi_1, \dots, \xi_d), (\xi_1, \xi_2, \dots, \xi_{d+1}), (\dots)$ lie on a finite number of parallel hyperplanes in the d dimensional space. Marsaglia proved that

$$\text{max number of parallel hyperplanes} = (d! m)^{1/d}. \quad (8)$$

0.3 Transformation of uniformly distributed random numbers

This means that if $m = 2^{32}$, the triples lie on at most 2953 planes, the 4-tuples at most 566, and so on. A good random number should be able to sample points **uniformly** in the d -dimensional hypercube, and the concept of **potency** is related to this problem: the higher the potency of the generator the higher the number of hyperplanes where the sampled points lie.

In the literature, there exist several statistical significance tests that investigate the properties of a random number generator. People interested in this issue could read G.S Fishman *Monte Carlo: Concepts, Algorithms and Applications*, Springer-Verlag New York,(1996).

Initialization of the seed

Almost all the pseudo-random generators have an initial state that can be initialized by using routines such as **srnd**, **srndom**, or **srnd48**. The reasons to do that are two-folds

1. The default state often generates the same sequence of numbers i.e. not random. If one wants to replicate a given simulation by starting from a different initial condition, the solution is to call the initialization method by using as a seed the less significant bits of the clock system;
2. The process must be always deterministic to be reproducible (it is sufficient to start from the same initial seed).

0.2.2 Sampling random points in higher dimensions

Several routines that generate random numbers furnish real numbers that belong to the interval $[0, 1]$. It is sometimes necessary to transform these numbers in such a way that they can assume values in the desired range. For example, one could be interested in generating integers that belong to the interval $[0, M]$ or in real numbers that belong to an interval $[a, b]$. In dimension higher than $1D$ it is sometimes necessary to generate **random points** in a given subset Ω of R^D . For example in 2D one could be interested in producing random points that are **uniformly** distributed in $\Omega \subset \mathbb{R}^2$. Examples are:

Square Random points (x, y) sampled uniformly on a square can be generated by calling a random number routine for the x component and another routine for the y component.

0.3 Transformation of uniformly distributed random numbers

In the previous sections, we have shown some algorithms that generate pseudo-random numbers $x \in \mathbb{R}$ uniformly in the unit interval $[0, 1]$. This means that the probability of occurrence of a given number x is simply given by

$$\mathbb{P}\{x \leq \xi \leq x + dx\} = p_u(x)dx = \begin{cases} 1 & \text{if } 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}, \quad (9)$$

where $p_u(x)$ is the uniform probability density distribution function (PDF). In many applications, however, we need to sample a sequence of random variables following other type of PDF $\rho(x)$, for example a Gaussian distribution with mean value μ and variance σ^2 . In general, we want to sample numbers $x \in [a, b]$ such that

$$\text{Prob}\{x_1 < \xi < x_2\} = \int_{x_1}^{x_2} \rho(x) dx. \quad (10)$$

Several methods transform the random variable x sampled uniformly in $[0, 1]$ into a random variable y that follows the desired probability distribution function $\rho(y)$. The simplest one is undoubtedly the so-called **Inversion method**.

Inversion method: discrete random variables

To better explain how this method works let us consider first with a simple **discrete example**. Suppose we have a process with 5 possible outcomes. Let p_i with $i = 1, 2, \dots, 5$ be the probability of the i -esim outcome. The normalisation requirement is given by $\sum_{i=1}^5 p_i = 1$, and the corresponding PDF is shown in Figure 0.3(a). To sample from this distribution we first compute the **cumulative distribution distribution** (CDF) $P(x)$ in the continuum case this is given by (10) with $x_1 = 0$ and $x_2 = x$. In the discrete case the CDF can be built by stacking bars on top of each other, starting at the left, see Figure 0.3(b). It is important to notice that the height of the rightmost bar must be 1 because of the normalisation. To sample from the distribution we generate a random number $\xi \in [0, 1]$ and use it to select one of the possible outcomes with a probability which is the probability of the outcome. This is obtained by projecting the event's probability onto the vertical axis and *the value of* ξ selected among them. The probability of the uniform sample hitting any particular bar is equal to the height of the bar (see Figure 0.3(c)).

Inversion method: continuum random variables

In the continuum limit the PDF becomes a smooth curve $\rho(x)$ and the CDF $F(x)$ is its integral, namely

$$F(x) = \Pr[\xi < x] = \int_0^x \rho(x') dx'. \quad (11)$$

Clearly, $F(x)$ is an increasing function with $F(a) = 0$ and $F(b) = 1$. If $F(x)$ is continuous and strictly positive the local inversion theorem tells us that there exists the inverse function $F^{-1}(x)$. Then to generate a random number X with distribution $F(x)$ it is sufficient to generate a random number ξ uniformly in $[0, 1]$ and compute $X = F^{-1}(\xi)$. Indeed,

$$\Pr[X < x] = \Pr[F^{-1}(\xi) < x] = \Pr[\xi < F(x)] = F(x). \quad (12)$$

The algorithm to draw a sample x_i from an arbitrary PDF $\rho(x)$ goes as follows:

1. Compute the CDF

$$F(x) = \int_0^x \rho(x') dx'. \quad (13)$$

Note that in general the lower limit of the integration is $-\infty$ but here we are assuming that $\rho(x) = 0$ for $x < 0$.

2. Compute the inverse $F^{-1}(x)$.
3. Generate a uniformly distributed random number ξ
4. Compute $x_i = F^{-1}(\xi)$.

Example. Exponential distribution

Given the parameter $\lambda > 0$, the normalised (between $[0, \infty]$) exponential probability distribution function $\rho(x)$, $\text{Exp}(\lambda)$ is defined as

$$\rho(x) = \begin{cases} \lambda e^{-\lambda x} & \text{if } x \geq 0, \\ 0 & \text{if } x < 0 \end{cases}. \quad (14)$$

0.3 Transformation of uniformly distributed random numbers

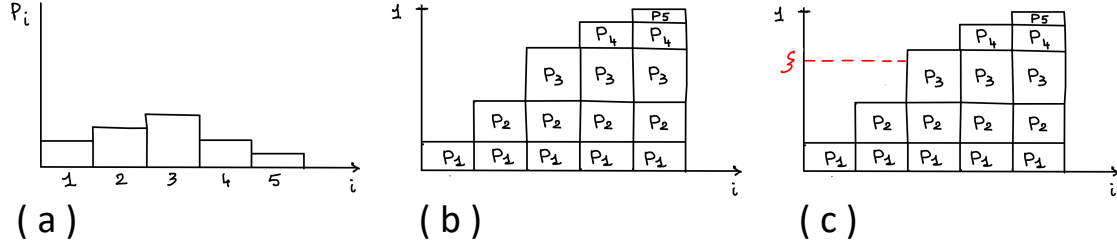


Figure 1: (a) A Discrete PDF for Five Events, Each with a Probability p_i . The sum of their probabilities $\sum_i p_i$ is necessarily (b) A Discrete CDF, Corresponding to the PDF in (b). Each column's height is given by the PDF for the event that it represents plus the sum of the PDFs for the previous events, $\sum_{j=1}^i p_j = P_i$ (c) To use the inversion method to draw a sample from the distribution described by the PDF in panel (a), a canonical uniform random variable is plotted on the vertical axis. By construction, the horizontal extension of ξ will intersect the box representing the i th outcome with probability p_i . If the corresponding event is chosen for a set of random variables ξ , then the resulting distribution of events will be distributed according to the PDF.

while its cumulative distribution $F(x)$ is

$$F(x) = \int_0^x \rho(x') dx' = \begin{cases} 1 - e^{-\lambda x} & \text{if } x \geq 0, \\ 0 & \text{if } x \leq 0 \end{cases} \quad (15)$$

The inverse of $F(x)$ can be computed and gives

$$F^{-1}(\xi) = -\lambda^{-1} \log(1 - \xi) \quad \forall \xi \in (0, 1]. \quad (16)$$

Hence given a random number ξ sampled uniformly in $[0, 1]$ the corresponding random number X that follows $\rho(x)$ is

$$X = -\lambda^{-1} \log U \sim \text{Exp}(\lambda). \quad (17)$$

In particular, the sequence of pseudo numbers $x_1, x_2, \dots, x_N$ where

$$x_i = -\frac{\log u_i}{\lambda} \quad \text{for } i = 1, \dots, N \quad (18)$$

can be regarded as realization of $\text{Exp}(\lambda)$ -distributed random variables, if $u_1, \dots, u_N$ are realizations of independent and uniformly on $(0, 1]$ distributed random variables $U_1, \dots, U_N$.

0.3.1 Transformation of coordinates

Suppose we have a d dimensional random variable $\vec{X} = \{X_1, \dots, X_d\}$ with PDF $\rho_{\vec{X}}(\vec{x})$ and we perform a transformation of variable $\vec{Y} = T(\vec{x})$ where T is in general a bijection. Then the densities $\rho_{\vec{X}}(\vec{x})$ and $\rho_{\vec{Y}}(\vec{y})$ are related by the relation

$$\rho_{\vec{Y}}(\vec{y}) = \rho_{\vec{X}}(T(\vec{x})) = \frac{\rho_{\vec{X}}(\vec{x})}{|\det J_T(\vec{x})|}. \quad (19)$$

0.3 Transformation of uniformly distributed random numbers

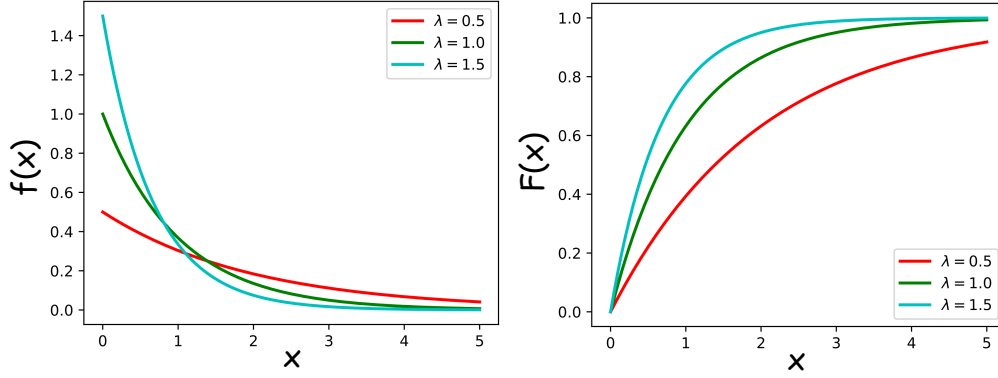


Figure 2: Exponential probability density distribution (left panel) and its cumulative distribution $F(x)$ (right panel) plotted for 3 different values of the first moment λ

where

$$J_T = \begin{pmatrix} \partial T_1 / \partial x_1 & \cdots & \partial T_1 / \partial x_d \\ \vdots & \ddots & \vdots \\ \partial T_d / \partial x_1 & \cdots & \partial T_d / \partial x_d \end{pmatrix} \quad (20)$$

is the Jacobian matrix of the transformation.

Example. Let us consider the polar coordinates transformation

$$\begin{aligned} x &= r \cos \theta = T_1(r, \theta) \\ y &= r \sin \theta = T_2(r, \theta). \end{aligned} \quad (21)$$

The Jacobian is

$$J_T = \begin{pmatrix} \partial(r \cos \theta) / \partial r & \partial(r \cos \theta) / \partial \theta \\ \partial(r \sin \theta) / \partial r & \partial(r \sin \theta) / \partial \theta \end{pmatrix} = \begin{pmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{pmatrix} \quad (22)$$

Hence $|\det J_T| = r$ and according to equation (19) we have

$$\rho(x, y) = \frac{\rho(r, \theta)}{r}. \quad (23)$$

Typically we need to do the opposite transformation i.e.

$$\begin{aligned} r &= T_1(x, y) \\ \theta &= T_2(x, y) \end{aligned} \quad (24)$$

In this case, it is easy to see that

$$\rho(r, \theta) = r \rho(x, y). \quad (25)$$

Example. In the case of spherical coordinates transformation

$$\begin{aligned} x &= r \sin \theta \cos \phi \\ y &= r \sin \theta \sin \phi \\ z &= r \cos \theta, \end{aligned} \quad (26)$$

It is possible to show **TODO** that

$$\rho(r, \theta, \phi) = r^2 \sin \theta \rho(x, y, z). \quad (27)$$

0.3.2 Sampling from n -points PDF: marginal and conditional distributions

Suppose we want to generate random numbers according to a 2D joint PDF $\rho(x, y)$. Sometimes multidimensional densities are separable and hence they can be expressed as the product of the 1D PDFs

$$\rho(x, y) = \rho_x(x)\rho_y(y). \quad (28)$$

In this case, the random variables (X, Y) can be sampled by sampling X from $\rho_x(x)$ and independently Y from $\rho_y(y)$. Sometimes this can be achieved simply by performing a transformation of coordinates.

Example. Suppose we want to draw two numbers X and Y from the (normalised) 2D Gaussian PDF

$$\rho(x, y)dx dy = \frac{1}{2\pi} \exp\left(-\frac{x^2 + y^2}{2}\right) dx dy \quad (29)$$

If we pass into polar coordinates the PDF becomes

$$\rho(r, \theta) = dr d\theta = \frac{1}{2\pi} \exp\left(-\frac{r^2}{2}\right) r dr d\theta = \rho_r(r)\rho_\theta(\theta)dr d\theta \quad (30)$$

where $\rho_r(r) = r \exp\left(-\frac{r^2}{2}\right)$ and $\rho_\theta = \frac{1}{2\pi}$. In this case, we can sample the two variables independently, θ uniformly in $[0, \pi]$ and r according to an exponential PDF. Of course, we can also use the method involving the marginal and the conditional PDFs as explained above.

Most of the time, however, densities aren't separable and we need a different approach to sample from $\rho(x, y)$ directly and in general from a n -point distribution $\rho(x_1, x_2, \dots, x_n)$.

The idea is to consider the **marginal density function** obtained by integrating one of the dimensions

$$p(x) = \int \rho(x, y) dy. \quad (31)$$

This can be seen as the PDF for X alone. The **conditional density function** $p(y|x)$ is the density function for y given that some particular value of x has been chosen

$$p(y|x) = \frac{\rho(x, y)}{p(x)}. \quad (32)$$

This suggests the following approach for a 2D sampling:

- First compute the marginal distribution to $p(x)$ to isolate one particle variable and draw a sample from $p(x)$ using standard 1D techniques.
- Given the sample X compute the conditional density function given that value X and draw a sample Y from the $p(y|x)$ again using 1D standard technique.

0.3.3 Von Neumann rejection method

Often it is not possible to find the inverse function $F^{-1}(x)$ and one has to rely on different methods. The simplest one is called the **rejection method** (or **dart-throwing method**), which was introduced by Von Neumann and goes as follows: Suppose we want to generate a random number following the pdf $\rho(x)$ for which $F^{-1}(x)$ is not known. We then sample x according to a second pdf, say $g(x)$, from which is relatively simple to generate random numbers distributed according to it and such that $\rho(x) \leq c g(x)$, $\forall x$ where $c > 1$ is a constant (In general c is not very far from 1). The algorithm goes as follows:

0.4 Numerical integration by Monte Carlo methods: direct sampling

1. Generate X according to $g(x)$ and a uniform random number $\xi \in [0, 1]$;
2. If $\xi \geq \rho(X)/(cg(X))$ go back to step 1 and sample new X and U , otherwise store X .

The idea is very simple: Let X is a random variable with distribution $g(x)$ and ξ a random number distributed uniformly in $[a, b]$ the point $(x = X, y = c\xi g(X))$ is uniformly distributed in the plane region

$$R = \{(x, y) : a \leq x \leq b, 0 \leq y \leq cg(x)\} \quad (33)$$

Then, to sample X distributed with $\rho(x)$ we accept the points (x, y) such that $y < \rho(x)$ and reject all the others. The accepted points will be distributed in the region

$$R' = \{(x, y) : a \leq x \leq b, 0 \leq y \leq \rho(x)\} \quad (34)$$

and hence X will be distributed according to $\rho(x)$. See Figure 3 for an illustrative example of this idea. From the picture and the above arguments it is clear that the efficiency of this scheme depends on how tightly $c g(x)$ bounds $\rho(x)$. This technique works in any dimension.

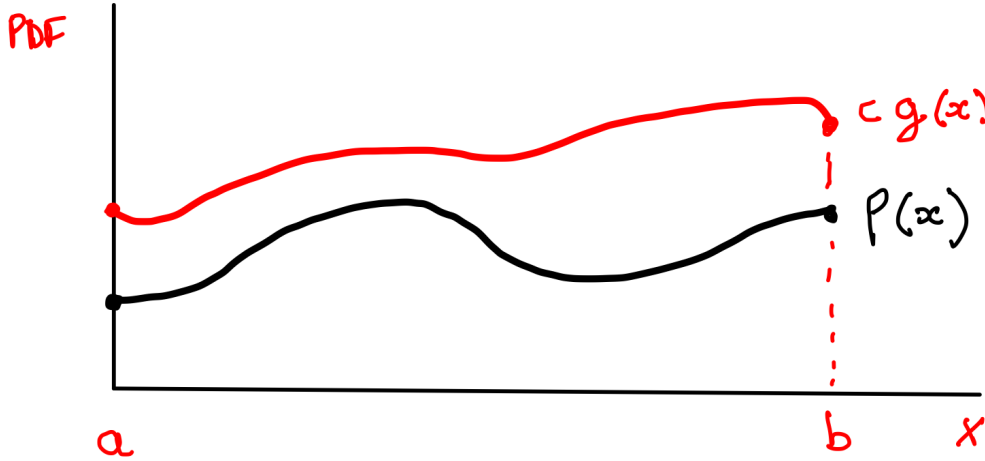


Figure 3: Rejection technique: the closer the function $cg(x)$ is to $\rho(x)$ the more efficient the sampling is

0.4 Numerical integration by Monte Carlo methods: direct sampling

Let us now describe some standard methods to perform numerical integrations using Monte Carlo (i.e. random numbers) methods.

0.4.1 Hit and miss method

This is the least efficient method. It relies upon sampling points inside the region of integration.

Suppose we want to integrate numerically the following 1D integral

$$I = \int_a^b f(x) dx. \quad (35)$$

Let $m \leq \min_{x \in [a, b]} f(x)$ and $M \geq \max_{x \in [a, b]} f(x)$. The algorithm goes as follows:

0.4 Numerical integration by Monte Carlo methods: direct sampling

1. Let $i=1$;
2. Choose two random numbers A_{2i} and A_{2i+1} between 0 and 1.
3. Let $x = (a + (b - a)A_{2i})$ and $y = (M - m)A_{2i+1}$;
4. If $y + m < f(x)$ let $\chi_i = 1$ (hit); otherwise $\chi_i = 0$ (miss);
5. If $i \geq N$, where N is the maximum number of iterations, stop; otherwise, let $i = i + 1$ and go back to step 2.

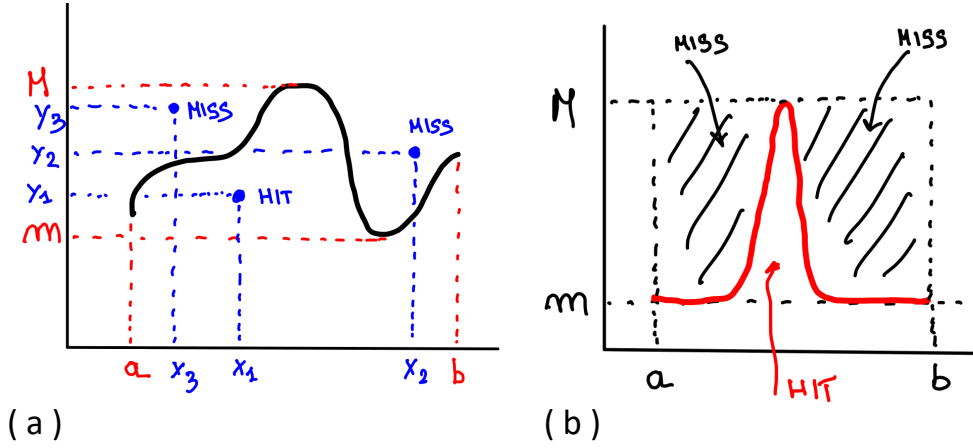


Figure 4: (a) Hit and miss method. (b) A case where the hit-and-miss method is very inefficient

The N points approximation of I in (35) is then given by

$$I_N = \left(\frac{1}{N} \sum_{i=1}^N \chi_i \right) (b-a)(M-m) + m(b-a) \quad (36)$$

Note that $(b-a)(M-m)$ is the area of the rectangle inscribing the area underneath the function. It is clear that if the function is very peaked around a given point and the rest of the domain is almost zero (see Figure) this method is very inefficient since it does not focus on the shape of the function.

0.4.2 Crude Monte Carlo

This algorithm starts from the following consideration: Suppose we generate random numbers $x \in [a, b]$ that are uniformly distributed. This means that these x are distributed according to the probability measure $d\mu = dx/(b-a)$ (normalised to 1). Hence the integral (35) can be formally written as

$$I = (b-a) \int_a^b d\mu f(x) = (b-a) \langle f \rangle_\mu, \quad (37)$$

where $\langle f \rangle_\mu$ is the average with respect to the measure μ that depends on the integration domain $[a, b]$. Eq. (37) suggests performing a N -points estimate of such an average. This can be done, for instance, by using the following simple algorithm:

0.4 Numerical integration by Monte Carlo methods: direct sampling

1. Let $i=1$;
2. Generate a random number A_i uniformly between 0 and 1;
3. Let $x = a + (b - a) A_i$ (a random number $\in [a, b]$) and $f_i = f(x)$;
4. If $i \geq N$, where N is the maximum number of iterations, stop; otherwise let $i = i + 1$ and go back to step 2.

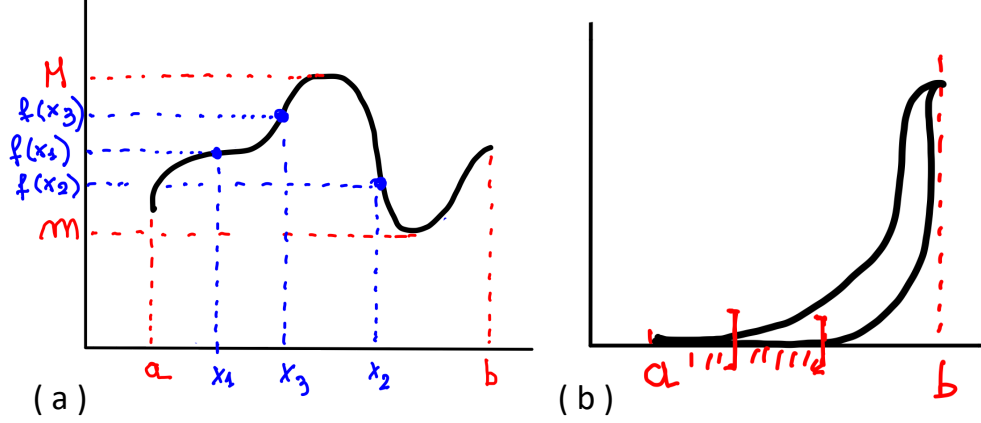


Figure 5: (a) Crude or analog Monte Carlo method. (b) Examples where the method is very inefficient

The N points estimate of $\langle f \rangle_\mu$ is then given by

$$\langle f \rangle_\mu = \frac{1}{N} \sum_{i=1}^N f_i \quad (38)$$

and multiplying by $b - a$ we have the N points estimate of I_N :

$$I_N = \frac{(b - a)}{N} \sum_{i=1}^N f_i. \quad (39)$$

As for the hit-and-miss method, if the function $f(x)$ is non-negligible within a small subset of the integration interval $[a, b]$ this method is not very efficient. Still, it is rather simple to generalise it in any dimension where Monte Carlo methods become more efficient than standard methods

0.4.3 Crude MC method for multidimensional integrals

The crude Monte Carlo method can be easily extended to compute multidimensional integrals such as

$$I = \int_{\Omega} f(x_1, \dots, x_D) dx_1 \cdots dx_D,$$

where, in general, Ω is a finite domain with an arbitrary shape. In this case, the method starts considering a hyper-rectangle H in R^D that contains the domain i.e. such that $\Omega \subset H = I_1 \times I_2 \times \cdots \times I_D$. Then, the algorithm goes as follows:

0.4 Numerical integration by Monte Carlo methods: direct sampling

1. Let $i = 1$ and $k = 0$;
2. Generate a random point $x = (x_1, x_2, \dots, x_D)$ belonging to the hyper-rectangle H . This means to call D times the pseudo-random generator routines, i.e. one for each component.
3. If $x \in \Omega$ let $N_\Omega = N_\Omega + 1$ and $f_{N_\Omega} = f(x)$.
4. If $i \geq N$, where N is the maximum number of iterations, stop; otherwise, let $i = i + 1$ and go back to step 2.

Again, it possible to show that

$$I_N = V \langle f \rangle_N = \frac{V}{N} \sum_{i=1}^{N_\Omega} f_i, \quad (40)$$

where N_Ω is the number of random points falling $\in \Omega$ and V is the volume of the hyper-rectangle H . Note that if we consider $f(x) = \chi_\Omega$, i.e. $f(x) = 1$ if $x \in \Omega$ and $f(x) = 0$ otherwise (χ_Ω is the **indicator** or **characteristic function** of the region Ω), the integral I furnishes the volume of the region Ω . The two methods described above are simple examples of **direct sampling**.

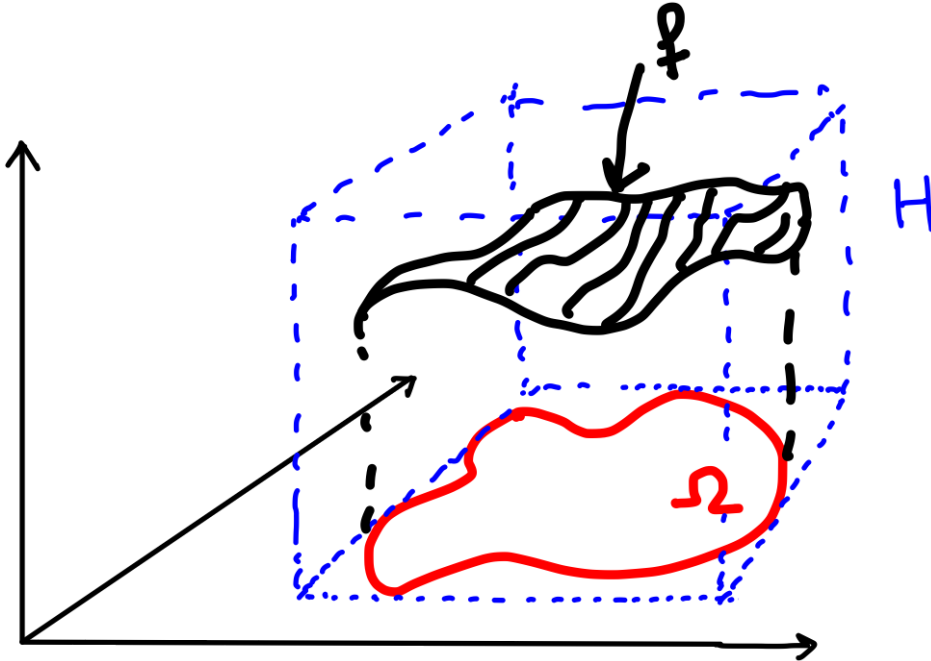
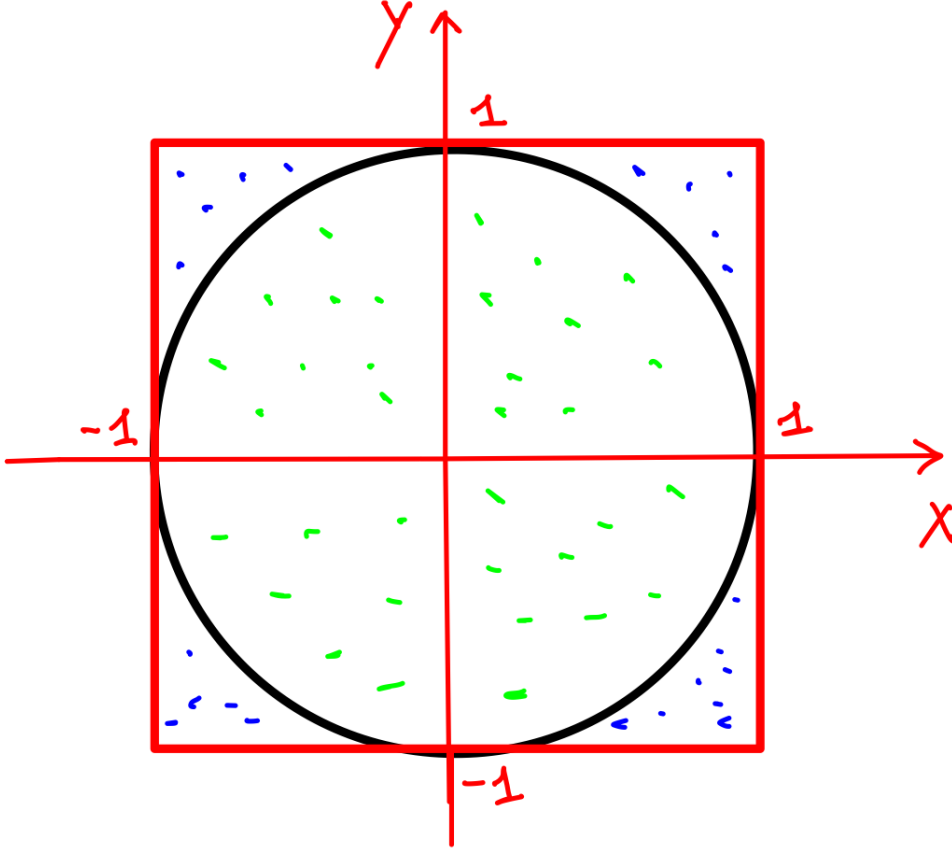


Figure 6: Crude MC method in D dimensions.

Example (Computation of π). *As an interesting application of the above MC method let us estimate the irrational number π .*

Figure 7: Set up for the estimate of π by the crude Monte Carlo method

Suppose we inscribe a circle of radius R within a square of linear size $L = 2R$. If we denote by A_c and A_s respectively the area of the circle and the area of the square we have

$$\mathbb{P}(x^2 + y^2 < 1) = \frac{A_c}{A_s} = \frac{\pi R^2}{4R^2} = \frac{\pi}{4}. \quad (41)$$

Hence, if we consider a circle with $R = 1$ and we generate a sequence of N random points (x_i, y_i) (uniformly in the square $[-1, 1] \times [-1, 1]$), we can define the following random variable

$$\chi_i = \begin{cases} 1 & \text{if } x_i^2 + y_i^2 < 1, \\ 0 & \text{otherwise} \end{cases} \quad (42)$$

They are independent and identically distributed random variables with expectation $\mathbb{E}[\chi_i] = \pi/4$. From the Strong Law of Large number, we know that as $N \rightarrow \infty$, the arithmetic mean

$$Y_N = \frac{1}{N} \sum_{i=1}^N \chi_i \quad (43)$$

converges in probability (or almost surely) to $\pi/4$. Hence, Y_N is an **unbiased** and consistent estimator of $\pi/4$,

0.4 Numerical integration by Monte Carlo methods: direct sampling

i.e. the probability of $4Y_N$ being a good approximation for π if N is sufficiently large.

$$\boxed{\frac{4M}{N} \rightarrow \pi} \quad (44)$$

in probability.

0.4.4 The needle problem: an old hit and miss method to estimate π

Perhaps the oldest example of such an approach is the so-called **Buffon's needle methods** (by Georges-Louis Leclerc, Comte de Buffon (1707-1788)) to estimate π . The idea is to throw a needle of length ℓ on a plane decorated by long parallel lines each separated by a distance d . (see Fig. 8) By calling x the position of the centre of mass of the needle (in general $0 \leq x \leq \ell$ but if the needle is uniform in shape x coincides with its middle point) and by θ the angle formed by the needle with the axis perpendicular to the lines ($-\pi/2 \leq \theta \leq \pi/2$), one can realise from the figure that the probability that the needle crosses the line is given by

$$P_c = \frac{2}{d\pi} \int_0^{\ell/2} dx \int_{-\arccos(2x/\ell)}^{\arccos(2x/\ell)} d\theta = \frac{2\ell}{d\pi} \int_0^1 \arccos y dy = \frac{2\ell}{d\pi} \quad (45)$$

To estimate the integral by the Monte Carlo method we can then imagine throwing the needle on the plane N times and counting the number N_c of times it crosses the lines. From the frequentistic definition of the probability of an event, we can estimate this probability as $P_c \approx N_c/N$. From this estimate, we finally get an estimate of π

$$\pi \approx \frac{2\ell}{d} \frac{N}{N_c} \quad (46)$$

0.4.5 Efficiency of the direct sampling MC method

To discuss this issue let us go back to the 1D case but generalize integral (37) to an evaluation of the mean of a function $f(x)$ of a random variable X that follows a given PDF $\rho(x)$. Formally we have

$$\langle f \rangle_\rho = \int_{\mathbb{R}} f(x) \rho(x) dx, \quad (47)$$

Note that the average $\langle f \rangle_\mu$ in (37) is a simple case of (47) where $\rho(x) = \frac{1}{(b-a)}$ i.e. the uniform probability distribution function on the interval $[a, b]$. In this case, if we can generate a sequence of N pseudorandom numbers $\{x_1, x_2, \dots, x_N\}$ from the PDF $\rho(x)$ the direct sampling method furnishes the N -points estimate of $\langle f \rangle_\rho$ as

$$\text{Est}[\langle f \rangle_\rho] \equiv \bar{f}_N = \frac{1}{N} \sum_{i=1}^N f(x_i). \quad (48)$$

On the other hand, if $\{x_i\}_{i=1, \dots, N}$ is a sequence of independent random variables (**irv**) also $\{f(x_i)\}_{i=1, \dots, N}$ is a sequence of **irv**. In this case from the **Strong law of large numbers**, we know that

$$\bar{f}_N \equiv \frac{1}{N} \sum_{i=1}^N f(x_i) \xrightarrow{N \rightarrow \infty} \langle f \rangle_\rho, \quad (49)$$

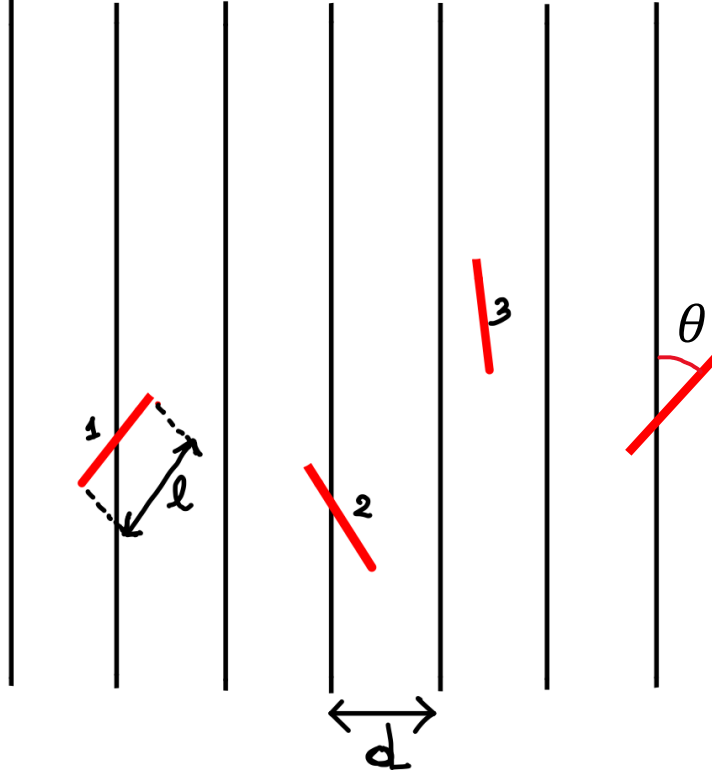


Figure 8: The needles 1 and 2 cross the lines, while needle 3 does not.

where the convergence is in probability i.e. more rigorously

$$\mathbb{P} \left\{ \lim_{N \rightarrow \infty} \bar{f}_N = \langle f \rangle_\rho \right\} = 1 \quad (50)$$

Moreover, by the **Central Limit Theorem** we know that the PDF of the random variable $\bar{f}_N$ tends to a Gaussian with mean $\langle f \rangle_\rho$ and variance σ^2 / N where

$$\sigma^2 = \mathbb{E} \left[(f(x) - \langle f \rangle_\rho)^2 \right] = \int_{\mathbb{R}} [f(x) - \langle f \rangle_\rho]^2 \rho(x) dx. \quad (51)$$

We can say that the crude Monte Carlo estimate of $\langle f \rangle_\rho$ is given by

$$\text{Est}[\langle f \rangle_\rho] = \bar{f}_N \pm \frac{\sigma}{\sqrt{N}}. \quad (52)$$

where $\sigma/\sqrt{N}$ defines the **one-sigma confidence interval**. The convergence is again in probability, i.e. more rigorously we have

$$\mathbb{P} \left\{ |\bar{f}_N - \langle f \rangle_\rho| \leq \frac{\lambda \sigma}{\sqrt{N}} \right\} = \sqrt{2\pi} \int_{-\lambda}^{\lambda} e^{-x^2/2} dx + O(1/\sqrt{N}). \quad (53)$$

Say it differently, with a probability p given by

$$p = \frac{\sqrt{N}}{\sigma\sqrt{2\pi}} \int_{\langle f \rangle_\rho - (\sigma/\sqrt{N})}^{\langle f \rangle_\rho + (\sigma/\sqrt{N})} \exp \left[-\frac{N(x - \langle f \rangle_\rho)^2}{2\sigma^2} \right] dx = \frac{1}{\sqrt{2\pi}} \int_{-1}^{+1} \exp \left[-\frac{y^2}{2} \right] dy = 0.68286 \quad (54)$$

we expect $\bar{f}_N$ to lie within $\pm\sigma/\sqrt{N}$ around $\langle f \rangle_\rho$, if N is sufficiently large.

Note. Since we do not know a priori σ we should approximate it by using the same MC method. The estimate would be

$$Est[\sigma^2] \equiv S_N^2 = \frac{1}{N} \sum_{i=1}^N f(x_i)^2 - \left[\frac{1}{N} \sum_{i=1}^N f(x_i) \right]^2. \quad (55)$$

The quantity $\pm S_N/\sqrt{N}$ is called the **statistical error** associated to the estimate $\bar{f}_N$ of the mean.

We can now come back to discuss the reliability of the estimate I_N of the integral I in eq. (37) computed with the direct sampling MC method. This is given by eq.(39) while, according to what we have just discussed, the associated statistical error is $\pm S_N/\sqrt{N}$, where

$$S_N^2 = \frac{b-a}{N} \sum_{i=1}^N [f(x_i)]^2 - [I_N]^2. \quad (56)$$

It is important to notice that **the statical error is proportional to $1/\sqrt{N}$ i.e. independent of the dimensionality D of the integral !**

0.4.6 Monte Carlo vs deterministic numerical methods

It is interesting to compare this rate of convergence with those obtained by using classical deterministic integrators such as the **trapezoidal rule**, the **Simpson rule** and the **Gaussian rule**. All these algorithms have to compute the integrand $f(x)$ in a large number of points N of the underlying domain $\Omega \subset \mathbb{R}^D$. This is usually done by building a grid on the smallest hyper-rectangle containing Ω of average linear length H . If L is the number of points in each of the D directions, the interval of integration is H/L and the total number of points is of order $N = L^D$. The errors of the different deterministic rules are, on the other hand, characterized by the following rate of convergence:

Trapezoidal rule Since the error ϵ for this rule goes like $\epsilon = f''(X)h^2$ the method will converge as $N^{-2/D}$

Simpson rule In this case $N^{-4/D}$.

Gauss rule with m points In this case $N^{-(2m-1)/D}$

The Monte Carlo method is less efficient for relatively small dimensions but becomes numerically advantageous when D is sufficiently large, a situation often encountered for instance in particle physics (quantum field theory) and statistical mechanics.

Note. Other reasons to use Monte Carlo methods can be summarized as follows:

- It is a very simple method to design and use.
- The errors are somehow “built into ” the method.

There is however an important drawback to consider: although the statistical error decreases as $1/\sqrt{N}$, the computing time increases linearly with the sample size (N). Hence, the crude Monte Carlo is practically not very useful and we should resort to methods that **reduce the variance without altering the mean**. These are called **variance reduction techniques** and perhaps the most used one is the **importance sampling method**

Chapter 1

Monte Carlo methods in Statistical Physics

For the sake of simplicity let us first consider a system with a discrete set of microscopic configurations. Let us denote by $\mathcal{C}$ and $E(\mathcal{C})$ respectively one of these configurations and its associated energy. If the system is at equilibrium with an external thermal bath at temperature T , according to the **Canonical ensemble** description we know that the probability of finding it in the microstate $\mathcal{C}$ is proportional to $\exp(-\beta E(\mathcal{C}))$ where $\beta = 1/k_B T$ and k_B the Boltzmann constant.

Let $O(\mathcal{C})$ be an observable of the system, for instance, the energy itself the magnetisation in a magnetic system, etc. The average value of $O(\mathcal{C})$ in the canonical ensemble is given by:

$$\langle O \rangle = \frac{1}{Z} \sum_{\mathcal{C}} O(\mathcal{C}) e^{-\beta E(\mathcal{C})}, \quad (1.1)$$

where the normalisation factor Z is the **canonical partition function**

$$Z = \sum_{\mathcal{C}} e^{-\beta E(\mathcal{C})}. \quad (1.2)$$

To fix the ideas, let us consider as a statistical system the **Ising model** for ferromagnets.

1.1 Ising model

A simple model of ferromagnetic was proposed by Prof. Lenz in 1923 to his student Ising who solved it in $d = 1$. In the Ising model, a spin has only two states, $\sigma = \pm 1$ (up and down state). For ferromagnets on a Bravais lattice of N vertices a generic configuration $\mathcal{C}$ would be given by the set of spins $\{\sigma_1, \sigma_2, \dots, \sigma_N\}$ where σ_i is the spin variable at site i . Pair of spins (i, j) at distance d_{ij} on the lattice may interact, and the energy associated with the pair is given by $E_{ij} = -J\sigma_i\sigma_j$ where $J > 0$ for a ferromagnetic measure the strength of the interaction. In the Ising model the interaction between pairs is different from zero only if the two sites are nearest neighbours on the lattice. This property is indicated by $\langle ij \rangle$. Therefore, the energy associated to a configuration $\mathcal{C}$ is given by

$$E(\mathcal{C}) = -J \sum_{\langle ij \rangle} \sigma_i \sigma_j. \quad (1.3)$$

Note that the energy of the system is invariant by a global flip of all the spins Z_2 symmetry. If we introduce an external homogeneous magnetic field that couples with all the spins the total energy becomes

$$E(\mathcal{C}, H) = -J \sum_{\langle ij \rangle} \sigma_i \sigma_j - H \sum_i \sigma_i. \quad (1.4)$$

Note that the magnetic field breaks explicitly the symmetry of the system. Clearly, at very low temperatures, $k_B T \ll J$, the system will show configurations with low energy, and large magnetization is achieved when most of the spins are aligned. On the other hand, at very large Temperatures the most probable configurations will be highly disordered (high entropy) and the macroscopic phase will be **paramagnetic**. One then could expect that the system transforms from a disordered paramagnetic phase to an ordered, ferromagnetic, one. The phase transition will be located at a given critical temperature T_C . Ising solved the problem exactly in $d = 1$ showing that in $d = 1$ there is no phase transition. In particular, the magnetization $M(T)$ decreases slowly and smoothly as T increases. Moreover, the magnetic susceptibility $\chi_M = \partial M / \partial T$ is finite at all temperatures. On the other hand under a mean-field approximation, the Ising model exhibits phase transition at $T_c = zK/k_B$ where z is the lattice coordination number, i.e. in the mean-field the critical temperature does not depend on d but only on the number of nearest neighbours. Onsager solved the problem in $d = 2$ showing that indeed in this dimension the paramagnetic-ferromagnetic transition occurs at a temperature T_c different from zero i.e. such that $k_B T_C / J = 2 / \ln(1 + \sqrt{2})$. To study the problem numerically one should in principle compute the partition function

$$Z(T) = \sum_{\mathcal{C}} \exp(-\beta E(\mathcal{C})). \quad (1.5)$$

or estimate the statistical average of an observable $O(\mathcal{C})$ as

$$\langle O \rangle = \frac{1}{Z(T)} \sum_{\mathcal{C}} O(\mathcal{C}) \exp(-\beta E(\mathcal{C})). \quad (1.6)$$

Since Eqs. (1.5), (1.6) are practically integrals the natural question is:

Can we estimate partition functions and in particular averages of observables of the statistical ensembles by Monte Carlo methods?

Let us consider the case of an Ising model on a square lattice $L \times L$. This system contains $N = L^2$ spins and $M = 2^N$ possible configurations $\mathcal{C}$.

1.1.1 The inefficient (uniform) method

The most straightforward procedure consists of sampling stochastically the configurational space $\{\mathcal{C}\}$ by generating randomly a given configuration $\mathcal{C}_i = (\sigma_1^{(i)}, \sigma_2^{(i)}, \dots, \sigma_N^{(i)})$. This is achieved, for example, by choosing L^2 spins either up or down with equal probability. Each time a configuration is generated from scratch it is uncorrelated from the previously generated ones. Given a “point” c we should then compute $e^{-\beta E(\mathcal{C}_i)}$ and sum over all the sampled configurations. This approach is similar to the one we used to estimate integrals in D dimensions by using direct MC methods, for instance. Notice that in that case $x_i \in (\mathbb{R})^D$ while here each “point” $\mathcal{C}_i$ has dimension N a parameter that is supposed to be very large eventually (thermodynamic limit).

This is however a very inefficient strategy to solve the problem, at least for two reasons:

1. Even if we consider a relatively small value of L (i.e. very far from the thermodynamic limit where phase transitions rigorously occur) the number of possible micro-states is very large to be sampled as suggested. For instance, if $L = 20$ there are $M = 2^{L^2} = M = 2^{400} = 10^{120}$ configurations !

2. For $\beta \gg 1$ the PDF

$$\pi_\beta(\mathcal{C}) = \frac{1}{Z_N(\beta)} \exp[-\beta \mathcal{H}(\mathcal{C})] \quad (1.7)$$

is non-negligible only for a few micro-states, those having the smallest values of E and representing the ferromagnetic phase in these models.

In this case, in which the sum involved in the average is dominated by very few micro-states a simple random generative method that does not take into account the PDF $\pi_\beta(\mathcal{C}) = \exp(-\beta E(\mathcal{C}))/Z$ in its sampling procedure would be extremely inefficient.

So, as for the case of numerical integration by MC in which the function is very peaked in small regions and almost negligible in most parts of the remaining domain, this method will spend most of the time in sampling regions of the configurational space that do not contribute to the averages in an appreciative way. To solve at least partially this problem it is also clear from the previous sections that we need to sample non-uniformly on the space of configurations, for instance by using the **importance sampling** method.

1.1.2 Importance sampling method in statistical mechanics

As done before, the idea is to rewrite the integral

$$\langle O \rangle_\pi = \frac{1}{Z(T)} \sum_{\mathcal{C}} O(\mathcal{C}) \exp(-\beta E(\mathcal{C})) = \sum_{\mathcal{C}} O(\mathcal{C}) \pi(\mathcal{C}) \quad (1.8)$$

as

$$\langle O \rangle = \frac{\sum_{\mathcal{C}} O(\mathcal{C}) \frac{\exp(-\beta E(\mathcal{C}))}{g(\mathcal{C})} g(\mathcal{C})}{\sum_{\mathcal{C}} \frac{\exp(-\beta E(\mathcal{C}))}{g(\mathcal{C})} g(\mathcal{C})} = \frac{\langle O \frac{e^{-\beta E(\mathcal{C})}}{g(\mathcal{C})} \rangle_g}{\langle \frac{e^{-\beta E(\mathcal{C})}}{g(\mathcal{C})} \rangle_g}, \quad (1.9)$$

where, in general, $g(\mathcal{C})$ is an arbitrary probability density function on $\{\mathcal{C}\}$. The above equation suggests that if one chooses as $g(\mathcal{C})$ the Boltzmann factor itself, i.e. if

$$g(\mathcal{C}^{(i)}) \propto \frac{1}{Z(T)} \exp(-\beta E(\mathcal{C}^{(i)})) \quad (1.10)$$

then the estimate of $\langle O \rangle$ simplifies drastically to the arithmetic mean

$$\langle O \rangle = \frac{\langle O \frac{e^{-\beta E}}{g} \rangle_g}{\langle \frac{e^{-\beta E}}{g} \rangle_g} \simeq \frac{1}{t_{max}} \sum_{i=1}^{t_{max}} O(\mathcal{C}^{(i)}). \quad (1.11)$$

where t_{max} is the number of configurations $\mathcal{C}^{(i)}$ sampled randomly with probability $g(\mathcal{C})$. The problem is then reduced to find an efficient way to generate configurations $\mathcal{C}^{(i)}$ randomly but with probability $g(\mathcal{C})$. An obvious choice would be to build-up at each time step i a new configuration from scratch, compute its potential energy $E(\mathcal{C}^{(i)})$ and then accept it with probability proportional to $\exp(-\beta E(\mathcal{C}^{(i)}))/Z$. This will generate random configurations with the desired probability g . Moreover the configurations will be uncorrelated (at least to the level in which the pseudo-random algorithm generates uncorrelated numbers). These approaches give rise to the so-called **Static Monte Carlo methods**. These methods are mostly, however, quite inefficient, mainly for two reasons

- The configurations can be very complicated and to generate them from scratch is often a time-consuming procedure (for example: the generation of a self-avoiding walk on a lattice).

- The rejection technique to generate configurations that follow the desired g distribution could be very inefficient because of the possible high rejection rates, especially if one chooses as g the Boltzmann weight.
- To consider $g(\mathcal{C}) \propto \exp(-\beta E(\mathcal{C}^{(i)})) / Z$ requires the knowledge of Z itself that is actually one of the quantities that we would like to estimate....

An alternative method relies on the so-called **dynamic Monte Carlo methods**.

1.2 Dynamic Monte Carlo methods

In this approach, we abandon the idea of sampling points $\mathcal{C}^{(i)}$ in $\{\mathcal{C}\}$ that are fully independent. Instead, the choice of a point $\mathcal{C}$ will depend on the previously (in time) generated point i.e. $\mathcal{C}(t+1) = T[\mathcal{C}(t)]$. This procedure is very convenient especially if the configurations $\mathcal{C}^{(i)}$ are very complicated objects (such as curves, surfaces, etc.). One of the drawbacks of this approach is that it will sample correlated configurations. This “dynamical” procedure should be done in such a way that the long-time sampling points $\mathcal{C}^{(i)}$ are distributed with the target probability π . The idea is then to sample the random configurations $\mathcal{C}^{(i)}$ by performing a stochastic path on the configurational space $\{\mathcal{C}\}$ having π as the equilibrium distribution (see Figure [\[TODO: Fare figura stilizzata\]](#)).

The underlined stochastic dynamics are usually based on the theory of **Markov chains** that we briefly summarise (in a non-exhaustive way) below.

1.3 Markov chains

Roughly speaking a Markov chain is a **Markov process with discrete time steps and in which the random variable ξ can assume values on a discrete space**. To be more precise let us give the following definition of a discrete random process taking discrete values:

Definition. *Discrete random process with discrete values*

Let us consider a sequence of random variables $\xi = \{\xi_1, \xi_2, \dots, \xi_n\}$ taking values in a discrete set S that can be finite or infinite. The elements of S will be called **states**, and, accordingly, S will be referred to as the **state space**. Usually the discrete index t in ξ_t is thought of as a time index if $\xi_n = s_i$ one says that the process is in state s_i at time n . Often the state space $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$ is represented directly by the labels of the elements i.e. $\mathcal{S} = \{1, 2, \dots, n\}$. In this case the condition $\xi_n = s_i$ is replaced by the more synthetic one $\xi_n = i$.

Example. *[Population dynamics]*

Suppose we want to study population growth in a model in which random effects are included. In this case, t might label the generation level, and ξ_t would be the size of the population in generation t . An appropriate state space would be $\{0, 1, 2, \dots\}$ i.e. $\mathcal{S} = \mathbb{Z}^+$.

Example. *[Sequence model]*

Suppose we want to model the appearing of the bases $\{A, G, C, T\}$ as we read a randomly chosen DNA segment starting, say, from the 5' end. In this case, the index t would label the t^{th} base from the 5' end (initial time), and the space $\mathcal{S}$ would be the DNA alphabet $\{A, G, C, T\}$. Hence ξ_t gives the base at site t . Note that in this example t is not really a time index but gives the direction along which the DNA sequence is read.

Definition. *Markov chain*

Let $\xi = \{\xi_1, \xi_2, \dots\}$ a stochastic discrete process in the discrete state space $\mathcal{S}$. This process is called a **Markov chain** if the conditional probability between the outcomes at different times satisfy the **Markov property**

$$\mathbb{P}(\xi_{n+1} = s_{n+1} | \xi_1 = s_1; \xi_2 = s_2; \dots; \xi_n = s_n) = \mathbb{P}(\xi_{n+1} = s_{n+1} | \xi_n = s_n). \quad (1.12)$$

for every sequence $s_1, \dots, s_{n+1}$ of elements of $\mathcal{S}$ and for every $n \geq 1$.

Definition. Transition probability

If $\{\xi_1, \xi_2, \dots\}$ is a Markov chain, and s_i, s_j are states in $\mathcal{S}$, the conditional probability

$$\mathbb{P}(\xi_{n+1} = s_j | \xi_n = s_i) \equiv \mathbb{P}(\xi_{n+1} = j | \xi_n = i) \equiv p_{ij}(n) \quad (1.13)$$

is called the **transition probability from i to j at step n** .

Definition. Transition probability matrix

Let $\{\xi_1, \xi_2, \dots\}$ a Markov chain with a state space $\mathcal{S} = \{1, 2, \dots\}$ that could be infinite but still countable set. One can then associate to $\{\xi_n, n \geq 1\}$ the following **transition matrix**

$$\mathcal{P}(t) = \begin{pmatrix} p_{11}(t) & p_{12}(t) & p_{13}(t) & \cdots \\ p_{21}(t) & p_{22}(t) & p_{23}(t) & \cdots \\ p_{31}(t) & p_{32}(t) & p_{33}(t) & \cdots \\ \cdots & \cdots & \cdots & \cdots \end{pmatrix}. \quad (1.14)$$

Notice that the row $[p_{i1}(t), p_{i2}(t), \dots]$ represents all the transition probabilities out of state i at time t . Therefore, the probabilities in each row must sum to 1 for each fixed value of t .

Definition. Stochastic matrix

A matrix $p_{ij}(t)$ that satisfies the following properties

$$p_{ij}(t) \geq 0, \quad \sum_{j=1}^{\infty} p_{ij}(t) = 1 \quad \forall i \quad \text{and} \quad \forall t = 1, 2, \dots \quad (1.15)$$

is called a **stochastic matrix**.

In the case in which the state space $\mathcal{S}$ is finite, i.e. $\mathcal{S} = \{1, 2, \dots, m\}$ the transition matrix, at discrete time t , is a $m \times m$ matrix, i.e.:

$$\mathcal{P}(t) = \begin{pmatrix} p_{11}(t) & p_{12}(t) & p_{13}(t) & \cdots & p_{1m}(t) \\ p_{21}(t) & p_{22}(t) & p_{23}(t) & \cdots & p_{2m}(t) \\ p_{31}(t) & p_{32}(t) & p_{33}(t) & \cdots & p_{3m}(t) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ p_{m1}(t) & p_{m2}(t) & p_{m3}(t) & \cdots & p_{mm}(t) \end{pmatrix}. \quad (1.16)$$

Definition. Homogeneous Markov chains

If the transition probability $p_{ij}(t)$ does not depend on the time t , i.e., $p_{ij}(t) = p_{ij}$, we say that the Markov chain is **time-homogeneous**. In this course on Monte Carlo, we will consider time-homogeneous Markov chains.

1.3.1 Markov chains as weighted random walks on a digraph

All finite Markov chains can be represented as weighted random walks on a digraph (directed graph), the weight being the transition probabilities. Let us be more precise.

Definition. Graph A **graph** or **undirected graph** G is an ordered pair $G = (V, E)$ that has the following properties:

- V is a set, whose elements are called **vertices** or **nodes**,
- E is a set of pairs (unordered) of distinct vertices, called **edges** or **lines**.

The vertices belonging to an edge are called the **ends**, **endpoints**, or **end vertices** of the edge. V (and hence E) are usually taken to be finite sets, and many well-known results are not true (or rather different) for **infinite graphs** because many of the arguments fail in the infinite case. The **degree** of a vertex is the number of other vertices it is connected to by edges.

Definition. **Directed graph** A **directed graph** or **digraph** G is an ordered pair $G = (V, A)$ with

- V is a set whose elements are called **vertices** or **nodes**,
- A is a set of **ordered** pairs of vertices, called **directed edges**, **arcs** or **arrows**.

An arc $e = (x, y)$ is considered to be directed from x to y ; y is the head and x the tail of the arc. A directed graph is called symmetric if every arc belongs to it together with the corresponding inverted arc.

Let $\mathcal{P} = (p_{ij})$ be an arbitrary (not necessarily stochastic) $m \times m$ matrix. We associate with $\mathcal{P}$ a digraph $G = (V, A)$ as follows. Let $V = [m]$ and $A = \{(i, j) : p_{ij} \neq 0\}$. We label the edge $i \rightarrow j$ with the number $p_{ij} \neq 0$ (the weight of the edges). This definition makes sense for any matrix $\mathcal{P}$ where edges indicate non-zero entries. If $\mathcal{P}$ is the transition matrix of a finite Markov chain $\mathcal{M}$, then we call the associated digraph the **transition digraph** of $\mathcal{M}$. The **vertices** of the transition digraph represent the *states* of $\mathcal{M}$ and the **edges** the **feasible transition**, i.e. transitions that occur with positive probability.

Example. See Figure 1.1 for two examples of transition digraphs and the corresponding stochastic matrix:

Definition. **State probability** From now on let us omit the s and call the denote the i -esim state simply as i . The probability of being in state j at time t is called the **state probability** and is defined as

$$\mu_t(j) \equiv \Pr\{\xi_t = j\}. \quad (1.17)$$

All the state probabilities can be grouped into a **state probability vector**

$$\mu_t = [\mu_t(0), \mu_t(1), \dots, \mu_t(i), \dots] \quad (1.18)$$

1.3.2 Dynamics of the Markov chain

Let the probability vector $\mu_{t=0} \equiv \Pi_0$ be the **initial probability condition**. The Markov property allows us to obtain the probability of any finite sequence $\{\xi_0, \xi_1 \dots \xi_t\}$ in terms of the initial distribution μ_0 . Indeed

$$\mathbb{P}(\xi_n = i_n, \xi_{n-1} = i_{n-1}, \dots, \xi_0 = i_0) = p_{i_n, i_{n-1}} p_{i_{n-1}, i_{n-2}} \dots p_{i_1, i_0} \mu(i_0) \quad (1.19)$$

Moreover,

$$\mathbb{P}(\xi_n = i \mid \xi_0 = j) = \sum_{i_{n-1}, \dots, i_1 \in S} p_{i, i_{n-1}} p_{i_{n-1}, i_{n-2}} \dots p_{i_1, j} \equiv p_{i, j}^{(n)} \quad (1.20)$$

where $p_{i, j}^{(n)}$ is the (i, j) entry of the matrix $\mathcal{P}^n$

Hence, given the initial distribution of the Markov chain μ_0 the distribution after n steps is (vectorial form)

$$\mu_n = \mu_0 \mathcal{P}^n \quad (1.21)$$

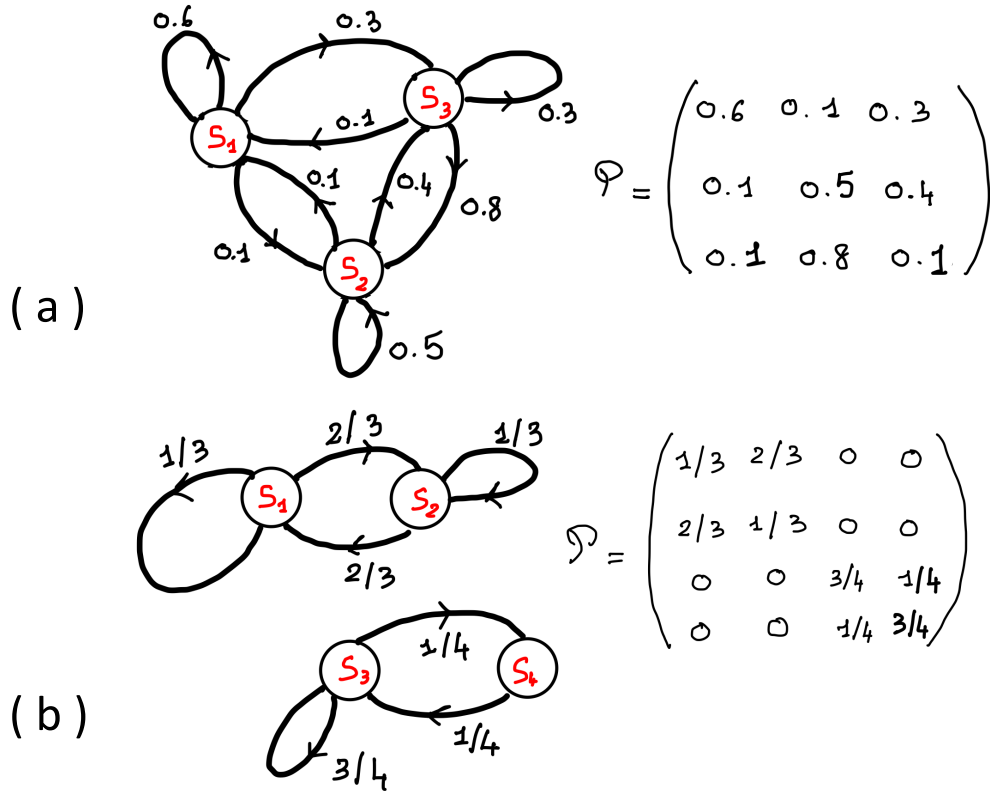


Figure 1.1: Two examples of transition digraphs with the corresponding stochastic matrix defining the Markov chain. (b) Note that in this case the graph is disconnected showing that the chain is not irreducible. For instance state s_3 does communicate with s_4 but does not communicate with either s_1 or s_2 . The two separate digraph, however, are irreducible on the spaces $S_A = \{s_1, s_2\}$ and $S_B = \{s_3, s_4\}$.

Clearly, μ_n satisfies the recurrence relation

$$\mu_n(j) = \sum_{i \in S} \mu_{n-1}(i) p_{i,j} \quad \text{or} \quad \mu_n = \mu_{n-1} \mathcal{P} \quad (1.22)$$

The matrices $\mathcal{P}^n$ satisfy the **Chapman-Kolmogorov equations**:

$$\mathcal{P}^{(l+m)} = \mathcal{P}^l \mathcal{P}^m \quad l, m \geq 1 \quad (1.23)$$

or, in components

$$p_{i,j}^{(l+m)} = \sum_{k \in S} p_{i,k}^{(l)} p_{k,j}^{(m)}, \quad l, m \geq 1 \quad (1.24)$$

From the properties of the conditional probabilities density functions and the Markov chains we have

$$\begin{aligned}
 p_{ij}^{(l+m)} &= \mathbb{P}(\xi_{l+m} = j \mid \xi_0 = i) = \sum_{k \in S} \mathbb{P}(\xi_{l+m} = j, \xi_l = k \mid \xi_0 = i) \\
 &= \sum_{k \in S} \mathbb{P}(\xi_{l+m} = j \mid \xi_l = k, \xi_0 = i) \mathbb{P}(\xi_l = k \mid \xi_0 = i) \\
 &= \sum_{k \in S} \mathbb{P}(\xi_l = k \mid \xi_0 = i) \mathbb{P}(\xi_{l+m} = j \mid \xi_l = k) \\
 &= \sum_{k \in S} p_{ik}^{(l)} p_{kj}^{(m)}
 \end{aligned} \tag{1.25}$$

These equations tell us that to go from state i to state j in $l + m$ steps, at a given intermediate step l the chain will be in one of the other possible states.

1.3.3 Ergodic Markov chain and invariant distribution

Since we are interested in the **long-time** behaviour of a Markov chain, among all possible probability distribution μ a crucial role is played by the **invariant distribution** π i.e. the **fixed-point** of the dynamics described by $\mathcal{P}$, namely

$$\pi \mathcal{P} = \pi. \tag{1.26}$$

Note. If we take the transpose of equation (1.26) we have

$$(\pi \mathcal{P})^T = \pi^T \iff \mathcal{P}^T \pi^T = \pi^T \tag{1.27}$$

In other words, the transposed transition matrix $\mathcal{P}^T$ has eigenvectors with eigenvalues 1 that are stationary distributions expressed as column vectors. One says that the stationary distribution is a **left eigenvector** of the transition matrix $\mathcal{P}$.

The questions we need to answer are the following:

Is there a unique invariant distribution π ?

What are the convergence properties of an initial distribution μ_0 towards π ?

In order to answer these questions we need some additional definitions related to the dynamics of a Markov chain.

1.3.4 Classification of the states under the MC dynamics

For a given state i let us first define the time

$$T_i = \inf\{n \geq 1; \xi_n = i\} \tag{1.28}$$

We can now make the following definition

Definition. *Recurrent and transient states*

A state $i \in S$ is called **recurrent** if $\Pr\{T_i < \infty\} = 1$ and **transient** vice-versa i.e. if $\Pr\{T_i < \infty\} < 1$.

Definition. Communicating states

A state i communicates with j (written $i \rightarrow j$), if $p_{ij}^{(m)} > 0$ for some m . In terms of the associated digraph a state i communicates with j if there is a path from i to j . One can also say that **the state j is accessible from state i** .

In Figure 1.1(b) we show an example of communicating and non-communicating states.

Definition. Irreducible Markov chain

A Markov chain is **irreducible** if all the states in S communicate one to another, i.e. for any pair $(i, j) \in S$, exists a number $n \geq 0$ (i.e. a path) such that the probability $\Pr(\xi_{t+n} = j | \xi_t = i) = (p^n)_{ij}$ is strictly positive.

Definition. Recurrent irreducible Markov chain

A Markov chain is **recurrent irreducible** if it is **irreducible** and all the states are **recurrent**.

Note. The distinction between recurrent and transient states for irreducible chains is only necessary if the space S is infinite. Indeed if S is finite we have the following result: **All irreducible Markov chains defined on a finite space S are recurrent irreducible**. In particular, when S is finite the irreducibility property translates in the linear algebra language as follows:

Definition. Reducible matrix A matrix $\mathcal{P}$ is **reducible** if there exists a permutation matrix $\mathcal{Q}$ such that

$$\mathcal{Q}\mathcal{P}\mathcal{Q}^T = \begin{pmatrix} A_1 & B \\ 0 & A_2 \end{pmatrix} \quad (1.29)$$

If this is not possible the matrix $\mathcal{P}$ is **irreducible**.

If the Markov chain is irreducible a rigorous result on the limiting distribution being the invariant distribution π can be proved. In the case of a finite state space S this is nothing but the **Perron-Frobenius** theorem which states that an irreducible (all elements of $\mathcal{P}$ are strictly positive) Markov chain has the largest eigenvalue λ equal to 1 and this eigenvalue has multiplicity 1 (no degeneracy). Moreover, the corresponding eigenvector is strictly positive.

Theorem. If $\mathcal{P}$ is recurrent irreducible then:

- There exists a unique invariant distribution π that is strictly positive;
- For any initial distribution μ_0

$$\frac{1}{n} \sum_{k=1}^n \chi(\xi_k = j) \rightarrow \pi(j), \quad a.e. \quad (1.30)$$

or

$$\frac{1}{n} \sum_{k=1}^n (p^k)_{i,j} \rightarrow \pi(j), \quad a.e. \quad \forall i, j \in S \quad (1.31)$$

Formally,

$$\mu_n = \mu_0 \bar{\mathcal{P}}_n \rightarrow \pi \quad (1.32)$$

where

$$\bar{\mathcal{P}}_n = \frac{1}{n} \sum_{k=1}^n \mathcal{P}^k \quad (1.33)$$

i.e. the “time average” of $\mathcal{P}^n$. In other words, the average number of times the process is in state i in the first n steps approaches the corresponding value of the stationary distribution $\pi(i)$

Note. The result above states that is the **time average** of $\mathcal{P}^n$ and not $\mathcal{P}^n$ that converges to π .

We would like to know under which conditions we have the stronger result

$$(p^n)_{i,j} \rightarrow \pi(j), \quad \forall i, j \in S \quad (1.34)$$

Let us consider for instance a random walk on $S = \mathbb{Z}/N$ where N is an even integer, i.e. let us consider the Markov chain

$$\xi_n = (\xi_0 + \zeta_1 + \cdots + \zeta_n) \bmod N \quad (1.35)$$

where ζ_i are i.i.d. random numbers in $\{-1, 1\}$. It is possible to show that this Markov chain is irreducible and, being S finite, also recurrent positive. On the other hand $(p^{k+1})_{i,i} = 0$ for each $i \in S$. In the particular case $N = 2$ $\mathcal{P}^{2k} = \mathbf{I}$ while $\mathcal{P}^{2k+1} = \mathcal{P}$. This is an example of a periodic Markov chain where the limit (1.34) is not valid. In order to get result (1.34) we need to exclude **periodic chains**. In figure 1.2 we show two examples of periodic Markov chains. Note that in case (a), for each $n \geq 0$ we have

$$\mathcal{P}^{2n} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathcal{P}^{2n+1} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (1.36)$$

i.e. the sequence p_{ij}^n is oscillating and the limit $\lim_{n \rightarrow \infty} p_{ij}^n$ does not exist. Still, it exists a unique invariant distribution $\pi = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

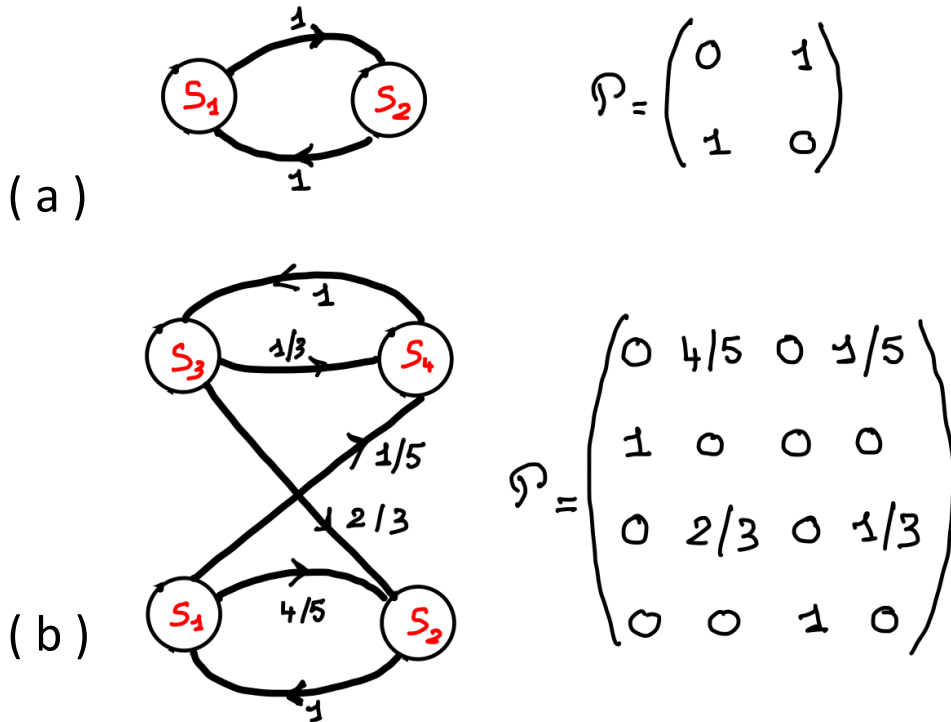


Figure 1.2: Two examples of transition digraphs (and corresponding transition matrix) of periodic Markov chains.

Definition. Aperiodic state

A state $j \in S$ is **aperiodic** if exists a number M such that

$$(\mathcal{P}^n)_{jj} > 0, \quad \forall n \geq M \quad (1.37)$$

An important result for irreducible Markov chains is the following

Theorem. If $\mathcal{P}$ is irreducible and exists a state that is aperiodic then all the other states are aperiodic.

Definition. Irreducible Aperiodic (or regular) Markov chain

An irreducible Markov chain is **aperiodic** if all states are aperiodic (actually it is sufficient to show that one is aperiodic for the results above). A finite Markov chain is called **regular** if it is **irreducible** and **aperiodic**.

An important result that allows us to check whether a Markov chain is **regular is the following**: A Markov chain with a transition matrix $\mathcal{P}$ is regular if and only if there exists an integer $n \geq 0$ such every entry of $(\mathcal{P})^n$ is positive. In this case every entry of $(\mathcal{P})^m$ is positive whenever $m \geq n$ [EO: See exercises on Markov chains]

1.3.5 Asymptotic properties of irreducible aperiodic Markov chains

Theorem. If the Markov chain is **aperiodic** and **irreducible** (i.e. **regular**) then

- There exists a unique invariant distribution π that is strictly positive;
- For any initial probability distribution μ_0

$$\mu_n = \mu_0 \mathcal{P}^n \rightarrow \pi \quad \text{as } n \rightarrow \infty \quad (1.38)$$

Note. If S is infinite but countable, there are two possibilities: either $\pi(j) = 0 \quad \forall j$, or π is the only solution of the fixed point equation.

1.3.6 Averages estimates via Markov chains

The probability distribution π represents the fraction of time that the Markov chain spends in each state (for sufficiently long times). More precisely, let $u_j^{(k)}$ be a function that is 1 if at time k the Markov chain is in j and 0 otherwise. Defining

$$f_j^{(n)} = \frac{1}{n} \sum_{k=1}^n u_j^{(k)}, \quad (1.39)$$

the frequency of passages of the MC through state j , the following extension of the law of large numbers to v.a. that are not independent but are Markov chain sequences holds

Theorem. Law of large numbers for Markov chains

If the Markov chain $\mathcal{P}$ is ergodic

$$\lim_{n \rightarrow \infty} \Pr \left\{ \left| f_j^{(n)} - \pi(j) \right| > \epsilon \right\} = 0, \quad \forall \epsilon > 0 \quad (1.40)$$

independently from the initial distribution π_0 .

In other words, if the Markov chain is **ergodic** and we wait a sufficiently long time, the samples will be distributed according to π . Moreover, to estimate the moments of a given observable O with respect to π it will be sufficient to compute the time averages along the Markov chain. More rigorously we have

Theorem. Ergodic theorem

For a function O defined on the state space S and assuming values in $\mathbb{R}$ and a **ergodic** Markov chain with stationary probability π the limit

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{t=1}^n O(\xi_t) \quad (1.41)$$

converges to the statistical average

$$\sum_{i \in S} O(i) \pi(i) \equiv \langle O \rangle_{\pi} \quad (1.42)$$

with probability 1. Moreover, according to the central limit theorem, the fluctuations around the average $\langle O \rangle_{\pi}$ are of order $1/\sqrt{n}$.

1.4 Markov chain implementation of dynamic Monte Carlo methods

From the theory of Markov chains, we have seen that, in order to generate configurations $\mathcal{C}^{(i)}$ in $\{\mathcal{C}\}$ according to the desired probability distribution π , we have to build up a transition matrix $\mathcal{P} = \{p_{ij}\}$ that satisfies the following conditions:

(A) **Regularity (Irreducible and aperiodic) of the transition matrix $\mathcal{P}$**

(B) **Stationarity of π (π as fixed point of $\mathcal{P}$)** i.e. for each $i \in S$

$$\sum_{j \in \{\mathcal{C}\}} \pi(j) p_{ji} = \pi(i). \quad (1.43)$$

Note. To simplify notations the i -esim state $\mathcal{C}^{(i)}$ is simply denoted by i and the transition matrix probability $Pr\{\mathcal{C}(t+1) = j \mid \mathcal{C}(t) = i\} \equiv p_{ij}$

In many cases instead of condition (B) one looks for a more stringent (but easier to implement) condition. This is the so-called

(C) **Detailed balance condition:** For each pair of states $i, j \in \{\mathcal{C}\}$

$$\pi(j) p_{ji} = \pi(i) p_{ij}. \quad (1.44)$$

By summing relation (1.44) over j one can immediately see that condition (C) implies (B). A Markov chain that satisfies condition (C) is called **reversible** since $\mathbb{P}(\mathcal{C}_t = i, \mathcal{C}_{t+1} = j) = \mathbb{P}(\mathcal{C}_t = j, \mathcal{C}_{t+1} = i)$ at equilibrium.

Theorem. Ergodic theorem

For a function O defined on the state space S and assuming values in $\mathbb{R}$ and a **ergodic** Markov chain with stationary probability π the limit

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{t=1}^n O(\xi_t) \quad (1.41)$$

converges to the statistical average

$$\sum_{i \in S} O(i) \pi(i) \equiv \langle O \rangle_{\pi} \quad (1.42)$$

with probability 1. Moreover, according to the central limit theorem, the fluctuations around the average $\langle O \rangle_{\pi}$ are of order $1/\sqrt{n}$.

1.4 Markov chain implementation of dynamic Monte Carlo methods

From the theory of Markov chains, we have seen that, in order to generate configurations $\mathcal{C}^{(i)}$ in $\{\mathcal{C}\}$ according to the desired probability distribution π , we have to build up a transition matrix $\mathcal{P} = \{p_{ij}\}$ that satisfies the following conditions:

(A) **Regularity (Irreducible and aperiodic) of the transition matrix $\mathcal{P}$**

(B) **Stationarity of π (π as fixed point of $\mathcal{P}$)** i.e. for each $i \in S$

$$\sum_{j \in \{\mathcal{C}\}} \pi(j) p_{ji} = \pi(i). \quad (1.43)$$

Note. To simplify notations the i -esim state $\mathcal{C}^{(i)}$ is simply denoted by i and the transition matrix probability $Pr\{\mathcal{C}(t+1) = j \mid \mathcal{C}(t) = i\} \equiv p_{ij}$

In many cases instead of condition (B) one looks for a more stringent (but easier to implement) condition. This is the so-called

(C) **Detailed balance condition:** For each pair of states $i, j \in \{\mathcal{C}\}$

$$\pi(j) p_{ji} = \pi(i) p_{ij}. \quad (1.44)$$

By summing relation (1.44) over j one can immediately see that condition (C) implies (B). A Markov chain that satisfies condition (C) is called **reversible** since $\mathbb{P}(\mathcal{C}_t = i, \mathcal{C}_{t+1} = j) = \mathbb{P}(\mathcal{C}_t = j, \mathcal{C}_{t+1} = i)$ at equilibrium.

Equation of State Calculations by Fast Computing Machines

NICHOLAS METROPOLIS, ARIANNA W. ROSENBLUTH, MARSHALL N. ROSENBLUTH, AND AUGUSTA H. TELLER,
Los Alamos Scientific Laboratory, Los Alamos, New Mexico

AND

EDWARD TELLER,* *Department of Physics, University of Chicago, Chicago, Illinois*
(Received March 6, 1953)

A general method, suitable for fast computing machines, for investigating such properties as equations of state for substances consisting of interacting individual molecules is described. The method consists of a modified Monte Carlo integration over configuration space. Results for the two-dimensional rigid-sphere system have been obtained on the Los Alamos MANIAC and are presented here. These results are compared to the free volume equation of state and to a four-term virial coefficient expansion.

I. INTRODUCTION

THE purpose of this paper is to describe a general method, suitable for fast electronic computing machines, of calculating the properties of any substance which may be considered as composed of interacting individual molecules. Classical statistics is assumed, only two-body forces are considered, and the potential field of a molecule is assumed spherically symmetric. These are the usual assumptions made in theories of liquids. Subject to the above assumptions, the method is not restricted to any range of temperature or density. This paper will also present results of a preliminary two-dimensional calculation for the rigid-sphere system. Work on the two-dimensional case with a Lennard-Jones potential is in progress and will be reported in a later paper. Also, the problem in three dimensions is being investigated.

* Now at the Radiation Laboratory of the University of California, Livermore, California.

II. THE GENERAL METHOD FOR AN ARBITRARY POTENTIAL BETWEEN THE PARTICLES

In order to reduce the problem to a feasible size for numerical work, we can, of course, consider only a finite number of particles. This number N may be as high as several hundred. Our system consists of a square† containing N particles. In order to minimize the surface effects we suppose the complete substance to be periodic, consisting of many such squares, each square containing N particles in the same configuration. Thus we define d_{AB} , the minimum distance between particles A and B , as the shortest distance between A and any of the particles B , of which there is one in each of the squares which comprise the complete substance. If we have a potential which falls off rapidly with distance, there will be at most one of the distances AB which can make a substantial contribution; hence we need consider only the minimum distance d_{AB} .

† We will use the two-dimensional nomenclature here since it is easier to visualize. The extension to three dimensions is obvious.

1.4.1 Algorithms satisfying detailed balance: Metropolis and Heat bath

While condition (A) (regularity) must be proved for each algorithm, condition (C) can be easily satisfied in general by using the so-called Metropolis filter. Let us see how it works by introducing an arbitrary regular stochastic matrix Γ on $\{C\}$ that we call **matrix of proposed (or trial) moves**. This matrix generates the set of proposed moves $i \rightarrow j$ that will be accepted or rejected according to a **probability matrix** a_{ij}

Note. A rejected move means a null transition i.e. $i \rightarrow i$.

Given these two matrices the transition matrix $\mathcal{P}$ of the Markov chain is given by

$$i \neq j \quad p_{ij} = \Gamma_{ij} a_{ij} \quad (1.45)$$

$$i = j \quad p_{ij} = p_{ii} = 1 - \sum_{j \neq i} p_{ij} = 1 - \sum_{j \neq i} \Gamma_{ij} a_{ij} \quad (1.46)$$

$$= \Gamma_{ii} + \sum_{i \neq j} \Gamma_{ij} (1 - a_{ij}) \quad (1.47)$$

where $0 \leq a_{ij} \leq 1 \quad \forall i, j \in \{C\}$. Note that in the last equality we have used the relation $\sum_{j=1}^N \Gamma_{ij} = 1$.

$$\text{The condition (C)} \quad \pi(i) p_{ij} = \pi(j) p_{ji} \quad (1.48)$$

$$\text{becomes} \quad \pi(i) \Gamma_{ij} a_{ij} = \pi(j) \Gamma_{ji} a_{ji} \quad \text{per } i \neq j. \quad (1.49)$$

With the above algorithm the matrix $\mathbf{P}$ satisfies the detailed balance in Eq. (1.49) if and only if

$$\frac{a_{ij}}{a_{ji}} = \frac{\pi(j)}{\pi(i)} \frac{\Gamma_{ji}}{\Gamma_{ij}} \quad i \neq j. \quad (1.50)$$

It is then sufficient to consider

$$a_{ij} = F \left(\frac{\pi(j) \Gamma_{ji}}{\pi(i) \Gamma_{ij}} \right) \quad \text{where } F : [0, \infty] \rightarrow [0, 1]; \quad \text{and } \frac{F(z)}{F(1/z)} = z \quad (1.51)$$

The most common choices are

Metropolis

$$F(z) = \min(z, 1), \quad (1.52)$$

Heat bath

$$F(z) = \frac{z}{1+z}. \quad (1.53)$$

In several algorithms, the matrix of the proposed moves is chosen to be **reversible** i.e. $\Gamma_{ij} = \Gamma_{ji}$. In these cases, the acceptance probability matrix can be written as

$$a_{ij} = F \left(\frac{\pi(j)}{\pi(i)} \right). \quad (1.54)$$

The above argument works for a generic probability distribution π . In statistical mechanical problems described in the canonical ensemble, we have

$$\pi(i) = \frac{1}{Z} e^{-\beta E(i)} \quad (1.55)$$

and the acceptance probability matrix (1.54) becomes

$$a_{ij} = F \left(e^{-\beta(E(j) - E(i))} \right). \quad (1.56)$$

1.4.2 The Metropolis algorithm

If we choose the Metropolis algorithm eq.(1.56) becomes

$$a_{ij} = \min \left(e^{-\beta \Delta E}, 1 \right) \quad \text{Metropolis.} \quad (1.57)$$

If we know the matrix Γ of the proposed moves a sketch of the Metropolis algorithm is the following:

1. If $i \in \{C\}$ is the configuration at time t , choose j with probability Γ_{ij} ;
2. Let E_i e E_j , the energies respectively of the configuration i (old) and j (new);
3. If $E_j - E_i \leq 0$ the proposed move is accepted and j becomes the new configuration at time $t + 1$;
4. If $E_j - E_i > 0$ the proposed move is accepted with probability $e^{-\beta(E(j)-E(i))}$. This can be done by:
 - 4a Choosing a random number ξ uniformly distributed in $[0, 1]$;
 - 4b if $\xi \leq e^{-\beta(E(j)-E(i))}$ the proposed move is accepted (j as the new configuration) otherwise i is kept also at time $t + 1$ (**null transition**).

1.4.3 Markov chain and Metropolis for the Ising model

Let us recall the Ising model. This is defined by placing N spins $\sigma_i = \pm 1$ on a lattice. A given configuration $\mathcal{C}$ is identified by the set of values $\{\sigma\} = \{\sigma_{r_1}, \sigma_{r_2}, \dots, \sigma_{r_N}\}$.

$$H(\mathcal{C}) = -J \sum_{\langle r_i r_j \rangle} \sigma_{r_i} \sigma_{r_j} \quad \text{with} \quad \sigma_{r_i} = \pm 1 \quad \text{e} \quad \langle r_i r_j \rangle \quad \text{sum over n.n.} \quad (1.58)$$

In order to build a Markov chain according to the Metropolis algorithm we should first define the matrix of the proposed moves, Γ_{ij} , that change the state $\mathcal{C}^{(i)}$ into the state $\mathcal{C}^{(j)}$ (from now on state i and j respectively). The simplest one is a **local** deformation move that proposes to swap the value of a spin σ_{r_k} whose spatial location r_k is chosen randomly among the N locations. This is the so-called **Glauber dynamics**.

Glauber dynamics: a local move that changes the system's magnetisation

Suppose we have chosen a site of the lattice at position r_k . The trail move of the Glauber's dynamic goes from the state $i = \{\sigma_{r_1}, \sigma_{r_2}, \dots, \sigma_{r_k}, \dots, \sigma_{r_N}\}$ to the state $j = \{\sigma_{r_1}, \sigma_{r_2}, \dots, \sigma'_{r_k}, \dots, \sigma_{r_N}\}$ by performing the spin-flip $\sigma'_{r_k} = -\sigma_{r_k}$.

Formally, in terms of the matrix Γ , this means

$$\Gamma_{ij}(r_k) = 1 \quad \text{if} \quad \sigma'(r_k) = -\sigma(r_k) \quad \text{and if} \quad \sigma(r_l)' = \sigma(r_l) \quad \text{for all} \quad r_l \neq r_k \quad (1.59)$$

$$\Gamma_{ij}(r_k) = 0 \quad \text{otherwise.} \quad (1.60)$$

One can show that $\Gamma(r_k)$ is irreducible and symmetric (**see list of exercises**). We can then use the Metropolis filter as

$$a_{ij}(r_k) = \min \left(e^{-\beta \Delta E}, 1 \right), \quad (1.61)$$

where

$$\Delta E \equiv H(\mathcal{C}') - H(\mathcal{C}) = 2 \sigma_{r_k} J \sum_l \sigma_{r_l} \quad (1.62)$$

where the sum is extended just to the nearest neighbours of σ_{r_k} (**local move**) and we have put $J = 1$ for simplicity. If $\Delta E < 0$ accept the proposed move otherwise generate a random number $R \in [0, 1]$ and if $R < \exp(-\beta \Delta E)$ accept the move otherwise $\sigma'_{r_k} = \sigma_{r_k}$ (null move, namely $\mathcal{C}' = \mathcal{C}$).

1.4 Markov chain implementation of dynamic Monte Carlo methods

Note. The resulting transition matrix $p_{ij} = \Gamma_{ij} a_{ij}$ refers to a single site r_k of the lattice, i.e. $p_{ij} = p_{ij}(r_k)$. The whole algorithm (equivalent to a single Monte Carlo step) implies the spanning of all the sites in the lattice. So a **unit time MC step** requires a $O(N)$ of local (Glauber) proposed moves.

The $O(N)$ spanning of the spins in the system can be done either **randomly** or **sequentially** (deterministic) along the lattice.

These two possibilities give rise to two different transition matrices:

Random spanning The sites of the lattice are chosen randomly. The transition matrix is given by

$$\mathcal{P} = \frac{1}{N} \sum_{r_k} \mathcal{P}(r_k). \quad (1.63)$$

Sequential spanning In this case we can span the sites in a type-writer fashion and the transition matrix will be given by

$$\mathcal{P} = \mathcal{P}(r_1) \mathcal{P}(r_2) \cdots \mathcal{P}(r_N). \quad (1.64)$$

Note. In this case the transition matrix $\mathcal{P}$ does not satisfy the detailed balance condition (C) but the more general stationarity condition (B).

Kawasaki dynamics: a local move that keeps the system's magnetisation constant

Another algorithm to build the matrix of the proposed moves in Ising-like models is based on the so-called **Kawasaki dynamic**. The idea is to choose a pair of sites (r_k, r_m) . They do not need to be nearest neighbours (but in general they are) but they have to be **anti-parallel** i.e. $\sigma_{r_k} = -\sigma_{r_m}$. The proposed move consists in interchanging the values of the two spins i.e.

$$i = \{\sigma_{r_1}, \dots, \sigma_{r_k}, \dots, \sigma_{r_m}, \dots, \sigma_{r_N}\} \rightarrow j = \{\sigma_{r_1}, \dots, \sigma_{r_m}, \dots, \sigma_{r_k}, \dots, \sigma_{r_N}\} \quad (1.65)$$

The trial matrix is then

$$\begin{aligned} \Gamma_{ij}(r_k, r_m) &= 1 & \text{if} & & \sigma'_{r_k} = \sigma_{r_m}, \sigma'_{r_m} = \sigma_{r_k} & \text{and} & \sigma'_{r_l} = \sigma_{r_l} \quad \forall r_l \neq r_k, r_m \\ \Gamma_{ij}(r_k, r_m) &= 0 & \text{otherwise} & & & & \end{aligned} \quad (1.66)$$

Note: this algorithm is not irreducible on all the state space $\{\mathcal{C}\}$ since the proposed move conserves the total magnetization $M = \sum_{r_k} \sigma(r_k)$. It is however easy to see that it is irreducible within each subset S_M with fixed total magnetization M .

1.4.4 Markov chain and Heat bath for the Ising model

Let us now see how the Monte Carlo algorithm changes if one considers the heat bath filter for a_{ij} instead then the Metropolis one. Given a chosen site r_k , the probability distribution of the spin σ_{r_k} conditioned to the presence of the other spins in the lattice $\{\sigma_{r_l}\}$ with $r_l \neq r_k$ is given by:

$$\mathbb{P}(\sigma_{r_k} | \{\sigma_{r_l}\}_{r_l \neq r_k}) = \text{const}(\{\sigma_{r_l}\}_{r_l \neq r_k}) e^{\beta \sigma_{r_k} \sum_{r_l} \sigma_{r_l}} \quad (1.67)$$

where σ_{r_l} are the nearest neighbours of r_k .

Note. The conditional probability (1.67) is equivalent to the probability of σ_{r_k} when placed in an effective magnetic field produced by the other spins. In the case of nearest neighbours interaction, these are the neighbouring spins of σ_{r_k} .

The new value of the spin σ_{r_k} , $\sigma'(r_k)$, can be chosen according to the conditional probability (1.67). The algorithm goes as follows:

1. Defining $A = \beta \sum_{r_l} \sigma_{r_l}$ we have

$$\begin{aligned} \mathbb{P}(\sigma_{r_k} = +1 \mid \{\sigma_{r_l}\}_{r_l \neq r_k}) &= c e^A \\ \mathbb{P}(\sigma_{r_k} = -1 \mid \{\sigma_{r_l}\}_{r_l \neq r_k}) &= c e^{-A}; \end{aligned} \quad (1.68)$$

2. Let $p_+ = \frac{e^A}{e^A + e^{-A}} = \frac{1}{1 + e^{-2A}}$;
3. Choose a random number ξ in $[0, 1]$;
4. If $p_+ > \xi$ let $\sigma'_{r_k} = 1$, otherwise $\sigma'_{r_k} = -1$.

1.5 Equilibration

The theory of Markov chains tells us that if ergodicity and stationarity (detailed balance) are satisfied, the dynamics will eventually converge to the desired distribution π . It does not tell you, however, **when** this will occur. Given that the initial configuration is taken at random it could take a while before the MC will start to sample from π . Figure 1.3 shows different time snapshots of an MC that should simulate a 2D Ising model at low temperature (ferromagnetic phase) starting from a random initial condition (top left panel). During simulation, larger and larger domains with aligned spins are formed and grow reaching sizes that from a given time on won't grow anymore. It is clear that before taking measures from the sampled configurations we should be confident that the MC is sampling from the correct equilibrium distribution. Therefore we need to estimate the **equilibration time** τ_{eq} , namely the time that the MC dynamics needs to reach equilibrium.

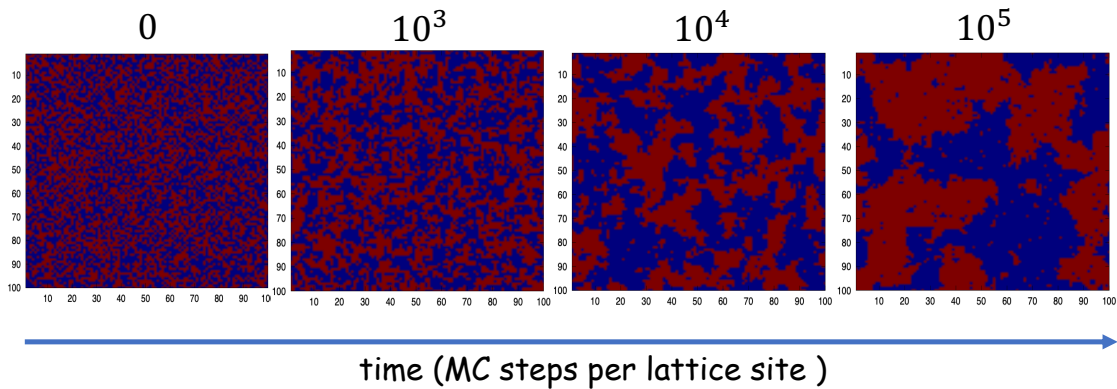


Figure 1.3: Configurations of a 100×100 square lattice Ising model simulated at $k_B T/J = 0.88 k_B T_C/J$. Different panels refer to different sampling times, starting from the initial (random) condition.

How can we estimate τ_{eq} ?

One can, for instance, compute some observables O along the trajectory (such as the magnetisation M , the energy E etc.) and plot them as a function of time (see Figure 1.4). By visual inspection one can see that eventually the observable will fluctuate around a constant value. At that time we can say that the MC has reached its equilibrium (stationarity). In the specific example of Figure 1.4, we consider an MC that in equilibrium should sample equilibrium states at low temperature i.e. in the ferromagnetic phase. One can see from the plots of m and E vs time that, although the starting configuration is random, the dynamic eventually reaches the desired value and from there starts to fluctuate around a constant value. From both observables, we can readily see that $\tau_{eq} \sim 6000$ MC steps.

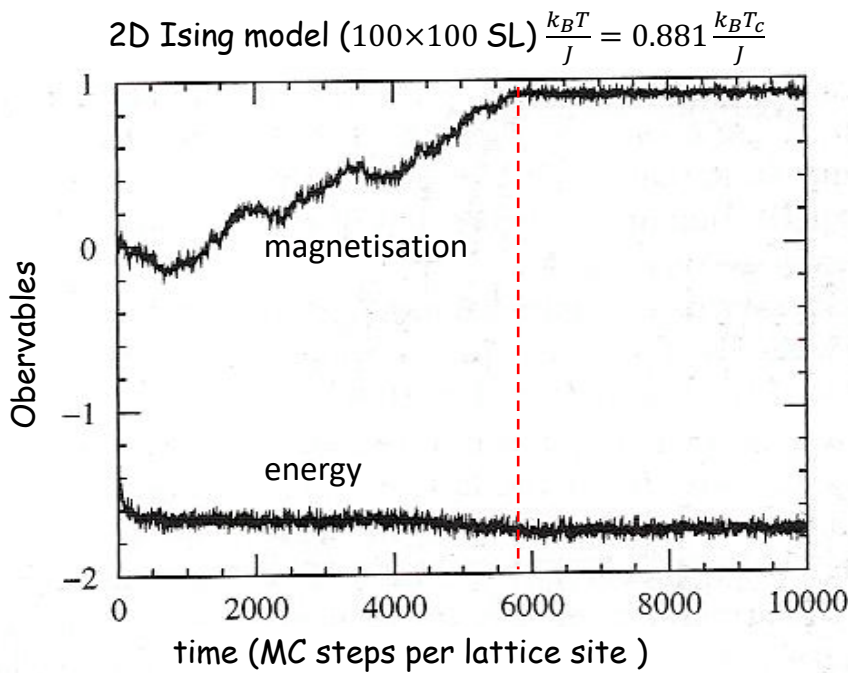


Figure 1.4: Time dependence of the magnetisation (top curve) and internal energy E (bottom curve) per site of a 100×100 (square lattice) Ising model with $J = 1$. The initial condition is a fully random configuration i.e. a state that is probable at $T = \infty$. The target π is the Boltzmann distribution $\exp(-H/k_B T)/Z$ with $k_B T/J = 0.881 k_B T_c/J$. Time is in units of Monte Carlo steps per lattice site. (Adapted from [3])

Sometimes (usually in more complex systems than 2D Ising or when the system size L is very large) the system can get trapped into a metastable state having the same properties as the equilibrium. This is due to the presence of **local minima** in the energy landscape, for instance.

How we can check for metastable states?

In general, this could be a complex task to achieve. A possible, very simple, solution consists of performing two or more simulations **starting from different initial conditions** (for instance in the Ising model could be a +/- ordered configuration and a fully random one) and seeing whether the observables of interest, after a

transient (**equilibration time**) begin to fluctuate around a common value. In Figure 1.5 we report two trajectories of the magnetisation for a 100×100 Ising model (square lattice) where the target $\pi = \exp(-H/k_B T)/Z$ with $k_B T/J = 0.98 k_B T_c/J$. We notice that starting from two different random conditions both trajectories converge ($\tau_{eq} \sim 4000$ MC steps per site) to the same stationary state; this is a good indication that this state is the equilibrium one. Of course, one should test the MC with other and more different initial conditions to be more confident in the result.

Note. The inset of Figure 1.5 reports a simulation of a single MC simulated at a lower temperature ($k_B T/J = 0.833 k_B T_c/J$, i.e. farther away from the critical point). In this case, two things are worth noticing:

- the equilibrium time $\tau_{eq} \approx 1000$ is much smaller than the case reported in the main plot;
- fluctuations around the average magnetisation are also much smaller.

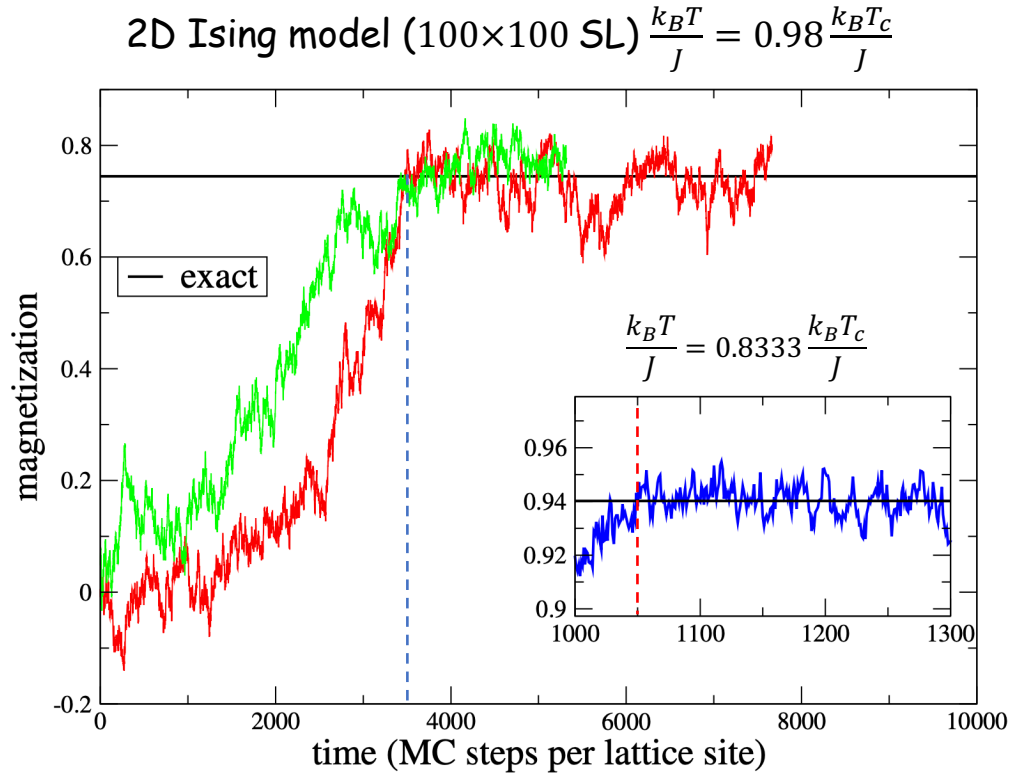


Figure 1.5: Magnetisation of a 100×100 Ising model in the ferromagnetic phase as a function of the simulation time. In the main panel, the Ising was simulated at $k_B T/J = 0.98 k_B T_c/J$. By starting from two different random initial conditions the MC dynamics converge to the same stationary state at $\tau_{eq} \sim 4000$ MC steps per site. Note the solid line representing the exact value of the magnetisation in the thermodynamic limit. The inset shows the dynamics of a single MC simulated at $k_B T/J = 0.833 k_B T_c/J$. Note the faster convergence and the decreases in the fluctuation's amplitudes.

1.6 Monte Carlo Averages and error bars at equilibrium

According to the theory of statistical mechanics, the required macroscopic properties are obtained by averaging over the ensemble sampled according to the algorithm previously described. For instance, the energy will be given by

$$E(\mathcal{C}) = -J \sum_{\langle i,j \rangle} \sigma_i(\mathcal{C}) \sigma_j(\mathcal{C}) \quad (1.69)$$

when the system is the configuration $\mathcal{C}$. Similarly the magnetisation per spin in the microstate $\mathcal{C}$ is given by

$$M(\mathcal{C}) = \sum_i \sigma_i(\mathcal{C}), \quad (1.70)$$

The estimate of these and other observables are obtained as averages over the configurations $\mathcal{C}$ generated by the Markov Chain. More precisely, let $O_t \equiv O(\mathcal{C}_t)$ be the value of the observable O when the system is in the microstate $\mathcal{C}_t$. Denoting by $\{O_t\}$ the time sequence of values of O generated by the MC dynamics, **after the system is equilibrated** (i.e. after τ_{eq}). The average of O is given by

$$\bar{O}_{t_{max}} = \frac{1}{t_{max} - \tau_{eq}} \sum_{t=\tau_{eq}}^{t_{max}} O_t; \quad (1.71)$$

which, in the limit $t_{max} \rightarrow \infty$ gives **(ergodicity theorem of the Markov chain)**

$$\lim_{t_{max} \rightarrow \infty} \bar{O} = \langle O \rangle \quad (1.72)$$

One can also look at the fluctuations of the observable O by estimating its second central moment i.e. the quantity

$$\langle O^2 \rangle - \langle O \rangle^2 \quad (1.73)$$

For instance, the specific heat is given by

$$C_V = \frac{\partial E}{\partial T} = \frac{\langle E^2 \rangle - \langle E \rangle^2}{k_B T^2}, \quad (1.74)$$

while the magnetic susceptibility

$$N\chi_T = \frac{\partial M}{\partial H} = \beta \frac{\langle M^2 \rangle - \langle M \rangle^2}{N} = \beta N (\langle m^2 \rangle - \langle m \rangle^2) \quad (1.75)$$

Error bars

The Monte Carlo estimate $\bar{O}_{t_{max}}$ is always associated with statistical errors since t_{max} is actually finite. On the other hand, we expect that in the limit $t_{max} \rightarrow \infty$ the random variable $\bar{O}_{t_{max}}$ becomes Gaussian distributed with mean $\langle O \rangle$ and variance $\sigma^2(O)/t_{max}$ (central limit theorem). The quantity $\sigma(O)/\sqrt{t_{max}}$ is called the **statistical error** or **one-sigma confidence interval**. In summary, we expect

$$\bar{O}_{t_{max}} \pm \frac{\sigma(O)}{\sqrt{t_{max}}} \quad (1.76)$$

1.7 Sampling uncorrelated stats: correlation time.

where $\sigma^2(O) = \langle O^2 \rangle - \langle O \rangle^2$ whose MC estimate is given by

$$S_{t_{max}}^2 = \frac{1}{(t_{max} - \tau_{eq} - 1)} \sum_{t=\tau_{eq}}^{t_{max}} (O_t - \bar{O}_{t_{max}})^2. \quad (1.77)$$

Note. Asymptotically the statistical error goes to zero as $t_{max}^{-1/2}$.

This is quite a slow convergence. Indeed, this means that to get estimates with an extra decimal accuracy we need to increase the length of the sampling set t_{max} by a factor of one hundred! Actually, things are even worse because the configurations $C_{\square}$ sampled with a dynamical Monte Carlo are **correlated** and this will increase the error bars. Let us see how in the next section

1.7 Sampling uncorrelated stats: correlation time.

Let us focus on the Metropolis algorithm applied to the Ising model. In the proposed local moves of the algorithm at most one spin is flipped at each time step. Consequently, observables such as the magnetisation for a lattice with N spins would change by a factor of $\mathcal{O}(1/N)$. The obvious consequence is that configurations generated by the MC are correlated. This property can be measured by looking at the **autocorrelation function** of the chosen observable O :

$$C_O(t) = \int dt' [O(t') - \langle O \rangle][O(t+t') - \langle O \rangle] = \int dt' [O(t')O(t+t') - \langle O \rangle^2]. \quad (1.78)$$

Notice that for very small values of t , $O(t') + t \approx O(t')$ and the integral will have a positive value. We can then say that $C_O(t) > 0$ if t is small enough (the two sampled values of O refer to two microstates that are very close in the MC dynamics). On the other hand, for sufficiently long times $C_O(t)$ is expected to decay to zero since the sampled configurations are very far apart to be correlated. So, if $O(t') > \langle O \rangle$ (positive fluctuation) we expect an equally likely value $O(t+t')$ above or below its average at times $t' + t$. When we integrated ver t' this should give a zero value for $C(t)$. Indeed, it is possible to show that $C_O(t)$ goes to zero as

$$C_O(t) \sim e^{-t/\tau_{int}^O} \quad (1.79)$$

The characteristic time τ_{int}^O is known as the **integrated autocorrelation time** of the observable O .

Note. Most of the time we are dealing with trajectories that are discrete in time. In this case, the integral is replaced by the temporal series

$$C_O(t) = \frac{1}{t_{max} - t} \sum_{t'=0}^{t_{max}-t} O(t')O(t'+t) - \frac{1}{t_{max} - t} \sum_{t'=0}^{t_{max}-t} O(t') \times \frac{1}{t_{max} - t} \sum_{t'=0}^{t_{max}-t} O(t'+t)$$

Estimate of τ_{int}^O : first method

A first, very straightforward, way to estimate τ_{int}^O is to compute the autocorrelation time $C(t)$ of the trajectory O_t fit it with an exponential and estimate τ_{int}^O as 2τ where τ is the time at which $C(t)$ is $1/e$. An example is reported in Figure 1.6 where the observable is the magnetisation per spin m .

1.7 Sampling uncorrelated stats: correlation time.

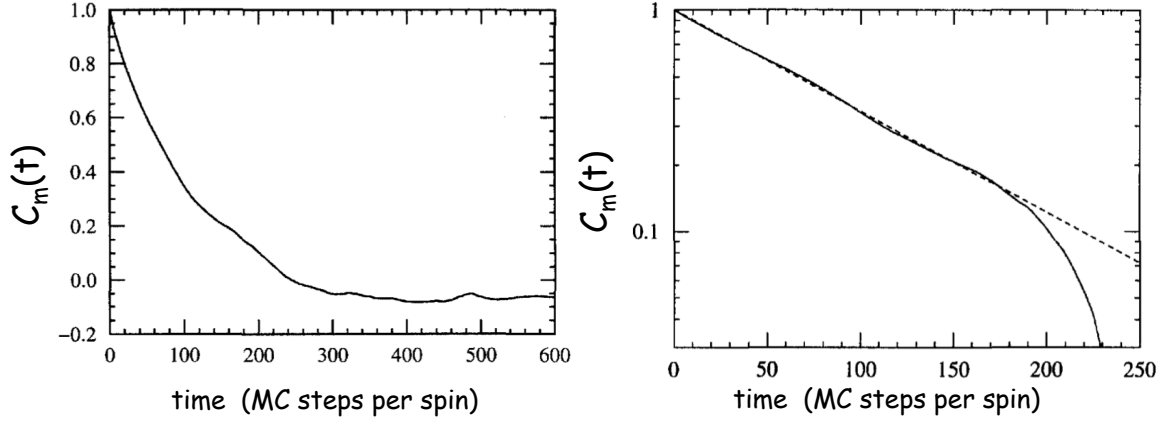


Figure 1.6: (left panel) Correlation function of the magnetisation for the 2D Ising model at $k_B T/J = 1.06 k_B T_c/J$. (right panel) Same as the left panel but now in a semi-logarithmic scale. The dashed straight line has slope $\tau = 95 \pm 5$ (MC steps per site) (Adapted from [3])

Given τ_{int}^O the number of measures of O that can be considered sufficiently uncorrelated is given by

$$n_{uncor} = \frac{t_{max}}{\tau_{int}^O}. \quad (1.80)$$

Note. The value of τ_{int}^O could depend largely on the chosen O . So if we are interested in many observables, the correct sampling interval time will be given by

$$\tau_{int} = \max_O \{\tau_{int}^O\} \quad (1.81)$$

Estimate of τ_{int}^O : second method

An alternative and more robust way to estimate τ_{int}^O is based on the equation

$$\int_0^\infty \frac{C_O(t)}{C_O(0)} dt = \int_0^\infty e^{-t/\tau} dt = \tau \quad (1.82)$$

The advantages of this method over the previous one are the following:

- It is easier (algorithmically) to compute than performing a fit;
- It is less sensible to the truncation of the series.

1.7.1 Correlation time and error bars

Ideally, the estimates of the average and fluctuation of a given observable O should be performed by using configurations that are sampled at time intervals longer than τ_{int}^O . In practice, this is not always possible (especially if the size N of the system is very large) because it would reduce sensibly the number of points at our disposal. Usually, one considers configurations sampled at time intervals Δt and estimates the error bars using the information of τ_{int}^O . For uncorrelated data, we have seen that the estimate of the variance on the mean of $t_{max} - \tau_{eq}$ points is given by

$$S_{t_{max}}^2 = \frac{1}{(t_{max} - \tau_{eq} - 1)} \sum_{t=\tau_{eq}}^{t_{max}} (O_t - \bar{O}_{t_{max}})^2. \quad (1.83)$$

If, on the other hand, we sample data over intervals $\Delta t < \tau_{int}^O$, it is possible to show that the correct estimate is given by

$$S_{t_{max}}^2 = \frac{1 + 2\tau/\Delta t}{(t_{max} - \tau_{eq} - 1)} \sum_{t=\tau_{eq}}^{t_{max}} (O_t - \bar{O}_{t_{max}})^2. \quad (1.84)$$

which indeed reduces to eq. (1.83) in the limit $\Delta t \gg \tau$.

1.8 Critical slowing down

The class of dynamic Monte Carlo algorithms defined so far are all **single-flip** algorithms, i.e. based on **local proposed moves**. These algorithms produce correlated data with an autocorrelation time τ a measure of the number of MC steps one needs to skip to get a sequence of sampled microstates that are uncorrelated after the system has equilibrated. These algorithms perform rather well away from a phase transition. However, especially near a critical point, the autocorrelation time goes like

$$\tau \sim \min(L, \xi)^z, \quad (1.85)$$

where L is the linear size of the system, ξ the **correlation length** of an infinite volume system at the same temperature and z is a **dynamical critical exponent**. This phenomenon is known as the **critical slowing down** and can hamper the simulations of a system in the proximity of its critical point by a dynamical Monte Carlo algorithm based on local moves. For instance, in the Ising model, we know that the two points spin connected correlation function behaves, away from T_c as

$$\langle s(\vec{r} + \delta\vec{r})s(\vec{r}) \rangle - \langle s(\vec{r} + \delta\vec{r}) \rangle \langle s(\vec{r}) \rangle \sim \exp(-|\Delta\vec{r}|/\xi) \quad (1.86)$$

where ξ is the above-mentioned correlation length. On the other hand, as $T \rightarrow T_c$ either from the left or the right this quantity diverges as

$$\xi \sim \left| \frac{T - T_c}{T_c} \right|^{-\nu} \quad (1.87)$$

The value of ν depends on the dimensionality of the system (and on the symmetry of the Hamiltonian that for an Ising model is a $\mathbb{Z}^2$ symmetry). For the 2D Ising model $\nu = 1$. Hence, from eq.(1.85) we should have

$$\tau \sim \min \left(L, \left| \frac{T - T_c}{T_c} \right|^{-1} \right)^z, \quad \text{as } T \rightarrow T_c. \quad (1.88)$$

What is the reason why local algorithms experience a critical slowing down?

The reason is that the proposed moves are local: in a single algorithm step, information is transmitted from a given site only to its nearest neighbours. If, for simplicity, we assume that this information diffuses around, i.e. it performs a random walk on the lattice, in order for the system to evolve into an effective new microstate (that is uncorrelated from the previous one) it is necessary that it travels a distance of order ξ . With this simple argument, one would guess that $\tau \sim \xi^2$ i.e. that $z = 2$. This estimate is not fully correct but is rather close to its real value for the local algorithms considered for spin systems.

How can we estimate z ?

In the likely event in which we know the exact location of T_c (as, for instance, in the case of 2D Ising model) we set the temperature T of the simulation at T_c . We next notice that, for a finite system of size L , the correlation length ξ can be at most as large as L and practically $\xi \sim L$. Hence from eqs.(1.85) and (1.88) we obtain

$$\tau \sim \xi^z \sim L^z \quad (1.89)$$

The last equation suggests that, in order to get an estimate of z , one should perform simulations at $T = T_c$ at different values of L and extrapolate the behaviour of τ as $L \rightarrow \infty$. Unfortunately most of the time, T_c is not known and one has to estimate it first or at once with z . In general, obtaining accurate estimates of z is very hard requiring an enormous amount of MC data, of the order of $\approx 10^4 \tau$, and τ is getting larger and larger as $T \rightarrow T_c$!.

To get an idea of how much computing effort one has to invest in order to study a system like an d dimensional Ising model in the proximity of its critical point let us use the following argument: Each sweep of the lattice requires a time of order L^d and we need $\approx 2\tau$ sweeps in order to get **one effective uncorrelated sample**. This amounts to a total computer time of the order of $L^d \xi^z \approx \xi^{d+z}$. On the other hand, to get high-precision statistics one should aim at sampling $\approx 10^6$ uncorrelated microstates. For $L = 100$, $d = 3$, and $z = 2$ one can see that runs become extremely long!

Note. While the factor ξ^d is inherent in **all** Monte Carlo algorithms the term ξ^z could be either reduced or ideally eliminated tout court by more clever algorithms than those based on local moves. From what we have seen previously the slow modes of the dynamics are the long-wavelength modes is the updating is purely local. The idea is to speed up these modes by some sort of **collective or nonlocal proposed moves** by **computationally efficient algorithms**. Both conditions are usually very difficult to satisfy but in literature have been designed very efficient non-local algorithms that can solve (or at least mitigate) the critical slowing down problem (see part on non-local algorithms)

1.9 Estimates of critical exponents: finite-size scaling analysis

The idea of estimating the equilibrium critical behaviour of a system by Monte Carlo simulations is somehow in contradiction with the fact that **a finite system cannot exhibit phase transitions**. To fix the idea let us consider the 2D Ising model embedded on a $L \times L$ square lattice. If T is far from the expected critical temperature T_c we know that $\xi(T) \ll L$; in this case, a finite system of size L is a good approximation of an infinite one. On the other hand, if $T \approx T_c$, $\xi \approx L$ i.e. is finite and no divergence can be observed. For instance, although the specific heat of the system should diverge as $T \rightarrow T_c$, from simulations at L finite we would observe rounded peaks at best, instead of a sharp divergence. Of course, as L increases we expect these peaks to become sharper and sharper and their heights steadily increase. For a finite system of length L we have

$$\xi(T) \underset{T \rightarrow T_c}{\sim} |T - T_c|^{-\nu} = L \quad (1.90)$$

From eq.(1.90) we get $|T - T_c| \sim L^{-1/\nu}$ and the critical behaviour of observables such as the magnetisation,

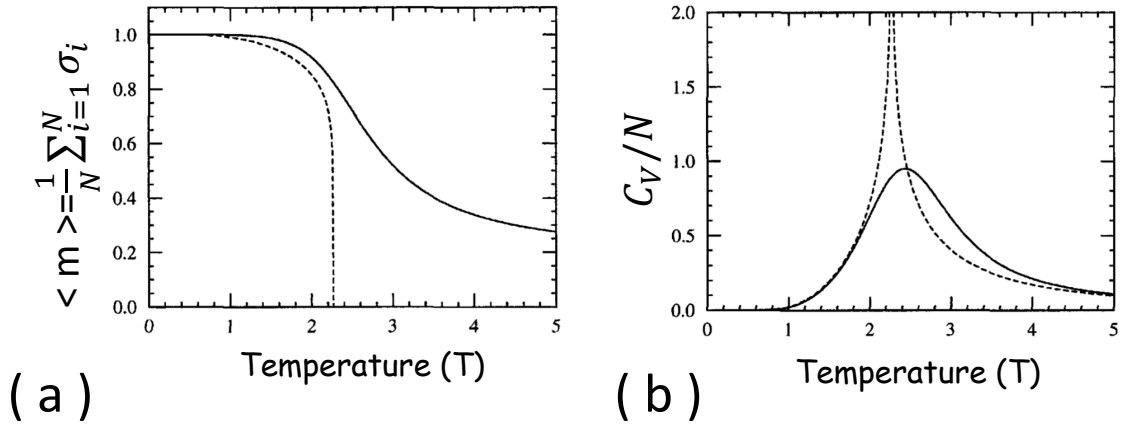


Figure 1.7: (a) Estimate of the magnetisation per spin vs temperature for the Ising model on the square lattice with $L = 5$. The dashed curve is the corresponding exact curve (i.e. in the thermodynamic limit) (b) Estimate of the specific heat per spin for the same system as in (a). The dashed curve is the exact solution. (Adapted from [3])

the specific heat, and the magnetic susceptibility can be expressed as a function of L as follows

$$\frac{M(T)}{V} \underset{T \rightarrow T_c}{\sim} (T_c - T)^\beta \underset{L \rightarrow \infty}{\sim} L^{-\beta/\nu} \quad (1.91)$$

$$\frac{\chi(T)}{V} \underset{T \rightarrow T_c}{\sim} |T - T_c|^{-\gamma} \underset{L \rightarrow \infty}{\sim} L^{\gamma/\nu} \quad (1.92)$$

$$\frac{C_V(T)}{V} \underset{T \rightarrow T_c}{\sim} |T - T_c|^{-\alpha} \underset{L \rightarrow \infty}{\sim} L^{\alpha/\nu} \quad (1.93)$$

$$|T_c(L) - T_c| \underset{L \rightarrow \infty}{\sim} L^{1/\nu} \quad (1.94)$$

The finite-size scaling equations (1.94) can be implemented in MC simulations by the following procedure:

- For a given L one estimates the value of T at which the specific heat displays its maximum (if any).
- Call the value of T at the maximum, $T_c(L)$.
- Estimate for instance $M(T)/V$ at $T = T_c(L)$ for different values of L .
- Plot $\ln[M(T)/V]$ against $\ln(L)$. The slope of the best fit straight line should provide an estimate of $-\beta/\nu$.

1.10 The Wolff algorithm

In 1989, U. Wolff introduced an algorithm that solves the problem of critical slowing down in simulations of the Ising model. His algorithm flips spins in a randomly generated cluster rather than applying single spin flips [3].

In a move of the Wolff algorithm, a cluster is initialized by including randomly picked seed spin. Then, neighboring spins with the same sign of the seed are added to the cluster with probability P_{add} , or neglected with probability $1 - P_{\text{add}}$. The procedure is then iterated for every added spin at the boundary of the cluster, again including new neighbors with probability P_{add} . Note that the addition of a given spin might be attempted multiple times if approached from the site of two or more of its neighbors.

To grant the fulfillment of the detailed balance condition, we need to tune the proposal Γ_{ij} and the acceptance probability a_{ij} , which enter in the probability $p_{ij} = \Gamma_{ij}a_{ij}$ of performing the move $i \rightarrow j$. Let us consider two states i and j as in Figure 1.9.

Their different spins are in the cluster highlighted by the dashed perimeter. Apart from common prefactors, the proposal probabilities are $\Gamma_{ij} \sim (1 - P_{\text{add}})^{b_i}$ and $\Gamma_{ji} \sim (1 - P_{\text{add}})^{b_j}$, where b_i and b_j count the bonds between a spin in the cluster and an equal spin outside the cluster (see the example in Figure 1.9).

For states with energies $E(i)$ and $E(j)$, in a canonical ensemble at inverse temperature β , the detailed balance condition reads

$$\frac{p_{ij}}{p_{ji}} = \frac{\Gamma_{ij}a_{ij}}{\Gamma_{ji}a_{ji}} = (1 - P_{\text{add}})^{b_i - b_j} \frac{a_{ij}}{a_{ji}} = e^{-\beta(E(i) - E(j))} \quad (1.95)$$

For the Ising model with coupling J , the energy difference is $E(i) - E(j) = 2J(b_i - b_j)$ and we can rewrite the detailed balance condition as

$$\frac{a_{ij}}{a_{ji}} = \left[(1 - P_{\text{add}}) e^{2\beta J} \right]^{b_j - b_i} \quad (1.96)$$

The ratio of acceptances can be made equal to one by choosing

$$P_{\text{add}} = 1 - e^{-2\beta J} \quad (1.97)$$

This wise choice, in practice, allows us to accept every proposed cluster flip by setting $a_{ij} = a_{ji} = 1$.

Equation (1.97) implies that spins are added to the cluster with a temperature-dependent probability. Since P_{add} raises with β , so does the average cluster size $\mathcal{S}(\beta)$. At high temperatures, $\beta \rightarrow 0$ leads to $P_{\text{add}} \rightarrow 0$. Hence, clusters are very small, comprising typically only the seed spin and the Wolff algorithm effectively becomes a single spin flipping, $\mathcal{S}(\beta \rightarrow 0) \gtrsim 1$. In this case, a simple Metropolis single-spin flip works better because it performs the same job with a slimmer algorithm. In the opposite limit of very low temperature, $P_{\text{add}} \rightarrow 1$ because $\beta \rightarrow \infty$. Thus, clusters may easily become as large as the whole lattice ($\mathcal{S}(\beta \gg \beta_c) \lesssim L^d$) and the Wolff algorithm always flips almost all spins from say positive to negative magnetization, or vice versa.

The efficiency of the Wolff algorithm in updating Ising model states with relatively uncorrelated ones allows one to sample efficiently and close to the critical point. However, for a fair comparison with the single

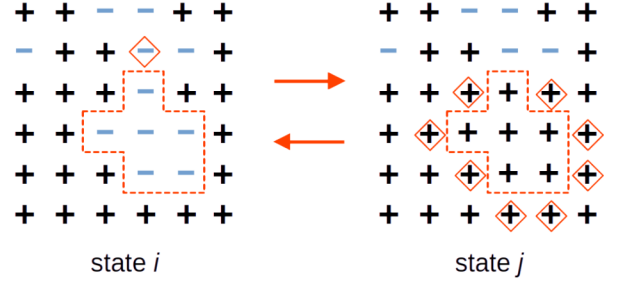


Figure 1.9: Example of cluster flip in the Wolff algorithm for the Ising model. The boundary spins with equal signs are highlighted with diamonds. Their number is $b_i = 1$ and $b_j = 8$.

spin flip algorithm, "time" must be rescaled properly. On average, a single Wolff cluster step takes more CPU time than a single-spin flip. A typical Wolff step is deemed to correspond in time to $\mathcal{S}(\beta)$ single spin flip steps. As usual, the time unit is that of a sweep that tries an update, stochastically, of L^d spins. In these units, the correlation time τ_{data} of a series of data recorded at every spin flip attempt is converted to an autocorrelation time $\tau = \tau_{\text{data}}/L^d$. We recall that at the critical point, τ scales as a power-law of the lattice size

$$\tau \sim \xi^z \sim L^z \quad (1.98)$$

For consistency with what we said, for the Wolff algorithm, τ becomes

$$\tau \sim \tau_{\text{data}} \frac{\mathcal{S}(\beta)}{L^d} \sim \xi^{z_W} \sim L^{z_W} \quad (1.99)$$

with $z_W \approx 0.25$ for the two-dimensional Ising model, which shows that the Wolff algorithm solves the problem of the critical slowing down. We stress that this τ corresponds to the autocorrelation time computed for a series of data recorded per Wolff step (τ_{data}), multiplied by $\mathcal{S}(\beta)/L^d$.

1.11 Multiple Markov Chains

In simulations of several systems, there might appear the phenomenon of "quasi-ergodicity": a small portion of the phase space is explored but never abandoned to reach other relevant portions because the intermediate states are very unlikely. Simulations plagued by this problem may yield long correlation times and poor sampling of the system's features. To improve this sampling, the "multiple Markov chains" (MMC) algorithm was introduced. Also known as "parallel tempering", MMC couples the evolution of several Markov chains. Typically, they are set up at different temperatures for a system with the same remaining parameters. Each Markov chain evolves independently a configuration until a *swap move* is attempted. This move introduces the possibility of exchanging configurations between chains, still fulfilling a global detailed balance condition. The basic idea is that each chain performs a random walk in the temperature space, visiting from time to time high temperatures, where it evolves and renews quickly. Then, stochastically, it returns to low temperatures with a different shape. In comparison, single chains at low temperatures would not decorrelate so well and often remain stuck inside local minima of the energy. A second advantage of the MMC method is that one achieves at once the sampling at different temperatures, which is often needed in any case for evaluating thermodynamic quantities.

Let us consider a set of M inverse temperatures $\beta_1 < \beta_2 < \dots < \beta_M$. We may have β_1 low enough to represent a high-temperature regime of the system, while β_M gives the lowest temperature of interest. These β 's might span the range of interest around a critical point and should contain some small enough $\beta_1 < \beta_2, \dots$ so that the evolution of the Markov chains at those values has small autocorrelation times. We recall that the canonical probability density for a given β is

$$\pi_k(\mathcal{C}) = \frac{e^{-\beta E(\mathcal{C})}}{Z_k} \quad \text{with} \quad Z_k = \sum_{\mathcal{C}} e^{-\beta E(\mathcal{C})} \quad (1.100)$$

We may think of the collection of configurations at different temperatures as a single macro-configuration $\mathcal{C} = (\mathcal{C}^{(1)}, \dots, \mathcal{C}^{(M)})$ with factorized probability

$$\pi(\mathcal{C}) = \prod_k \pi_k(\mathcal{C}^{(k)}) \quad (1.101)$$

In between blocks of the usual Monte Carlo evolution for each chain, one introduces the chance of swapping configurations between chains, say with index k_1 and k_2 (see the sketch in Fig. A.12). Thus, the "old"

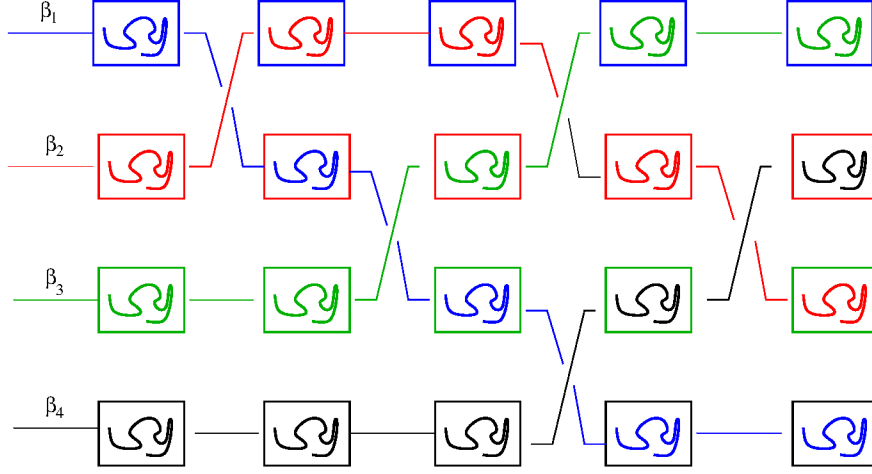


Figure 1.10: MMC swapping scheme for 4 chains. Each chain starts with a configuration colored the same way. To keep track visually of the swappings, colors are then preserved.

and "new" global configurations are

$$\begin{aligned}\mathcal{C} &= (\mathcal{C}^{(1)}, \dots, \mathcal{C}^{(k_1)}, \dots, \mathcal{C}^{(k_2)}, \dots, \mathcal{C}^{(M)}) \\ \mathcal{C}' &= (\mathcal{C}^{(1)}, \dots, \mathcal{C}^{(k_2)}, \dots, \mathcal{C}^{(k_1)}, \dots, \mathcal{C}^{(M)}) \\ &\quad \beta_1, \dots, \beta_{k_1}, \dots, \beta_{k_2}, \dots, \beta_{k_M}\end{aligned}\tag{1.102}$$

The new configuration $\mathcal{C}'$ is accepted fulfilling the detailed balance applied to the generalized ensemble with M temperatures,

$$\pi(\mathcal{C})p_{\mathcal{C}\mathcal{C}'} = \pi(\mathcal{C}')p_{\mathcal{C}'\mathcal{C}}\tag{1.103}$$

in which all terms of the other $M - 2$ temperatures cancel out (as well as the partition functions). By simplifying the notation with $E_a = E(\mathcal{C}^{(k_1)})$, $E_b = E(\mathcal{C}^{(k_2)})$, we rewrite (1.103) as

$$e^{-\beta_{k_1} E_a} e^{-\beta_{k_2} E_b} p_{\mathcal{C}\mathcal{C}'} = e^{-\beta_{k_1} E_b} e^{-\beta_{k_2} E_a} p_{\mathcal{C}'\mathcal{C}}\tag{1.104}$$

which can be further reshaped as

$$\frac{p_{\mathcal{C}\mathcal{C}'}}{p_{\mathcal{C}'\mathcal{C}}} = e^{(\beta_{k_2} - \beta_{k_1})(E_b - E_a)}\tag{1.105}$$

This form is suitable for a Metropolis proposal with

$$p_{\mathcal{C}\mathcal{C}'} = \min \left(1, e^{(\beta_{k_2} - \beta_{k_1})(E_b - E_a)} \right)\tag{1.106}$$

Assuming $\beta_{k_2} > \beta_{k_1}$, the move is always accepted if $E_b > E_a$, i.e., if the configuration of higher energy E_b is sent to a higher temperature while a configuration with lower E_a is sent to a more fitting lower temperature environment. The opposite takes place stochastically according to (1.107). This kind of swap is simple and fast to perform, as no configuration change is required. In the practical implementation, usually, one does not copy and exchange configurations but rather uses a list of pointers I_k , one per temperature pointing to its current configuration $\mathcal{C}^{(k)} = \mathcal{C}[I_k]$. Hence, the move in practice amounts to swapping I_{k_1} with I_{k_2} .

Typically, two adjacent chains k_1 and $k_2 = k_1 + 1$ are considered. The reason for this choice is that typically they have a higher chance of swapping configurations than any other pair of distant β 's. It is recommended to have a significant swapping rate between Markov chains. Some say it should be around 20%. The swapping occurs often if the Boltzmann weights of the swapped configurations are not so different from those before swapping, which occurs if the probability density of energies at β_{k_1} and β_{k_2} are significantly overlapped. It is important to guarantee a reasonable swapping rate between any pairs of the MMC. If by mistake some pair $\beta_{k_1}, \beta_{k_1+1}$ is too different and prevents swapping, there emerges a bottleneck that hampers the diffusion of configurations in the whole temperature space and, eventually, leads to poor performance of the MMC.

As a final note, we add that, although this introduction of MMC is presented with the version at different temperatures, one may extend the technique to cases where canonical Boltzmann weights are replaced by generic π 's. The exchange between a configuration $\mathcal{C}_a$ initially in chain k_1 and a configuration $\mathcal{C}_b$ initially in chain k_2 is then performed with Metropolis probability

$$p_{\mathcal{C}\mathcal{C}'} = \min \left(1, \frac{\pi_{k_2}(\mathcal{C}_a)\pi_{k_1}(\mathcal{C}_b)}{\pi_{k_1}(\mathcal{C}_a)\pi_{k_2}(\mathcal{C}_b)} \right) \quad (1.107)$$

Again, this rule is easy to interpret: the new configuration $\mathcal{C}'$ with swapped $\mathcal{C}_a, \mathcal{C}_b$ is always accepted if their combined probability $\pi_{k_2}(\mathcal{C}_a)\pi_{k_1}(\mathcal{C}_b)$ is larger than that of the initial $\mathcal{C}$, $\pi_{k_1}(\mathcal{C}_a)\pi_{k_2}(\mathcal{C}_b)$. Otherwise, a stochastic choice is made.

1.12 Reweighting

1.12.1 Umbrella sampling

Umbrella sampling is an example of a class of Monte Carlo techniques, whose strategy is to facilitate unlikely trajectories through states that are very rare in the canonical ensemble. These trajectories are transitions needed to connect otherwise isolated relevant regions of the phase space. Thus, umbrella sampling tries not to miss the sampling of some important configurations. A typical example is that of two energy minima separated by a high saddle point of the energy function (high compared to $1/\beta$). The sampling of configurations only in one of the two minima would lead to *quasi ergodicity*.

We consider a generic change of measure $\pi \rightarrow g$,

$$\langle O \rangle_\pi = \sum_{\mathcal{C}} O(\mathcal{C}) \pi(\mathcal{C}) \quad (1.108a)$$

$$= \sum_{\mathcal{C}} O(\mathcal{C}) \frac{\pi(\mathcal{C})}{g(\mathcal{C})} g(\mathcal{C}) = \left\langle O \frac{\pi}{g} \right\rangle_g \quad (1.108b)$$

Umbrella sampling adopts a modified energy function $E_g(\mathcal{C}) = E(\mathcal{C}) + U(\mathcal{C})$ and

$$g(\mathcal{C}) = \frac{e^{-\beta[E(\mathcal{C})+U(\mathcal{C})]}}{\sum_{\mathcal{C}} e^{-\beta[E(\mathcal{C})+U(\mathcal{C})]}} \quad (1.109)$$

where $U(\mathcal{C})$ is an auxiliary potential energy, so that

$$\langle O \rangle_\pi = \frac{\left\langle O \frac{e^{-\beta E}}{g} \right\rangle_g}{\left\langle \frac{e^{-\beta E}}{g} \right\rangle_g} = \frac{\langle O e^{\beta U} \rangle_g}{\langle e^{\beta U} \rangle_g} \quad (1.110)$$

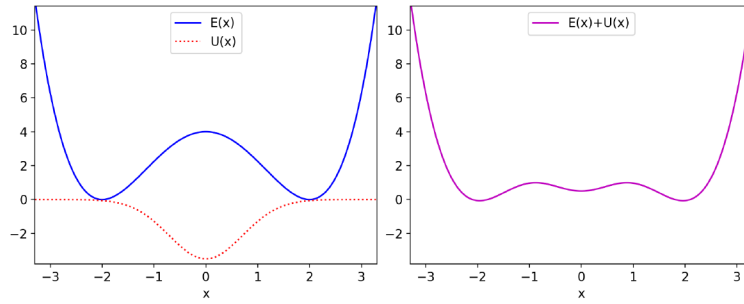


Figure 1.11: Sketch of the umbrella sampling technique. Here, x represents a reaction coordinate on a multi-dimensional system, i.e. a combination of microscopic variables that well represents the difference between configurations. In umbrella sampling, one introduces $U(x)$ so that the saddle between minima of the energy $E(x_s)$ becomes smaller for $E(x_s) + U(x_s)$ (here $x_s = 0$). Thus, for low β , sampling of configurations occurring with frequency $\propto \exp[-\beta(E(x_s) + U(x_s))]$ is possible, as opposed to that with frequency $\propto \exp[-\beta E(x_s)]$.

Then, as described for importance sampling (Sec. 1.1.2), rather than inefficiently sampling $\mathcal{C}$'s with a uniform distribution and applying the weights $g(\mathcal{C})$ to compute averages $\langle \dots \rangle_g$, one uses importance sampling with g in the simulations to obtain a series $\{\mathcal{C}_n\}$ of $n \leq N_{\text{data}}$ samples, from which

$$\langle O \rangle_\pi \simeq \frac{\sum_n O(\mathcal{C}_n) e^{\beta U(\mathcal{C}_n)}}{\sum_n e^{\beta U(\mathcal{C}_n)}} \quad (1.111)$$

One wants to exploit the additional freedom of choosing $U(\mathcal{C})$ to allow the system to perform transitions between rare states. For example, $U(\mathcal{C})$ may be tailored to lower some saddle point energies, e.g., by setting its minima where these saddle points are expected (see the example in Fig. 1.11). Sampling is then augmented around these saddles. This enrichment in the statistics is compensated by their lower weight $e^{\beta U}$, compared to a weight 1 of pure importance sampling.

1.12.2 Histogram method

The histogram method collects data and reweights them to different temperatures. Going back to the previous section, it amounts to running a simulation (importance) sampling with $\pi \propto e^{-\beta E}$ and then reweighting at $\beta' \neq \beta$ with $g \propto e^{-\beta' E}$:

$$\langle O \rangle = \frac{\left\langle O \frac{e^{-\beta E}}{g} \right\rangle_g}{\left\langle \frac{e^{-\beta E}}{g} \right\rangle_g} = \frac{\langle O e^{(\beta' - \beta) E} \rangle_g}{\langle e^{(\beta' - \beta) E} \rangle_g} \quad (1.112)$$

It is termed *histogram* method because a histogram of the energy is sufficient for computing energy and specific heat at a different temperature. However, for a generic observable O , one needs to record tables of samples n with O_n, E_n , in order to apply (1.112). In practice, one sample at β' and reweights the results to β . The method works well as long the energy distributions at the two temperatures are overlapped.

1.13 Continuous-time Markov processes

Markov chains described above evolve in discrete time steps according to transition probabilities p_{ij} . Furthermore, in Monte Carlo, their dynamics need not be related to any natural dynamics of the studied system. It is just a user's choice. Therefore, Markov chains are unsuitable for describing systems where the jumping between states does not occur in discrete time, as in most natural systems. Simulating the dynamics of these systems requires an approach based on the Master Equation (ME). Besides and unrelated to realism, there can be another reason for adopting a continuous rather than discrete time-step evolution in some systems. If a Markov chain typically consists of rejected moves, it could be computationally more efficient to compute a residence time in the configuration, followed by an enforced move to a different state. Think, for example, of a configuration with all spins up for an Ising model at a very low temperature. Single spin-flip moves would unsuccessfully try updating a spin many times. It would be more efficient to extract a single residence time from the statistics of these stretches of unsuccessful moves, followed by a flipping of a spin.

In the following, we will define continuous-time jump Markov processes, their ME, and how to implement them numerically with the Gillespie algorithm. We will also discuss the connection between this approach and that of Markov chains.

1.13.1 Trajectories of continuous-time Markov processes

We focus on systems with N discrete states $i, j, \dots$. The dynamics are defined by the transition rates w_{ij} for any possible jump $i \rightarrow j$. From Markovianity, $p_{ij} = w_{ij}dt$ is the probability per dt to jump from i to j , given that the system is in state i . This holds for a short time step $dt \ll 1/\max_{i,j}(w_{ij})$, during which it is unlikely to have more than one jump. The p_{ij} can be introduced as the transition probability of a Markov chain, for $i \neq j$. The probability of returning to the same site, for normalization, is

$$p_{ii} = 1 - \sum_{j \neq i} p_{ij} = 1 - \lambda_i dt \quad (1.113)$$

where we introduced the escape rate from state i ,

$$\lambda_i \equiv \sum_{j \neq i} w_{ij} \quad (1.114)$$

All these p_{ij} probabilities compose the transition matrix P . One can check that the sum of the elements in a column of P equals 1. Hence, from these continuous-time Markov processes, it is possible to build (discrete-time) Markov chains.

Since the system is Markovian, the probability of observing for q infinitesimal steps dt that the system stays still in state i , up to a time $t = q dt$, is factorized as

$$p(t|i) = p_{ii}^q = (1 - \lambda_i dt)^q \simeq e^{-\lambda_i q dt} \xrightarrow{dt \rightarrow 0} e^{-\lambda_i t} \quad (1.115)$$

Then, the probability of jumping during the next infinitesimal time step dt is $\lambda_i dt$. The specific probability that such a jump is to state j is $p_{ij} = w_{ij}dt$. We can thus write the probability $\mathcal{P}(\omega)$ of a trajectory ω with n jumps, described by a sequence of states $(C_0, C_1, C_2, \dots, C_n)$ and by their residence times $(t_0, t_1, t_2, \dots, t_n)$,

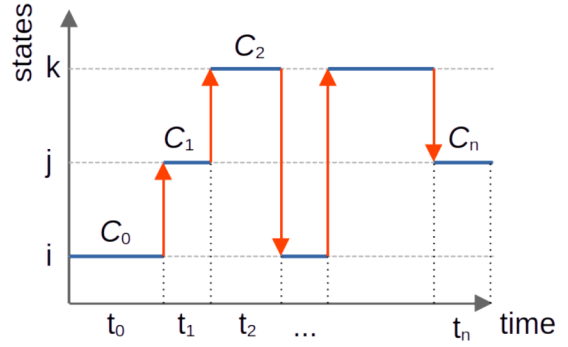


Figure 1.12: Sketch of a Markov jump process' trajectory.

as the one sketched in Fig. 1.12. Such probability depends also on the probability $\rho_0(\mathcal{C}_0)$ of finding the initial state $\mathcal{C}_0$:

$$\begin{aligned}\mathcal{P}(\omega) &\sim \rho_0(\mathcal{C}_0) \prod_{\alpha=0}^{n-1} \left[e^{-\lambda_{\mathcal{C}_\alpha} t_\alpha} p_{\mathcal{C}_\alpha \mathcal{C}_{\alpha+1}} \right] e^{-\lambda_{\mathcal{C}_n} t_n} \\ &\sim (dt)^n \rho_0(\mathcal{C}_0) e^{-\sum_{\alpha=0}^n \lambda_{\mathcal{C}_\alpha} t_\alpha} \prod_{\alpha=0}^{n-1} w_{\mathcal{C}_\alpha \mathcal{C}_{\alpha+1}}\end{aligned}\quad (1.116)$$

To generate stochastically a trajectory, an inefficient method is discretizing time in dt steps and applying the Markov chain for such a choice. A better option is the following algorithm.

1.13.2 The Gillespie algorithm

The Gillespie algorithm realizes the dynamics just described and yields trajectories according to the probability (1.116). It was introduced in the context of chemical reactions.

One starts from a state $\mathcal{C}$ of the system. The probability density function of the waiting time for a jump from $\mathcal{C}$ is $p_{\mathcal{C}}(t) = \lambda_{\mathcal{C}} e^{-\lambda_{\mathcal{C}} t}$. The first step of the Gillespie algorithm is extracting at random a time $\tau \sim p_{\mathcal{C}}(t)$ from this exponentially decaying probability density.

The second step of the Gillespie algorithm requires choosing the state j where to jump from $\mathcal{C}$. The probability $p(j|\mathcal{C})$ of picking randomly a given j must be proportional to the transition rate $w_{\mathcal{C}j}$, which is the probability per unit of time of observing such transition. In practice, for normalization, $p(j|\mathcal{C}) = w_{\mathcal{C}j}/\lambda_{\mathcal{C}}$. The j is chosen by extracting a random number and using the cumulative distribution of $p(j|\mathcal{C})$, as described earlier in these notes.

1.13.3 The Master equation

The ME describes the evolution of the probability of states, $\rho(i)$, for a continuous time Markov jump process. It reads

$$\frac{d}{dt} \rho(i) = \sum_{j \neq i} \rho(j) w_{ji} - \sum_{i \neq j} \rho(i) w_{ij} \quad (1.117)$$

The first, positive term of the r.h.s. represents the incoming flux of probability from external states that may jump to a state i with rate w_{ji} . The second, negative, term is the outgoing flux of probability from the state i . The latter is proportional to the escape rate λ_i , defined in (1.114), with which we may rewrite the ME as

$$\frac{d}{dt} \rho(i) = \sum_{j \neq i} \rho(j) w_{ji} - \rho(i) \lambda_i \quad (1.118)$$

For continuous-time jump Markov processes, the condition of detailed balance reads

$$\rho(j) w_{ji} = \rho(i) w_{ij} \quad (1.119)$$

We adopt this condition for simulating systems evolving according to conservative forces, thus converging to, or evolving in equilibrium. However, the ME is a generic approach that works also for irreversible systems (with some $w_{ij} \neq 0$ while $w_{ji} = 0$), for effective descriptions (e.g., population dynamics of species), etc.

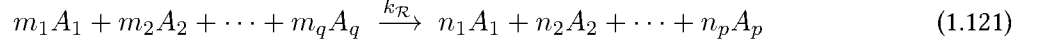
It is often convenient to keep the notation compact by using matrices. By defining a matrix W with elements w_{ij} outside the diagonal, and $w_{ii} = -\lambda_i$, the ME for the vector $\boldsymbol{\rho} = (\rho(1), \rho(2), \dots, \rho(N))$ simplifies to

$$\frac{d}{dt} \boldsymbol{\rho} = \boldsymbol{\rho} W \quad (1.120)$$

Note that $\pi W = 0$ for a stationary distribution π , which is the left eigenvector of W with the largest real component of the eigenvalue (specifically, zero). Other left eigenvectors of W must have real components smaller than zero and can be complex if the system is out of equilibrium.

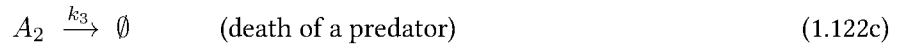
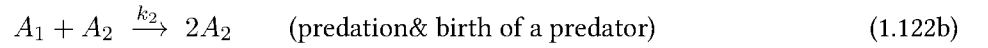
1.13.4 Coupled chemical reactions

In many processes in physics and chemistry, one deals with systems described as coupled chemical reactions. In general, for different types of particles A_l , the chemical reactions can be written as



with stoichiometric coefficients m_l, n_l . The reaction rate $k_{\mathcal{R}}$ is such that $k_{\mathcal{R}} dt$ is the probability that the combination of particles on the left-hand side of (1.121) reacts in a time interval dt . If the reaction (1.121) is reversible, its inverse in general takes place with a different backward rate.

The A_l are chemical molecules or atoms, but the same scheme applies, for example, to interacting species with predator-prey population dynamics. In the Lotka-Volterra model, there are A_1 prey and A_2 predators. The evolution takes place with transitions



One usually assumes that the system is well-mixed, which happens if diffusion is fast enough to efficiently mix the particles on time scales much shorter than the typical jumping times (it is the so-called *mean field approximation*). Under this approximation, the state of the system with $\mathcal{N}$ types of particles is given by $\mathcal{C} = (X_1, \dots, X_{\mathcal{N}})$, where X_l is the number of particles of type A_l . A transition $\mathcal{R} : \mathcal{C} \rightarrow \mathcal{C}'$ with rate $w_{\mathcal{R}}$ depends on a rate constant $k_{\mathcal{R}}$ and on the number of particles. For example, transition (1.122a) occurs with rate $w_1 = k_1 X_1$ because it is proportional to the number X_1 of prey, and connects the state $\mathcal{C} = (X_1, X_2)$ to the state $\mathcal{C}' = (X_1 + 1, X_2)$ (see Fig. 1.13). Similarly, transition (1.122b) has rate $w_2 = k_2 X_1 X_2$ and leads to $\mathcal{C}' = (X_1 - 1, X_2 + 1)$. Finally, (1.122c) has $w_3 = k_3 X_2$ and leads to $\mathcal{C}' = (X_1, X_2 - 1)$. We may visualize states as points in the $\mathbb{N} \times \mathbb{N}$ square lattice, as in Fig. 1.13, and note that only three neighbors of each point are at most reachable. Furthermore, in this model, the transitions are not reversible. In general, these chemical rates are of the form

$$w_{\mathcal{R}} = k_{\mathcal{R}} \{\text{number of reagents combinations for reaction } \mathcal{R}\} \quad (1.123)$$

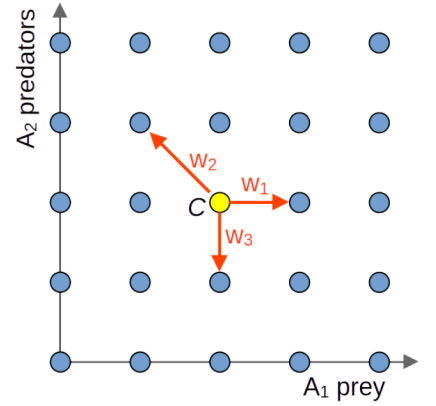


Figure 1.13: Sketch of the possible transitions from a state $\mathcal{C}$ to another one in the Lotka-Volterra model.

1.13.5 Rate equations

The rate equations describe the time evolution of the average number of particles

$$\langle X_l \rangle = \sum_{\mathcal{C}} X_l \rho(\mathcal{C} = (X_1 \dots, X_N)) \quad (1.124)$$

and is derived from the ME (1.117). In general, one cannot deduce dynamical equations for the $\langle X_l \rangle$'s in closed form, due to the possible presence of products of the particle numbers in (1.123). These equations would rather be of the form

$$\begin{aligned} \frac{d}{dt} \langle X_l \rangle &= \sum_{\mathcal{C}} X_l \frac{d}{dt} \rho(\mathcal{C} = (X_1 \dots, X_N)) \\ &= \mathcal{F}(\langle X_1 \rangle, \langle X_2 \rangle, \dots, \langle X_1 X_2 \rangle, \langle X_1 X_3 \rangle, \dots, \langle X_1 X_2 X_3 \rangle, \langle X_1 X_2 X_4 \rangle, \dots) \end{aligned} \quad (1.125)$$

One can get back to differential equations in a closed form by neglecting correlations, i.e., by approximating terms as $\langle X_l X_k \dots X_q \rangle \approx \langle X_l \rangle \langle X_k \rangle \dots \langle X_q \rangle$. These equations are deterministic, not exact, and determine only the evolution of the average number of particles. They do not describe fluctuations around these averages and are accurate only if these fluctuations are relatively small, which occurs if the number of all particles is large, $\langle X_l \rangle \gg 1$. Conversely, if the number of particles is small, fluctuations are relevant and the rate equations description is no longer valid.

For example, for the Lotka-Volterra model, the rate equations for the average numbers $X_l \equiv \langle X_l \rangle$ become

$$\frac{d}{dt} X_1 = k_1 X_1 - k_2 X_1 X_2 \quad (1.126a)$$

$$\frac{d}{dt} X_2 = k_2 X_1 X_2 - k_3 X_2 \quad (1.126b)$$

They have two stationary solutions ($\frac{d}{dt} X_1 = \frac{d}{dt} X_2 = 0$): a trivial $\mathcal{C}^0$ with $X_1 = X_2 = 0$ and a solution $\mathcal{C}^*$ with $X_1 = k_3/k_2$, $X_2 = k_1/k_2$. Furthermore, they have oscillatory solutions, which is why they are interesting for describing population dynamics. The oscillatory solutions emerge by expanding X 's to linear order around $\mathcal{C}^*$: $X_1 = \epsilon_1 + k_3/k_2$, $X_2 = \epsilon_2 + k_1/k_2$. One verifies that these solutions are of the form $\epsilon_1 = \epsilon \cos(\omega t + \phi)$ and $\epsilon_2 = \epsilon \sqrt{k_1/k_3} \sin(\omega t + \phi)$. Figure 1.14 shows an example of solutions for initial conditions close to the stable point $\mathcal{C}^*$ (left panel) and far from it (right panel).

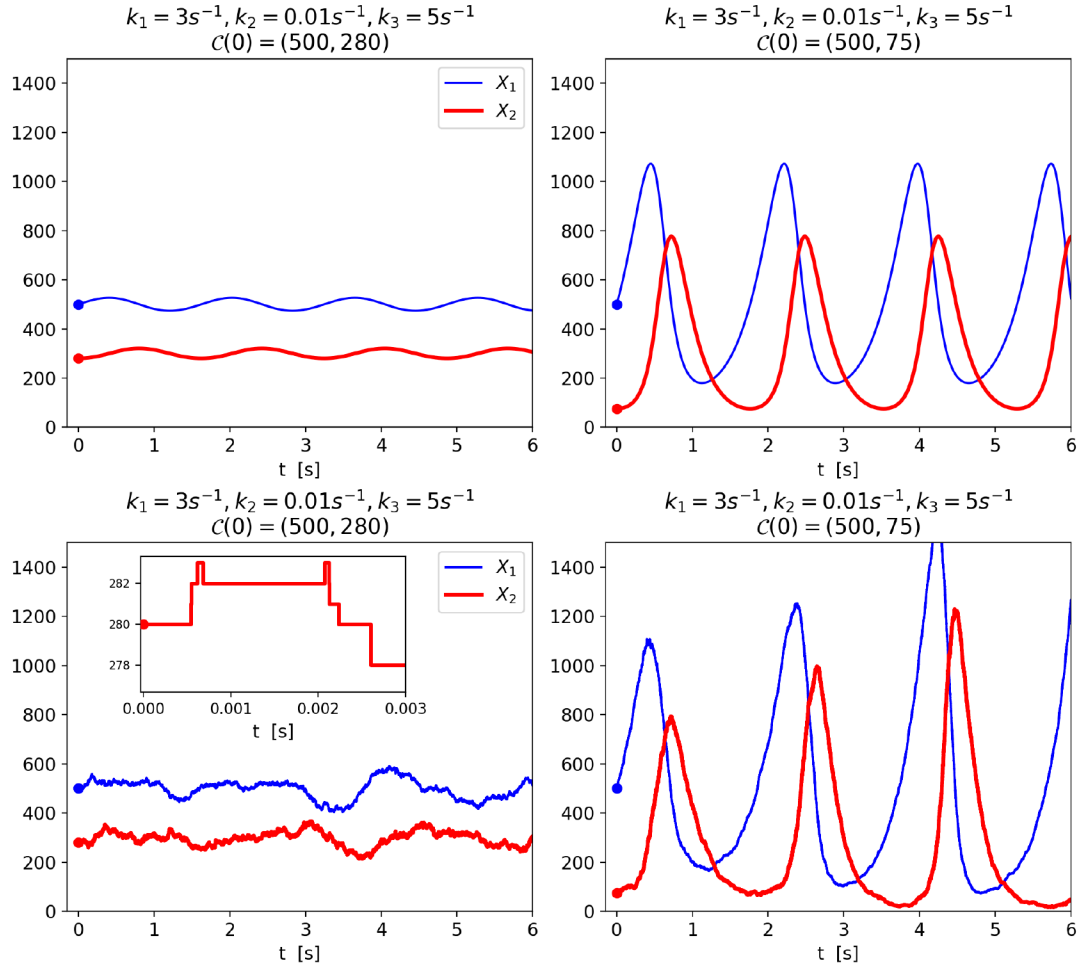


Figure 1.14: Upper panels: Numerical solutions of the deterministic Lotka-Volterra rate equations (1.126), for $k_1 = 3s^{-1}$, $k_2 = 1/100s^{-1}$, $k_3 = 5s^{-1}$, starting close to the stable point (left panel) and far from it (right panel). Note that these constants have the dimension s^{-1} because X 's are adimensional numbers in this model. Lower panels: stochastic trajectories generated with the Gillespie algorithm (same parameters of their corresponding upper panels). The zoom in the inset on the left shows some jumps between states in the X_2 dynamics.

1.13.6 The Brusselator

Often, the physiological state of a living organism follows an oscillation of 24 hours. A circadian clock is present if there is an endogenous component in this oscillation; namely, the cycle remains relatively stable even without external daily stimuli. This implies that the level of hormones and other chemical elements oscillates with a period of about 24h.

The so-called Brusselator is the minimal paradigmatic model for generating chemical periodicity, such as in the Belousov–Zhabotinsky reaction. A state $C = (X, Y)$ of the system represents the numbers X and Y of molecules of two species A_X, A_Y . The chemical reactions of the Brusselator are

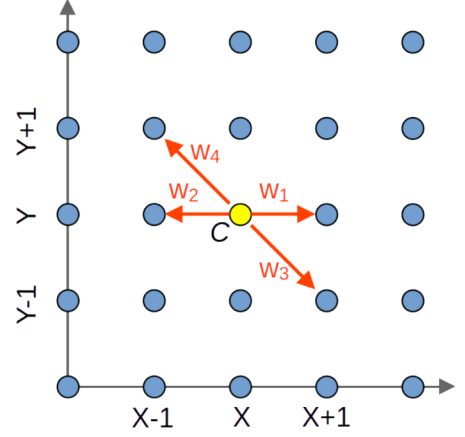
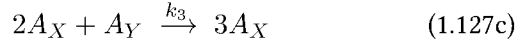


Figure 1.15: Sketch of the possible transitions from a state C to another one in the Brusselator model.

The stochastic version of the Brusselator, translating the original deterministic dynamics into a Markov jump process, introduces the finite size of the system via a “volume” Ω . Following the sketch of Fig. 1.15, these C -dependent rates are rewritten as

$$w_1 = a\Omega \quad \text{for } (X, Y) \rightarrow (X + 1, Y) \quad (1.128a)$$

$$w_2 = X \quad \text{for } (X, Y) \rightarrow (X - 1, Y) \quad (1.128b)$$

$$w_3 = \frac{1}{\Omega^2} X(X - 1)Y \quad \text{for } (X, Y) \rightarrow (X + 1, Y - 1) \quad (1.128c)$$

$$w_4 = bX \quad \text{for } (X, Y) \rightarrow (X - 1, Y + 1) \quad (1.128d)$$

Typical values are $a = 2$ and $b = 5$. The latter is large enough to generate oscillations with a limit cycle that is an attractor for the stochastic dynamics. We may also set $\Omega = 1000$ for reference.

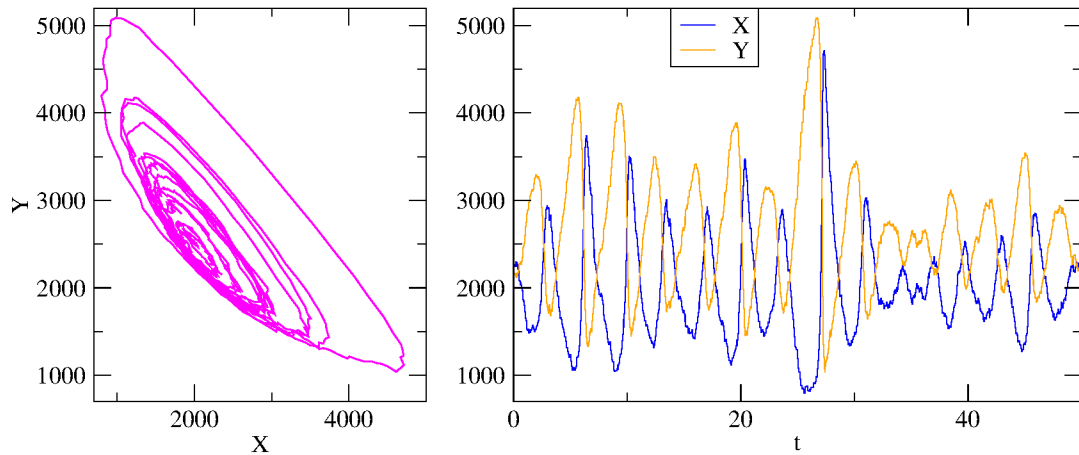


Figure 1.16: Example of stochastic limit cycle and of the corresponding oscillations in time, for the Brusselator.

2.1 Introduction

Numerical simulations provide an alternative way to investigate physical systems. By means of simulations, one can test a model and compute observables. If these observables are in agreement (qualitative or quantitative) with real experiments, the model can be validated or refuted. Alternatively, theoretical predictions can be tested, when real experiments are not available; as such, numerical simulations present an alternative way to validate theoretical approaches. These aims can be achieved using any kind of numerical simulation technique. However, Molecular Dynamics (or "MD", in short) and other "Dynamics" simulations are, in a sense, closer to real experiments than other techniques. The approach of Molecular Dynamics is to solve the equations of motion of a system constituted by a large number N^1 of interacting particles. One starts from an initial condition (see Sec. 2.2.4) and solves the equations until the observables (see Sec. 2.2.5) do not change any more with time or fluctuate around a certain value. In this respect, a MD simulation is similar to an experiment, where one waits for the system to settle to equilibrium or to a steady state. MD allows to generate trajectories, in the *phase space*, that reflect the dynamics of the system, i.e. the different kinds of interactions that are present in the situation of interest. This, in turn, allows Molecular Dynamics access to the dynamics of the system, i.e. allows to reliably estimate the relevant time scales.

It is worth to stress that we will consider only classical MD, which means that we forget about the quantum nature of the system and we solve Newton's equation of motion (no relativistic effects). Albeit these sound like restrictive limitations, there are a plethora of different systems that are classical and non-relativistic; biological systems are an example. In general, all systems where the relevant energy scale is the thermal energy at room temperature, usually grouped under the name Soft Matter, fall into this category. Typically, we deal with micro- or nano-scopic systems, with characteristic lengths ranging from a few Angstroms to a few micrometers.

In general, a MD simulation comprises three different steps:

1. understand the aims of the simulation
2. set up the simulation
3. run the simulation

The order of appearance is, in this case, very much connected with the importance. In fact, understanding what is the physical problem and how to approach it is the most important task; this goes beyond the scope of the course. However, it is important to realize its crucial place. Next, setting up a simulation is, as will become clear later, also quite important as depends on the system under investigation. As such, care should be taken and some "tricks of the trade" may be necessary. Finally, running the simulation is ultimately trivial and should come with little effort, assuming the code is written properly (or assuming one is using an open source simulation code). In what follows, we will discuss the essential ingredients of a MD simulations, namely

1. the common features of all MD simulations (and of most off-lattice simulations)
2. the algorithms to integrate the equations on motion
3. the interaction potentials
4. the different ensembles

that enter, in different ways, in the three steps mentioned above.

¹typically $N \geq 3$

2.2 Setting up a MD simulation

Setting up a MD simulation² is, probably, the most human-intensive part of the workload entailed by numerical simulations. In fact, once the simulation is set, the task can be carried on by the computer until needed, without human intervention. We will discuss, in what follows, some of the basic concepts which pertain not only Molecular Dynamics but also Monte Carlo or other kinds of simulations. However, to some extent, MD is affected by some issues more than other methods. We will further discuss how to set up initial conditions and what are the problems that can be encountered. Finally, we will briefly introduce LAMMPS and we will discuss what can we learn from it for our simulation code.

2.2.1 Atomistic and coarse-grained perspectives

In general, an MD simulation will consider a system made of N “units”, each of them characterised by their position, velocity, forces and, if necessary, other degrees of freedom (such as orientation, angular momenta, torques, charge etc). These units are the “atoms” of our modeling, in the sense that they are the fundamental constituents of our description of the system; they set the finest level of detail available. There is, in principle, freedom in the choice of such a unit and, roughly speaking, one can make the choice of an *atomistic* description or a *coarse-grained* description. The former implies that the units of the system are single atoms while the latter implies that the units of the system are groups of atoms, represented by the position of their centre of mass. Since the position of the centre of mass becomes the finest level of detail available, we lose the information of the internal arrangement of the atoms and their individual velocity and forces. Which description should one choose? It depends.

Atomistic simulations may be necessary to pinpoint subtle change in conformation of complex macromolecules or to highlight molecular properties, such as orientation of a group of molecule at the interface. They are computationally very expensive; only short simulations of relatively small systems (see Sec. 2.2.2) are achievable. Coarse-grained simulations are useful to assess material or thermodynamic properties, as one can simulate large systems for very long times (see Sec. 2.2.2). However, if one aims at achieving a quantitative comparison with experiments, particular care has to be taken in the interaction potential used (see Sec. 2.2.2)

2.2.2 Reduced units

As mentioned, the units (or particles) that compose the system that we want to simulate are characterised by their position, velocity, forces and so on. As always, each of these quantities has its own a unit of measure and the units have to be consistent. In principle, it is possible to use any system of units, for example the International System of Units (SI). However, in practice, a standard CPU benefits greatly, in terms of speed, by dealing with numbers of the order of the unity when performing standard arithmetic calculations. As mentioned, systems of interest are micro- or nano-scopic and the energy is of the order of the thermal energy ($\sim 4 \cdot 10^{-21}$ J). There is a way to circumvent this problem: *reduced units*. Define a unit of length σ , a unit of mass $\bar{m}$ and a unit of energy ϵ : the reduced units are thus

$$r^* = \frac{r}{\sigma}; \quad m^* = \frac{m}{\bar{m}}; \quad E^* = \frac{E}{\epsilon} \quad (2.1)$$

where r , m and E are measured in S.I. units. This choice imposes a unit of time $\tau = \sqrt{\bar{m}\sigma^2/\epsilon}$; other quantities, such as friction, elastic constants etc can be converted in a similar way. Depending on the choice, results can be converted back and forth. In most coarse-grained simulations, the reduced units are the default units and are sometimes called Lennard Jones (LJ) units. This habit established for two reasons. On the one hand, coarse-grained simulations often aim at highlighting universal behaviour by using simple and unspecific interaction

²this is mostly true also for other kinds of simulations

potentials (such as the Lennard Jones potential). Without a reference system, a generic set of reduced units is the only fitting choice. On the other hand, coarse-grained models can fit different length scales; thus, it becomes easier to backmap the results *a posteriori*, which can be done easily, rather than set a conversion table for each system.

2.2.3 Boundary conditions

In a MD simulation as well as in any particle-based simulation (where we evolve or sample system made of a discrete number of units) the simulation is contained in a box. Depending on the dimensionality of the problem, the box can be a segment (1D) a polygon (2D) or a solid figure (3D): for the sake of simplicity, we will consider only rectangles and squares (2D) or cubes and parallelepipeds (3D). The box is essentially a volume, placed in a system of coordinates: each side has a lower bound and an upper bound. The term “boundary conditions” indicates the special rules imposed on the particles at the edge of the simulation box. The most common types of boundary conditions are:

- Periodic (PBC): with periodic boundaries, a particle crossing the boundary “comes back” in the box from the opposite side. This has to be enforced in the code: if the particles position along one axis is higher/lower than the box upper/lower bound, then we subtract/add the value of the box size. This has a more compact form in case the lower corner of the box is in the origin of the reference system

$$x = x - \text{floor}(x/x_{\text{size}}) \cdot x_{\text{size}} \quad (2.2)$$

Notice that you can always shift the coordinate on-the-fly, when the corner is not in the origin. PBCs affect also the calculation of forces, as one has to consider the neighbouring particles across the boundaries as well (see Sec. 2.4). For this, the *minimum image criterion* has to be implemented: if the box is periodic, then the maximum distance (along a certain axis) is half the box size. If two particles are further away, then one can find a *periodic image* of the one of them such that the distance is smaller than half the box size. In practice, calling dx the distance along x , the minimum image criterion is

$$dx = dx - \text{round}(dx/x_{\text{size}}) \cdot x_{\text{size}} \quad (2.3)$$

Notice the difference between the two functions floor and round.

- Reflective (RBC): with reflective boundaries the particle cannot leave the simulation box, in the direction where the boundary is set. They are necessary when dealing with confined systems, such as nano-channels. They can be implemented in different ways, the simplest being the “bounce-back” rule: whenever a particle crosses a boundary, its position is bounced back by the same amount that was travelled beyond the boundary. At the same time, its velocity, along the direction of the crossed boundary, is reversed.

If we allow the number of particles to fluctuate, adsorbing boundaries can also be considered. PBCs are employed to simulate “bulk” systems, i.e. quasi-infinite systems with a finite density. PBCs are a good approximation for a thermodynamics system if the box is large enough so that the spatial fluctuations do not influence themselves through the boundaries or, for extended macromolecules, if the macromolecule does not come in contact with itself through the boundaries. In order to assess these issues, one can change the box size and assess the so-called finite-size effects.

2.2.4 Initial conditions

The initial conditions of a MD simulation are the set of positions and velocities, that will be used as the starting point of the computation. As in real experiments, one may need to wait a certain amount of (simulation) time, usually named *equilibration*, before considering the simulated trajectory as representative of the system. However, in simulations one has the luxury to skip equilibration, if one possesses a suitable set of positions and velocities, i.e. a suitable initial condition. Thus, we can distinguish between:

- the initial condition is a quasi-arbitrary collection of positions and velocity that needs an *equilibration run*
- the initial condition is already an equilibrium or steady state configuration of the system.

Generating a good initial condition is the main challenge of the simulation process. In some specific cases, such as the random close packing problem or the equilibrated conformation of a long polymer chain, dedicated (MC) algorithms allow to generate suitable initial configurations. In all the other cases, the simplest option is to generate a configuration that is easy to code and use MC or MD simulations to reach the desired equilibrated state.

Finally, concerning the initial values of the velocity, a good choice for equilibrium is always to draw the component of the velocity from a Gaussian distribution, with zero mean and variance $m/k_B T$. As a result, the magnitude of the velocity will be distributed according to the Maxwell-Boltzmann distribution, which is a true distribution (or a reasonable approximation) in equilibrium. If the system is connected to a (dissipative) heat bath (see Sec. 2.5), the dynamics will make whatever initial distribution converge to the steady state distribution.

2.2.5 Observables

We list here some observables that are typically useful to understand when the system reached equilibrium and may be useful to characterize the system.

Energy

In a MD simulation one can compute the total kinetic energy as

$$E_K = \sum \frac{1}{2} m_i v_i^2$$

where m_i and $v_i^2 = v_{i,x}^2 + v_{i,y}^2 + v_{i,z}^2$ are the mass and velocity of each particle. Regarding the total potential energy, one has to first set the interaction potential(s). Depending on the system, it is useful to separate the different contributions (see Sec. 2.4). Usually, the energy per particle is considered. For simulations in the microcanonical ensemble, an important check for the correct implementation of (symplectic) algorithms (see Sec. 2.3) is that the total energy is conserved; such a check is also useful for setting the algorithmic time step.

Temperature

For a system in equilibrium at some given temperature T , the equipartition theorem says that the average kinetic energy is related to the temperature T as

$$\frac{m}{2} \langle v^2 \rangle = \frac{3}{2} k_B T$$

thus one can compute the temperature using the relation

$$T_K = \frac{1}{3Nk_B} \sum_i m_i v_i^2 \quad (2.4)$$

As the kinetic energy fluctuates in time, the previous relation defines an “instantaneous” temperature, that also fluctuates. For a sufficiently large system, T_K converges to a constant value which, in the canonical ensemble (see Sec.2.5, is the chosen temperature.

Mean Square Displacement and Diffusion coefficient

The Mean Square Displacement (MSD) $\langle \Delta r^2(t) \rangle$ provides a measure of how much a particle has traveled in a certain interval of time. It is defined as

$$\langle \Delta r^2(t) \rangle = \langle (\vec{r}(t) - \vec{r}(0))^2 \rangle \quad (2.5)$$

where the average is an ensemble average. In equilibrium or in steady state, the average can be performed also along the trajectory (a time average) and it becomes a function of the *lag time* τ

$$\langle \Delta r^2(\tau) \rangle = \langle (\vec{r}(t_0 + \tau) - \vec{r}(t_0))^2 \rangle \quad (2.6)$$

A dynamical way to check equilibrium/steady state is to compute the MSD (ensemble average only!) for different lag times.

The Stokes-Einstein relation relates the MSD to the diffusion coefficient

$$\langle \Delta r^2(\tau) \rangle = 2dD\tau \quad (2.7)$$

for purely diffusive processes, d being the dimensionality of the system. Thus, assuming the MSD grows linearly in time, the diffusion coefficient can be computed as

$$D = \lim_{t \rightarrow \infty} \frac{\langle \Delta r^2(\tau) \rangle}{2d\tau} \quad (2.8)$$

In practice, one can also perform a fit in the linear regime. Alternatively, within the same assumptions, one can compute D using the Green-Kubo relation

$$D = \frac{1}{3} \lim_{t \rightarrow \infty} \int_0^t \langle \vec{v}(t_0 + \tau) \cdot \vec{v}(t_0) \rangle d\tau \quad (2.9)$$

that relates D to the autocorrelation function of the velocity.

Notice that the MSD might not be linear: in such a case, usually one can measure a *mobility* μ by fitting the relation $\langle \Delta r^2(\tau) \rangle = \mu \tau^\alpha$.

Pressure

The pressure of the system is computed from the virial theorem

$$PV = Nk_B T + \frac{1}{3} \sum_{i=1}^N \langle \vec{r}_i \cdot \vec{F}_i \rangle \quad (2.10)$$

Consider the work

$$W = \sum_{i=1}^N \vec{r}_i \cdot \vec{F}_i^{TOT} \quad (2.11)$$

where $\vec{F}_i^{TOT}$ is the total force acting on the i -th particle and take the time average. Using Newton’s second equation and integrating by parts, one finds

$$\langle W \rangle = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t \sum_{i=1}^N \vec{r}_i \cdot \vec{F}_i^{TOT} dt' = - \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t \sum_{i=1}^N m_i |\dot{\vec{v}}_i|^2 dt' = -2 \langle E_k \rangle \quad (2.12)$$

i.e. the average work is equal to twice the average kinetic energy. From equipartition we get

$$\langle W \rangle = -3Nk_B T \quad (2.13)$$

The force has, without loss of generality, two components

$$\vec{F}_i^{TOT} = \vec{F}_i + \vec{F}_i^w \quad (2.14)$$

where $\vec{F}_i$ is due to the interaction with all the other particles in the system, while $\vec{F}_i^w$ arises from the interaction with the walls of the container. Consider a rectangular parallelepiped container with sides of length L_x, L_y, L_z . We assume that the force $\vec{F}_i^w$ is always perpendicular to the sides; then, for each size, we get a contribution to the average that is given by

$$\left\langle \sum_{i=1}^N \vec{r}_i \cdot \vec{F}_i^w \right\rangle = L_x \langle \vec{F}_i^w \rangle = -PL_y L_z L_x = -PV \quad (2.15)$$

where we considered, as an example, the side perpendicular to the x axis. Thus, summing on the three sides, we get

$$\left\langle \sum_{i=1}^N \vec{r}_i \cdot \vec{F}_i^w \right\rangle = -3PV \quad (2.16)$$

Combining the Eqs (2.11),(2.13),(2.14), (2.16), we get the result Eq. ((2.10)). Proof of the equation Eq. ((2.10)) is left as an exercise.

If the interparticle force arise from a pair potential (see Sec. 2.4), then we can write

$$PV = Nk_B T - \frac{1}{3} \sum_{i < j} \left\langle \vec{r}_{ij} \cdot \frac{dV(r_{ij})}{dr_{ij}} \right\rangle \quad (2.17)$$

Density, Radial Distribution Function and Structure Factor

Systems at finite density (gas, liquid, crystals or other phases, such as gels, glasses, quasi-crystals etc) can be characterized by how the particles organize in space. The particle density and the closely related *particle distribution functions* provide a complete description of the structure of the fluid. Often, it is sufficient to know only low order particle distribution functions: in particular, the pair density $\rho_N^{(2)}$ is often sufficient to calculate the equation of state and the other thermodynamic properties. The definition of the n -particle density is

$$\rho_N^{(n)}(\mathbf{r}^n) = \frac{N!}{(N-n)!} \frac{1}{Z_N} \int \exp(-\beta V_N) d\mathbf{r}^{(N-n)} \quad (2.18)$$

The quantity $\rho_N^{(n)}(\mathbf{r}^n) d\mathbf{r}^n$ gives the probability of finding n particles of the system with coordinates in the volume element $d\mathbf{r}^n$, irrespective of the positions of the remaining particles and of all the momenta. As such

$$\int \rho_N^{(n)}(\mathbf{r}^n) d\mathbf{r}^n = \frac{N!}{(N-n)!} \quad (2.19)$$

For example, $\int \rho_N^{(1)}(\mathbf{r}) d\mathbf{r} = N$, i.e. the single particle density in a uniform fluid is simply equal to the number density. The n -particle distribution function $g_N^{(n)}(\mathbf{r}^n)$ is defined as

$$g_N^{(n)}(\mathbf{r}^n) = \frac{\rho_N^{(n)}(\mathbf{r}_1 \dots \mathbf{r}_n)}{\prod_{i=1}^n \rho_N^{(1)}(\mathbf{r}_i)} = \frac{\rho_N^{(n)}(\mathbf{r}_1 \dots \mathbf{r}_n)}{\rho^n} \quad (2.20)$$

in terms of the corresponding particle density; the latter equality holds for uniform systems. If the system is isotropic, the pair distribution function $g_N^{(2)}(\mathbf{r}_1, \mathbf{r}_2)$ depends only on the distance $r_{12} = |\mathbf{r}_2 - \mathbf{r}_1|$ and then

is usually called radial distribution function $g(r)$. When the distance r is much larger than the range of the interparticle potential, then $g_2(r)$ approaches the ideal-gas limit, $(1 - 1/N) \approx 1$.

The particle densities defined by Eq. (2.18) can be expressed in terms of δ functions of the position. We start from

$$\langle \delta(\mathbf{r} - \mathbf{r}_1) \rangle = \frac{1}{Z_N} \int \dots \int \exp[-\beta V_N(\mathbf{r}, \mathbf{r}_2 \dots \mathbf{r}_N)] d\mathbf{r}_2 \dots d\mathbf{r}_N \quad (2.21)$$

where the ensemble average is a function of the coordinate $\mathbf{r}$ but is independent of the particle label (1 in this case). A sum over all particle labels is equal to N times the contribution of a single particle. We then get

$$\rho_N^{(1)}(\mathbf{r}) = \left\langle \sum_{i=1}^N \delta(\mathbf{r} - \mathbf{r}_i) \right\rangle \quad (2.22)$$

which is the ensemble average of a microscopic particle density. Similarly,

$$\rho_N^{(2)}(\mathbf{r}, \mathbf{r}') = \left\langle \sum_{i=1}^N \sum_{j=1}^{N'} \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \right\rangle \quad (2.23)$$

where the prime on the summation indicated $i \neq j$. If the system is homogeneous and isotropic

$$\left\langle \sum_{i=1}^N \sum_{j=1}^{N'} \delta(\mathbf{r} - \mathbf{r}_j + \mathbf{r}_i) \right\rangle = \frac{\rho^2}{N} \int g_N^{(2)}(\mathbf{r}, \mathbf{r}') d\mathbf{r}' = \rho g(r) \quad (2.24)$$

The definition of $g(r)$ implies that, on average, the number of particles lying within the range r to $r + dr$ from a reference is $4\pi r^2 \rho g(r) dr$; integration of $4\pi r^2 \rho g(r) dr$ up to its first minimum provides an estimate of the “coordination number”. In more practical terms, the radial distribution function is computed as a histogram of frequencies, i.e. how often one finds two particles at a certain distance, normalized by the factor $1/(N\rho 4\pi r^2 dr)$. Another way to interpret the radial distribution function is as the relative “excess” of particles, with respect to the ideal gas at the same density. In this perspective, it is thus more correct to divide by the volume of the spherical shell between r and $r + dr$ rather than by $4\pi r^2 dr$; however, the discrepancy is sufficiently small in most cases to be negligible.

Finally, in reciprocal space, one can compute the structure factor, defined as

$$S(\mathbf{k}) = \left\langle \frac{1}{N} \rho_{\mathbf{k}} \rho_{-\mathbf{k}} \right\rangle \quad (2.25)$$

where $\rho_{\mathbf{k}}$ is a Fourier component of the microscopic density

$$\rho_{\mathbf{k}} = \int \rho(\mathbf{r}) \exp(-i\mathbf{k} \cdot \mathbf{r}) d\mathbf{r} = \sum_{i=1}^N \exp(-i\mathbf{k} \cdot \mathbf{r}_i) \quad (2.26)$$

More in practice

$$S(\mathbf{k}) = \left\langle \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \exp(-i\mathbf{k} \cdot \mathbf{r}_i) \exp(i\mathbf{k} \cdot \mathbf{r}_j) \right\rangle \quad (2.27)$$

For a homogeneous monodisperse (or single component) fluid, the structure factor depends on the magnitude k of the wave vector $\mathbf{k}$ and can be computed as the Fourier transform of the radial distribution function

$$S(k) = 1 + \rho \int g(r) \exp(-i\mathbf{k} \cdot \mathbf{r}) d\mathbf{r} = 1 + \rho \int r^2 g(r) \frac{\sin(kr)}{kr} dr \quad (2.28)$$

2.3 Integration schemes

We discuss, in this section, some of the algorithms that can be used to integrate Newton's equations of motion for an N -body system. One common technique develops low order solutions using the time Taylor series expansion of the position and the velocity $\vec{r}_i, \vec{v}_i$ around $\Delta t = 0$. A more general technique, called Trotter expansion, uses the so-called Liouville operator. Other integration schemes may be used and we will briefly discuss advantages and disadvantages.

2.3.1 Euler algorithm

Consider a Taylor expansion of the particle trajectories, truncated to the third order, so that we retain all the terms with a physical significance

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + O(\Delta t^3) \quad (2.29)$$

This is the so-called Euler algorithm. It is much worse with respect to all the other algorithms that will be derived later on virtually all accounts. In particular, it is not reversible or area preserving and suffers from severe energy drifts. It is not recommended.

2.3.2 Verlet Algorithm

The simplest (but good) integration scheme is the Verlet algorithm. Consider the Taylor expansion up to the fourth order for $\vec{r}_i(t + \Delta t)$ and $\vec{r}_i(t - \Delta t)$:

$$\begin{aligned} \vec{r}_i(t + \Delta t) &= \vec{r}_i(t) + \vec{v}_i(t)\Delta t + \frac{1}{2}\vec{a}_i(t)\Delta t^2 + \frac{1}{6}\frac{d^3\vec{r}_i(t)}{dt^3}\Delta t^3 + O(\Delta t^4) \\ \vec{r}_i(t - \Delta t) &= \vec{r}_i(t) - \vec{v}_i(t)\Delta t + \frac{1}{2}\vec{a}_i(t)\Delta t^2 - \frac{1}{6}\frac{d^3\vec{r}_i(t)}{dt^3}\Delta t^3 + O(\Delta t^4) \end{aligned} \quad (2.30)$$

with $\vec{v}_i(t) = d\vec{r}_i/dt$ and $\vec{a}_i(t) = d^2\vec{r}_i/dt^2$ that, when summed up and rearranged, gives

$$\vec{r}_i(t + \Delta t) = 2\vec{r}_i(t) - \vec{r}_i(t - \Delta t) + \vec{a}_i(t)\Delta t^2 + O(\Delta t^4) \quad (2.31)$$

The positions are accurate up to the fourth order in Δt . Note that the velocities are not calculated explicitly in this procedure but can be derived knowing the trajectory

$$\vec{v}_i(t) = \frac{\vec{r}_i(t + \Delta t) - \vec{r}_i(t - \Delta t)}{2\Delta t} + O(\Delta t^2) \quad (2.32)$$

which is accurate only up to order Δt^2 . Hence, quantities depending on the velocity are not so accurate. Further, the position of the particles at $t + \Delta t$ depends on the position at time t as well as at time $t - \Delta t$. As such, we have a problem of initialization: we cannot directly apply Eq. (2.31) using standard initial conditions, i.e. a set of coordinates and velocities. As for other "high order" numerical solvers, the problem is solved by using lower order solvers for the first step, such as, given $\vec{r}_i(0)$ and $\vec{v}_i(0)$,

$$\vec{r}_i(\Delta t) \approx \vec{r}_i(0) + \vec{v}_i(0)\Delta t + \frac{\vec{a}_i(0)}{2}\Delta t^2 \quad (2.33)$$

where $\vec{a}_i(0)$ should be computed from the coordinates $\vec{r}_i(0)$. Once this approximate step is done, the forces (hence $\vec{a}_i(\Delta t)$) can be calculated and Eq. (2.31) can be used iteratively. The technique just presented turns

out to be appropriate for obtaining algorithms that are accurate to second order in time for Hamiltonian systems and that preserves the time reversal symmetry of the equation of motions. If, on the other hand, higher order in accuracy is requested, this technique requires spatial derivatives of forces that are in general computationally expensive. Moreover, when applied to non-Hamiltonian systems the Taylor series approach generally yields schemes which do not preserve the invariant phase space measure.

2.3.3 Leap Frog Algorithm

Several algorithm are equivalent to the Verlet scheme. The simplest among these is the so-called Leap Frog algorithm, that evaluates the velocity at half-integer steps and uses these velocities to compute the new positions. To derive the algorithm from the Verlet scheme, we start by defining the velocities at half-integer time steps as follows:

$$\begin{aligned}\vec{v}(t - \Delta t/2) &\equiv \frac{\vec{r}(t) - \vec{r}(t - \Delta t)}{\Delta t} \\ \vec{v}(t + \Delta t/2) &\equiv \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}\end{aligned}\quad (2.34)$$

From the latter equation we obtain an expression for the new positions, based on the old positions and velocities

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t + \Delta t/2)\Delta t \quad (2.35)$$

and from the Verlet algorithm we get an expression for the update of the velocities

$$\vec{v}(t + \Delta t/2) = \vec{v}(t - \Delta t/2) + \frac{\vec{f}(t)}{m}\Delta t \quad (2.36)$$

As the Leap Frog algorithm is derived from the Verlet, it gives rise to identical trajectories. Note that the velocities are not defined at the same time as the positions. This means that kinetic and potential energy are not defined at the same time, and hence we cannot compute the total energy directly in the Leap From scheme.

2.3.4 Velocity Verlet Algorithm

An alternative recast of the algorithm, where positions and velocity are simultaneously updated, is the Velocity Verlet. This algorithm will look like a Taylor expansion, with a different update of the velocity with respect to the Euler scheme.

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{\vec{f}(t)}{2m}\Delta t^2 \quad (2.37)$$

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{\vec{f}(t) + \vec{f}(t + \Delta t)}{2m}\Delta t \quad (2.38)$$

Note that we compute the new velocities only after we have computed the new positions and, from these, the new forces.

The velocity Verlet can be translated into the following pseudo-code:

- A momentum translation by an amount (**half a kick**)

$$\vec{p}(t) + \frac{\Delta t}{2}\vec{f}(t) \longrightarrow \vec{p}(t + \Delta t/2) \quad (2.39)$$

- A position translation by an amount (**drift**)

$$\vec{r}(t) + \Delta t \vec{v}(t + \Delta t/2) = \vec{r}(t) + \frac{\vec{p}(t)}{m} \Delta t + \frac{\vec{f}(t)}{2m} \Delta t^2 \quad (2.40)$$

using the momentum obtained in step 1.

- Computation of the forces for $\vec{r} = \vec{r}(t + \Delta t)$.
- A momentum translation by an amount (**half a kick**)

$$\vec{p}(t + \Delta t/2) + \frac{\Delta t}{2} \vec{f}(t + \Delta t) \longrightarrow \vec{p}(t + \Delta t) \quad (2.41)$$

The physical interpretation is that two impulses of equal size are imparted to the particles just before and just after each time t and that between these time points the particles drift with constant velocity.

We will derive the algorithm later; we show now that the scheme is equivalent to the Verlet algorithm. Note that

$$\vec{r}(t + 2\Delta t) = \vec{r}(t + \Delta t) + \vec{v}(t + \Delta t) \Delta t + \frac{\vec{f}(t + \Delta t)}{2m} \Delta t^2 \quad (2.42)$$

and Eq. (2.37) can be written as

$$\vec{r}(t) = \vec{r}(t + \Delta t) - \vec{v}(t) \Delta t - \frac{\vec{f}(t)}{2m} \Delta t^2 \quad (2.43)$$

and by addition we have

$$\vec{r}(t + 2\Delta t) + \vec{r}(t) = 2\vec{r}(t + \Delta t) + (\vec{v}(t + \Delta t) - \vec{v}(t)) \Delta t + \frac{\vec{f}(t + \Delta t) - \vec{f}(t)}{2m} \Delta t^2 \quad (2.44)$$

Substitution of Eq.(2.38) yields the Verlet algorithm.

2.3.5 Velocity Verlet Algorithm: Trotter expansion

A different approach is based on the Liouville operator formulation of classical mechanics and it is the one we will discuss here. Let us focus first on Hamiltonian systems only. We now that these systems are characterized by certain fundamental properties such as the symplectic property and time reversibility. For simplicity let us begin by examining a simple one-particle system in one dimension for which the equation of motion are

$$\dot{q} = \frac{\partial H}{\partial p}, \quad \dot{p} = -\frac{\partial H}{\partial q} = F(q(t)). \quad (2.45)$$

Recall that any function $O(q, p)$ evolves in time according to

$$\frac{dO}{dt} = \{O, H\} \quad (2.46)$$

where the Poisson brackets are

$$\{O, H\} = \frac{\partial O}{\partial q} \frac{\partial H}{\partial p} - \frac{\partial O}{\partial p} \frac{\partial H}{\partial q}. \quad (2.47)$$

Clearly, if $O(q, p) = q$ we have

$$\frac{dq}{dt} = \dot{q} = \{q, H\} = \frac{\partial q}{\partial q} \frac{\partial H}{\partial p} - \frac{\partial q}{\partial p} \frac{\partial H}{\partial q} = \frac{\partial H}{\partial p} \quad (2.48)$$

since q does not depend on p . Similarly, if $O(q, p) = p$

$$\frac{dp}{dt} = \dot{p} = \{p, H\} = \frac{\partial p}{\partial q} \frac{\partial H}{\partial p} - \frac{\partial p}{\partial p} \frac{\partial H}{\partial q} = -\frac{\partial H}{\partial q} \quad (2.49)$$

The Hamiltonian equations can be cast in a more compact form by defining the two-dimensional phase space $\Gamma = (q, p)$:

$$\frac{d\Gamma}{dt} = \{\Gamma, H\}. \quad (2.50)$$

One can then define an operator $\hat{L}$ acting on Γ by

$$i\hat{L}\Gamma = \{\Gamma, H\}. \quad (2.51)$$

The operator $\hat{L}$ is known as the Liouville operator i.e a differential operator given by

$$i\hat{L} = \frac{\partial H}{\partial p} \frac{\partial}{\partial q} - \frac{\partial H}{\partial q} \frac{\partial}{\partial p}. \quad (2.52)$$

For the the more general case of a N particle system in $d = 1$ the phase space $\Gamma = (q_1, p_1, \dots, q_N, p_N)$ and

$$i\hat{L} = \sum_{k=1}^N \left[\frac{\partial H}{\partial p_k} \frac{\partial}{\partial q_k} - \frac{\partial H}{\partial q_k} \frac{\partial}{\partial p_k} \right]. \quad (2.53)$$

With the Liouville operator the equations of motions can be written in operator form as

$$\frac{d\Gamma}{dt} = i\hat{L}\Gamma \quad (2.54)$$

whose formal solution is

$$\Gamma(t) = e^{i\hat{L}t}\Gamma(0). \quad (2.55)$$

where $\Gamma(0) = (q_1(0), p_1(0), \dots, q_N(0), p_N(0))$. The *unitary* operator $\exp(i\hat{L}t)$ is called the *classical propagator* and the presence of i furnishes an immediate analogy with the quantum mechanical propagator $\exp(-i\hat{H}t/\hbar)$. Unfortunately Eq. (2.55) is not of very practical use since the analytical evaluation of $\exp(i\hat{L}t)$ can be done for very few simple cases. This can be seen by first noting that, according to eq. (2.52), $i\hat{L}$ is (for one particle system) of the form

$$i\hat{L} = i\hat{L}_1 + i\hat{L}_2 \quad (2.56)$$

where

$$i\hat{L}_1 = \frac{\partial H}{\partial p} \frac{\partial}{\partial q} = \dot{q} \frac{\partial}{\partial q} \quad i\hat{L}_2 = -\frac{\partial H}{\partial q} \frac{\partial}{\partial p} = \dot{p} \frac{\partial}{\partial p} \quad (2.57)$$

where in the last terms we have used the Hamiltonian equations of motion. Hence, for any phase space function $g(q, p)$ it follows that

$$(i\hat{L}_2)(i\hat{L}_1)g(q, p) = -\frac{\partial H}{\partial q} \frac{\partial}{\partial p} \left[\frac{\partial H}{\partial p} \frac{\partial g}{\partial q} \right] = -\frac{\partial H}{\partial q} \frac{\partial^2 H}{\partial p^2} \frac{\partial g}{\partial q} - \frac{\partial H}{\partial q} \frac{\partial H}{\partial p} \frac{\partial^2 g}{\partial p \partial q} \quad (2.58)$$

$$(i\hat{L}_1)(i\hat{L}_2)g(q, p) = \frac{\partial H}{\partial p} \frac{\partial}{\partial q} \left[-\frac{\partial H}{\partial q} \frac{\partial g}{\partial p} \right] = -\frac{\partial H}{\partial p} \frac{\partial^2 H}{\partial q^2} \frac{\partial g}{\partial p} - \frac{\partial H}{\partial p} \frac{\partial H}{\partial q} \frac{\partial^2 g}{\partial p \partial q}. \quad (2.59)$$

Therefore, for any phase space function $g(q, p)$

$$(i\hat{L}_1 i\hat{L}_2 - i\hat{L}_2 i\hat{L}_1)g(q, p) \neq 0 \quad (2.60)$$

or

$$[i\hat{L}_1, i\hat{L}_2] \neq 0 \quad (2.61)$$

Since $i\hat{L}_1$ and $i\hat{L}_2$ do not commute, the classical propagator $\exp(i\hat{L}t)$ *cannot* be factorized as $\exp(i\hat{L}_1t) \exp(i\hat{L}_2t)$. In particular by Taylor expanding the two expressions it is possible to show that

$$e^{i\hat{L}t} = e^{i\hat{L}_1t} e^{i\hat{L}_2t} e^{i[\hat{L}_1, \hat{L}_2]t}. \quad (2.62)$$

On the other hand there is a theorem that can be use to factorize the classical propagator. This is the *Trotter theorem*, which states that

$$\exp(i\hat{L}t) = \exp[(i\hat{L}_1 + i\hat{L}_2)t] = \lim_{M \rightarrow \infty} \left[\exp\left(\frac{i\hat{L}_2t}{2M}\right) \exp\left(\frac{i\hat{L}_1t}{M}\right) \exp\left(\frac{i\hat{L}_2t}{2M}\right) \right]^M. \quad (2.63)$$

The Trotter theorem is the starting point for generating practical numerical integrators. Indeed let us consider the approximation to $\exp[(i\hat{L}_1 + i\hat{L}_2)t]$ obtained by choosing M large but still finite:

$$\exp[(i\hat{L}_1 + i\hat{L}_2)t] \approx \left[\exp\left(\frac{i\hat{L}_2t}{2M}\right) \exp\left(\frac{i\hat{L}_1t}{M}\right) \exp\left(\frac{i\hat{L}_2t}{2M}\right) \right]^M. \quad (2.64)$$

By taking the $1/M$ power in both sides we have

$$\exp[(i\hat{L}_1 + i\hat{L}_2)t/M] \approx \exp\left(\frac{i\hat{L}_2t}{2M}\right) \exp\left(\frac{i\hat{L}_1t}{M}\right) \exp\left(\frac{i\hat{L}_2t}{2M}\right). \quad (2.65)$$

Defining $\Delta t = t/M$ for M finite we obtain a single-time-step approximation to the propagator of the form

$$\exp[(i\hat{L}_1 + i\hat{L}_2)\Delta t] \approx \exp\left(\frac{i\hat{L}_2\Delta t}{2}\right) \exp(i\hat{L}_1\Delta t) \exp\left(\frac{i\hat{L}_2\Delta t}{2}\right). \quad (2.66)$$

Let $\tilde{U}(\Delta t)$ be the right side of the previous equation. It represents an approximation of the classical propagator. It is easy to see that $\tilde{U}^\dagger(\Delta t) = \tilde{U}(-\Delta t) = \tilde{U}^{-1}(-\Delta t)$. Hence, $\tilde{U}(\Delta t)$ is *unitary* and therefore it preserves the time-reversibility of the dynamics. Moreover, by performing a Taylor expansion, it can be shown that $\tilde{U}(\Delta t)$ is accurate up to order Δt^2 :

$$\begin{aligned} & \exp\left(\frac{i\hat{L}_2\Delta t}{2}\right) \exp(i\hat{L}_1\Delta t) \exp\left(\frac{i\hat{L}_2\Delta t}{2}\right) \\ &= \left[1 + \frac{\Delta t}{2}i\hat{L}_2 + \frac{\Delta t^2}{8}(i\hat{L}_2)^2 \dots \right] + \left[1 + \Delta ti\hat{L}_1 + \frac{\Delta t^2}{2}(i\hat{L}_2)^2 \dots \right] + \left[1 + \frac{\Delta t}{2}i\hat{L}_2 + \frac{\Delta t^2}{8}(i\hat{L}_2)^2 \dots \right] \\ &= 1 + (i\hat{L}_1 + i\hat{L}_2)t + \frac{1}{2} \left[(i\hat{L}_1)(i\hat{L}_2) + (i\hat{L}_2)(i\hat{L}_1) + \frac{1}{2}(i\hat{L}_2)^2 + \frac{1}{4}(i\hat{L}_2)^2 + (i\hat{L}_1)^2 + \frac{1}{4}(i\hat{L}_2)^2 \right] \Delta t^2 + \dots \\ &= 1 + i\hat{L}\Delta t + \frac{1}{2}(i\hat{L}_1 + i\hat{L}_2)^2 \Delta t^2 + \dots \end{aligned} \quad (2.67)$$

Hence there is equivalence up to second order in Δt i.e.

$$\exp(i\hat{L}\Delta t) = \exp\left(\frac{i\hat{L}_2\Delta t}{2}\right) \exp(i\hat{L}_1\Delta t) \exp\left(\frac{i\hat{L}_2\Delta t}{2}\right) + O(\Delta t^3). \quad (2.68)$$

We now look at the action of the approximate Liouville operator on the phase space (q, p) . First let us remind the identity

$$e^{c \frac{\partial}{\partial q}} q = q + c \quad (2.69)$$

that can be obtained by simply Taylor expanding the left side :

$$\left[1 + c \frac{\partial}{\partial q} + \frac{1}{2} c^2 \frac{\partial^2}{\partial q^2} + \dots \right] q = q + c \quad (2.70)$$

since all derivatives higher than the first order derivative go to 0 when acting on q . This identity can be used to obtain a more one:

$$e^{c \frac{\partial}{\partial q}} g(q, p) = g(q + c, p). \quad (2.71)$$

Indeed

$$e^{c \frac{\partial}{\partial q}} g(q, p) = \sum_{k=0}^{\infty} \frac{1}{k!} \left[c \frac{\partial}{\partial q} \right]^k g(q, p) = \sum_{k=0}^{\infty} c^k g^{(k)}(q, p) \quad (2.72)$$

and the last line is just the Taylor series expansion of $g(q + c, p)$. Similar identities are valid for the action of $\exp(c\partial/\partial p)$ on p and on $h(q, p)$. For simplicity let us now consider the classical Hamiltonian

$$H = \frac{p^2}{2m} + U(q) \quad (2.73)$$

In this case $\partial H/\partial p = p/m$ and $\partial H/\partial q = -dU/dq = F(q)$. With the above identities it is possible to evaluate the action of $\tilde{U}(\Delta t)$ on q :

$$\tilde{U}(\Delta t)q = e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} e^{\Delta t \frac{\partial H}{\partial p} \frac{\partial}{\partial q}} e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} q = e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} e^{\Delta t \frac{\partial H}{\partial p} \frac{\partial}{\partial q}} q \quad (2.74)$$

and since the second operator changes q to $q + \Delta t \frac{\partial H}{\partial p}$ we get:

$$\tilde{U}(\Delta t)q = e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} \left(q + \Delta t \frac{\partial H}{\partial p} \right). \quad (2.75)$$

Since the last operator has no effect on q but acts on p by changing it into $p - \frac{\Delta t}{2} \frac{\partial H}{\partial q} = p + \frac{\Delta t}{2} \dot{p}$ we get

$$\begin{aligned} \tilde{U}(\Delta t)q &= q + \frac{\Delta t}{m} \left(p - \frac{\Delta t}{2} \frac{\partial H}{\partial q} \right) \\ &= q + \Delta t \frac{p}{m} + \frac{\Delta t^2}{2m} F(q) \end{aligned} \quad (2.76)$$

Starting from the initial conditions $(q(0), p(0))$ we then have

$$\tilde{U}(\Delta t)q(0) \equiv q(\Delta t) = q(0) + \Delta t \frac{p(0)}{m} + \frac{\Delta t^2}{2m} F(q(0)). \quad (2.77)$$

The action of $\tilde{U}(\Delta t)$ on p is

$$\tilde{U}(\Delta t)p = e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} e^{\Delta t \frac{\partial H}{\partial p} \frac{\partial}{\partial q}} e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} p. \quad (2.78)$$

The first operator changes p to $p - \frac{\Delta t}{2} \frac{\partial H}{\partial q} = p + \frac{\Delta t}{2} \dot{p} = p + \frac{\Delta t}{2} F(q)$ giving

$$\tilde{U}(\Delta t)p = e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} e^{\Delta t \frac{\partial H}{\partial p} \frac{\partial}{\partial q}} \left(p + \frac{\Delta t}{2} F(q) \right). \quad (2.79)$$

The next operator changes $F(q)$ into $F(q + \Delta t \frac{p}{m})$ and has no effect on p :

$$\tilde{U}(\Delta t)p = e^{-\frac{\Delta t}{2} \frac{\partial H}{\partial q} \frac{\partial}{\partial p}} \left[p + \frac{\Delta t}{2} F \left(q + \frac{\Delta t}{m} p \right) \right]. \quad (2.80)$$

Finally the last operator changes p into $p + \frac{\Delta t}{2} F(q)$ in all the places where it occurs:

$$\begin{aligned} \tilde{U}(\Delta t) &= p + \frac{\Delta t}{2} F(q) + \frac{\Delta t}{2} F \left(q + \frac{\Delta t}{m} \left(p + \frac{\Delta t}{2} F(q) \right) \right) \\ &= p + \frac{\Delta t}{2} \left[F(q) + F \left(q + \frac{\Delta t}{m} p + \frac{\Delta t^2}{2m} F(q) \right) \right] \end{aligned} \quad (2.81)$$

that can be written as

$$p(\Delta t) = p(0) + \frac{\Delta t}{2} [F(q(0)) + F(q(\Delta t))] \quad (2.82)$$

Combining the evolutions for q and p we get

$$\begin{aligned} q(\Delta t) &= q(0) + \Delta t \frac{p(0)}{m} + \frac{\Delta t^2}{2m} F(q(0)) \\ p(\Delta t) &= p(0) + \frac{\Delta t}{2} [F(q(0)) + F(q(\Delta t))] \end{aligned} \quad (2.83)$$

Symplecticity An important property of the map generated by the operator $\tilde{U}(\Delta t)$ is that it preserves the invariant phase space measure. In other words the map is *symplectic*. This is equivalent to say that the Jacobian of the map

$$J((q(\Delta t), p(\Delta t)); (q(0), p(0))) = \frac{\partial (q(\Delta t), p(\Delta t))}{\partial (q(0), p(0))} \quad (2.84)$$

is equal to one. This is easy to show since the Jacobian of the transformation from $\{q(0), p(0)\}$ to $\{q(\Delta t), p(\Delta t)\}$ is the product of the Jacobian of the three transformations

$$e^{\dot{p}\partial/\partial p \Delta t/2} f(q(0), p(0)) = f \left[q(0), \left(p(0) + \frac{\Delta t}{2} \dot{p}(0) \right) \right] \quad (2.85)$$

$$e^{\dot{q}\partial/\partial q \Delta t} f \left(q(0), \left(p(0) + \frac{\Delta t}{2} \dot{p}(0) \right) \right) = f \left[\left(q(0) + \Delta t \dot{q} \left(\frac{\Delta t}{2} \right) \right), \left(p(0) + \frac{\Delta t}{2} \dot{p}(0) \right) \right] \quad (2.86)$$

$$\begin{aligned} &e^{\dot{p}\partial/\partial p \Delta t/2} f \left[\left(q(0) + \Delta t \dot{q} \left(\frac{\Delta t}{2} \right) \right), \left(p(0) + \frac{\Delta t}{2} \dot{p}(0) \right) \right] \\ &= f \left[\left(q(0) + \Delta t \dot{q} \left(\frac{\Delta t}{2} \right) \right), \left(p(0) + \frac{\Delta t}{2} \dot{p}(0) + \frac{\Delta t}{2} \dot{p}(\Delta t) \right) \right] \end{aligned} \quad (2.87)$$

For example for the first transformation the Jacobian is

$$\begin{vmatrix} \frac{\partial p'}{\partial p} & \frac{\partial p'}{\partial q} \\ \frac{\partial q}{\partial p} & \frac{\partial q}{\partial q} \end{vmatrix} = \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} = 1 \quad (2.88)$$

where $p' = p + \frac{\Delta t}{2} \dot{p}$. Note that the symplectic nature of the map ensures that the error of the map is bounded, i.e., there will be no exponential growth in the energy conservation error which can affect the long-time statistical behaviour of the dynamics. The Trotter theorem is powerful not only because it shows directly that the algorithm is symplectic and time-reversible but also because the same evolution can be obtained by viewing the approximate operator $\tilde{U}(\Delta t)$ as a combination of three sequential updates without requiring the closed form expression of eq. (2.83). Indeed

$$\tilde{U}(\Delta t) \equiv \exp \left(\frac{i\hat{L}_2 \Delta t}{2} \right) \exp \left(i\hat{L}_1 \Delta t \right) \exp \left(\frac{i\hat{L}_2 \Delta t}{2} \right) \quad (2.89)$$

can be seen as the combination of

$$\begin{aligned}
 \exp\left(\frac{i\hat{L}_2(q,p)\Delta t}{2}\right)(q,p) &\rightarrow (q',p') \\
 \exp\left(i\hat{L}_1(q',p')\Delta t\right)(q',p') &\rightarrow (q'',p'') \\
 \exp\left(\frac{i\hat{L}_2(q'',p'')\Delta t}{2}\right)(q'',p'') &\rightarrow (q(\Delta t),p(\Delta t))
 \end{aligned} \tag{2.90}$$

If we now use the expressions for $i\hat{L}_1$ and $i\hat{L}_2$ we get :

$$\begin{aligned}
 e^{\frac{\Delta t}{2}\dot{p}\partial/\partial p}\begin{pmatrix} q \\ p \end{pmatrix} &= \begin{pmatrix} q(0) \\ p(0) + \frac{\Delta t}{2}F(q(0)) \end{pmatrix} = \begin{pmatrix} q' \\ p' \end{pmatrix} \\
 e^{\Delta t\dot{q}\partial/\partial q}\begin{pmatrix} q' \\ p' \end{pmatrix} &= \begin{pmatrix} q' + \Delta t\dot{q}' \\ p' \end{pmatrix} = \begin{pmatrix} q'' \\ p' \end{pmatrix} \\
 e^{\frac{\Delta t}{2}\dot{p}\partial/\partial p}\begin{pmatrix} q'' \\ p' \end{pmatrix} &= \begin{pmatrix} q'' \\ p'' + \frac{\Delta t}{2}F(q'') \end{pmatrix} = \begin{pmatrix} q(\Delta t) \\ p(\Delta t) \end{pmatrix}
 \end{aligned} \tag{2.91}$$

Finally, notice that, following this route, Eq. (2.40) gets the more general form

$$q(t) + \Delta t v(t + \Delta t/2) = q(t) + \Delta t \frac{\partial H}{\partial p} \Big|_{p=p(t+\Delta t/2)} \tag{2.92}$$

Time reversibility The condition of being time reversible requires that only even-ordered decomposition must be used and that the operators $e^{\hat{L}_1\Delta t}$ and $e^{\hat{L}_2\Delta t}$ enter symmetrically in the decomposition. For example, the time reversibility property of the second order decomposition

$$e^{(\hat{L}_1+\hat{L}_2)\Delta t} = e^{\hat{L}_2\Delta t/2}e^{\hat{L}_1\Delta t}e^{\hat{L}_2\Delta t/2} \equiv \tilde{U}(\Delta t) \tag{2.93}$$

can be shown by the property

$$\tilde{U}(\Delta t)\tilde{U}(-\Delta t) = e^{\hat{L}_2\Delta t/2}e^{\hat{L}_1\Delta t}e^{\hat{L}_2\Delta t/2}e^{-\hat{L}_2\Delta t/2}e^{-\hat{L}_1\Delta t}e^{-\hat{L}_2\Delta t/2} = 1. \tag{2.94}$$

Similarly, $\tilde{U}(-\Delta t)\tilde{U}(\Delta t) = 1$. Hence each time step in the temporal evolution is reversible leading to a time reversible trajectory.

2.3.6 Other & Higher Order Algorithms

For most Molecular Dynamics applications, Verlet-like algorithms are perfectly adequate. However, higher order algorithms makes it possible to use a longer time step without loss of (short-term) accuracy or they can be helpful if higher accuracy is required.

We first mention two schemes that yield the same trajectories as the Verlet algorithm but provide better estimates for the velocity. The first is the so-called Beeman algorithm

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{4\vec{f}(t) - \vec{f}(t - \Delta t)}{6m}\Delta t^2 \tag{2.95}$$

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{2\vec{f}(t + \Delta t) + 5\vec{f}(t) - \vec{f}(t - \Delta t)}{6m}\Delta t \tag{2.96}$$

By eliminating $\vec{v}(t)$ from Eq. (2.95) using Eq. (2.96), it is easy to show that the positions satisfy the Verlet algorithm. However, the velocities are more accurate than the original Verlet algorithm; as a consequence,

the total energy conservation looks better. A disadvantage of the Beeman algorithm is that the expression for the velocity does not possess time-reversal symmetry. This can be corrected and leads to the so-called velocity-corrected Verlet algorithm for which both position and velocity error is $O(\Delta t^4)$.

It is derived as follows: write a Taylor expansion for $\vec{r}(t + 2\Delta t)$, $\vec{r}(t + \Delta t)$, $\vec{r}(t - \Delta t)$ and $\vec{r}(t - 2\Delta t)$

$$\begin{aligned}\vec{r}(t + 2\Delta t) &= \vec{r}(t) + 2\vec{v}\Delta t + \frac{\vec{a}}{4}(2\Delta t)^2 + \frac{\dot{\vec{a}}}{3!}(2\Delta t)^3 + \dots \\ \vec{r}(t + \Delta t) &= \vec{r}(t) + \vec{v}\Delta t + \frac{\vec{a}}{4}\Delta t^2 + \frac{\dot{\vec{a}}}{3!}\Delta t^3 + \dots \\ \vec{r}(t - \Delta t) &= \vec{r}(t) - \vec{v}\Delta t + \frac{\vec{a}}{4}\Delta t^2 - \frac{\dot{\vec{a}}}{3!}\Delta t^3 + \dots \\ \vec{r}(t - 2\Delta t) &= \vec{r}(t) - 2\vec{v}\Delta t + \frac{\vec{a}}{4}(2\Delta t)^2 - \frac{\dot{\vec{a}}}{3!}(2\Delta t)^3 + \dots\end{aligned}$$

By combining these equations, we can write

$$12\vec{v}(t)\Delta t = 8[\vec{r}(t + \Delta t) - \vec{r}(t - \Delta t)] - [\vec{r}(t + 2\Delta t) - \vec{r}(t - 2\Delta t)] + O(\Delta t^4) \quad (2.97)$$

or, equivalently

$$\vec{v}(t) = \frac{\vec{v}(t + \Delta t/2) + \vec{v}(t - \Delta t/2)}{2} + \frac{\Delta t}{12} [\ddot{\vec{a}}(t - \Delta t) - \ddot{\vec{a}}(t + \Delta t)] + O(\Delta t^4) \quad (2.98)$$

Note that this velocity can be computed only after the next step, i.e. we must know the positions and forces at $t + \Delta t$ to compute the velocity at time t .

In general, higher order schemes require more storage and are either not time reversible or not symplectic (non area preserving). This is true in particular of the so-called predictor-corrector algorithm. The basic idea is to use information about the position and its first n derivatives at time $t + \Delta t$. We then compute the forces (and the accelerations) at the predicted positions; we find that these accelerations are *not* equal to the values that we predicted and, thus, we adjust our predictions. Further, on the basis of the observed discrepancy, we try to improve our estimation of the positions. This latter step is usually the “corrector” part of the algorithm. The one reported is one possible form.

Consider the Taylor expansion (for simplicity, in 1D)

$$x(t + \Delta t) = x(t) + \frac{dx}{dt}\Delta t + \frac{d^2x}{dt^2}\frac{(\Delta t)^2}{2!} + \frac{d^3x}{dt^3}\frac{(\Delta t)^3}{3!} + \dots \quad (2.99)$$

using the short-hand notation: $x_1(t) = \frac{dx}{dt}\Delta t$, $x_2(t) = \frac{d^2x}{dt^2}\frac{\Delta t^2}{2!}$, $x_3(t) = \frac{d^3x}{dt^3}\frac{\Delta t^3}{3!}$, we can write the following predictions for $x(t + \Delta t)$:

$$\begin{aligned}x(t + \Delta t) &= x(t) + x_1(t) + x_2(t) + x_3(t) \\ x_1(t + \Delta t) &= x_1(t) + 2x_2(t) + 3x_3(t) \\ x_2(t + \Delta t) &= x_2(t) + 3x_3(t) \\ x_3(t + \Delta t) &= x_3(t)\end{aligned} \quad (2.100)$$

Having $x(t + \Delta t)$ we can compute the forces at the predicted positions, thus compute $x_2(t + \Delta t)$; we denote $\Delta x_2 \equiv x_2^{\text{corrected}} - x_2^{\text{predicted}}$. We estimate corrected values for x and its derivatives as

$$x_n^{\text{corrected}} = x_n^{\text{predicted}} + C_n \Delta x_2 \quad (2.101)$$

where the C_n are constants, fixed for a given order algorithm. The values are chosen to yield an optimal compromise between accuracy and stability. For example, a fifth order algorithm has: $C_0 = 19/120$, $C_1 =$

$3/4$, $C_2 = 1$, $C_3 = 1/2$, $C_4 = 1/12$.

Finally, higher order integration schemes can be implemented using higher order decomposition, such as the fourth-order Suzuki-Trotter decomposition:

$$e^{(\hat{L}_1 + \hat{L}_2)\Delta t} = \prod_{i=1}^5 e^{p_i \hat{L}_1 \Delta t/2} e^{p_i \hat{L}_2 \Delta t} e^{p_i \hat{L}_1 \Delta t/2} + O(\Delta t^5) \quad (2.102)$$

where $p_1 = p_2 = p_4 = p_5 \equiv p = \frac{1}{4-4^{1/3}}$, $p_3 = 1 - 4p$. It is composed by 15 exponential operators. However, consecutive operators with L_1 in the exponent can be combined yielding to a total of 11 operators. Another fourth-order scheme is based on the so called Forest-Routh decomposition

$$e^{(\hat{L}_1 + \hat{L}_2)\Delta t} = e^{p \hat{L}_1 \Delta t/2} e^{p \hat{L}_2 \Delta t} e^{(1-p) \hat{L}_1 \Delta t/2} e^{(1-2p) \hat{L}_2 \Delta t} e^{(1-p) \hat{L}_1 \Delta t/2} e^{p \hat{L}_2 \Delta t} e^{p \hat{L}_1 \Delta t/2} + O(\Delta t^5) \quad (2.103)$$

where $p = 1/(2 - 2^{2^{1/3}})$. Because this decomposition involves 7 operators instead of 11, the cpu time required for one integration step using eq.(2.103) is less then using eq.(2.102). On the other hand, truncation errors arising from the use of eq. (2.103) are much larger.

2.4 Interaction potentials

In this section, we introduce different interaction potentials, commonly used in Molecular Dynamics simulations. We will first discuss the differences between atomic (colloidal) and molecular (or polymeric/macromolecular) systems; we will also discuss how to compute forces in the case of central potentials and what it means to “truncate and shift”. We will then introduce some potential energy functions, commonly used in Soft Matter. It is important to stress that, in Molecular Dynamics, forces are essential as they enter directly in the algorithm that integrates Newton’s equations of motion. Thus, forces must be computed; on the contrary, computing the potential energy is “optional” as there is no term in Newton’s equations that depends directly on the potential energy. Nevertheless, as mentioned in Sec. 2.2.5, computing the potential energy helps to understand if the system has reached equilibrium and helps discriminating between true equilibrium states and metastable states; it is thus good practice to include its calculation in the code and to look at it as a function of time.

2.4.1 Atomic vs Molecular systems

With the term “atomic” (or “colloidal”) system, one usually refers to systems where the individual units that we want to study are not connected to each other irreversibly. They can interact with each other via some interaction potential, usually a pair potential, i.e. an interaction that is a function of the coordinates of two units; however, the average distance between any two units has no physical meaning³. The shortest distance achievable is usually dictated by the interaction potential; this is meaningful if the potential diverges at short distances. The longest distance is dictated by the simulation box: using periodic boundary conditions, the maximum distance along any axis is half the box size.

In molecular systems, instead, a specific pair of neighboring units form a permanent bond that is rather stable and not easily disrupted. This means that a unit, involved in a bond, cannot find itself too far from its companion. For example, polymers are macromolecules composed by repeated units, called monomers. If we identify each monomer as a unit, the polymer can be represented by a sequence of bonds, usually called the **backbone**. Depending on the model, the distance between neighbouring monomers can be fixed or can fluctuate: however, the average distance has always a physical meaning, i.e. the bond length, and is fundamental to describe the physics of polymers, often posing as the unit of length.

While in Monte Carlo it is relatively easy to provide moves that keep the bond length fixed, in Molecular Dynamics such a task is rather cumbersome. It is much more efficient and, in the majority of context, equivalent, to use intramolecular bonding interactions. These are usually described by simple terms, such as the harmonic potential (see Sec. 2.4.5). In addition, physical constraints or other kinds of interactions may induce rigidity of the backbone, i.e. there is an energy penalty when the conformation of neighboring bonds is not the preferred one. In this case, a bending interaction can be included (see Sec. 2.4.7).

Typically, these quantities are obtained from experimental data, or from first principles quantum mechanical calculations. These potentials often aim at describing (small) oscillations of the various parameters around their equilibrium values. In addition, atoms which are not bonded can still interact by means of pair interactions, if needed.

Finally, it is worth noting a feature that distinguishes pair interaction from bonding ones. Pair interaction vanishes at infinity; instead, bonded interaction grows at infinity, as their purpose is to keep the bonded pairs at the desired distance.

2.4.2 Calculating forces from central potentials

If the interaction potential (pair or bond) depends on the distance between two units, there is a simple recipe to compute the forces. It boils down to the chain rule. We need the force in cartesian coordinates $\mathbf{f} = (f_x, f_y, f_z)$.

³As mentioned in Section 2.2.5, the distribution of the pair distances carries physical insight.

If the force comes from a potential $V(r)$ that depends only on the distance r , then

$$\mathbf{f} = (f_x, f_y, f_z) = -\nabla V(r) = -\frac{dV(r)}{dr} \nabla r \quad (2.104)$$

where r is the magnitude of the vector between two units. Taking, for example, particle i and j , with coordinates $\mathbf{r}_i = (x_i, y_i, z_i)$ and $\mathbf{r}_j = (x_j, y_j, z_j)$, then $r = r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$. Thus, Eq. (2.104) becomes, for particle i

$$\mathbf{f}^{(i,j)} = (f_x^{(i)}, f_y^{(i)}, f_z^{(i)}) = -\frac{dV(r_{ij})}{dr_{ij}} \nabla r_{ij} = -\frac{dV(r_{ij})}{dr_{ij}} \left(\frac{\partial r_{ij}}{\partial x_i}, \frac{\partial r_{ij}}{\partial y_i}, \frac{\partial r_{ij}}{\partial z_i} \right) \quad (2.105)$$

and for particle j

$$\mathbf{f}^{(j,i)} = (f_x^{(j)}, f_y^{(j)}, f_z^{(j)}) = -\frac{dV(r_{ij})}{dr_{ij}} \nabla r_{ij} = -\frac{dV(r_{ij})}{dr_{ij}} \left(\frac{\partial r_{ij}}{\partial x_j}, \frac{\partial r_{ij}}{\partial y_j}, \frac{\partial r_{ij}}{\partial z_j} \right) \quad (2.106)$$

We focus, for simplicity, on the x component of the force for particle i

$$f_x^{(i,j)} = -\frac{dV(r_{ij})}{dr_{ij}} \frac{\partial r_{ij}}{\partial x_i} = -\frac{dV(r_{ij})}{dr_{ij}} \frac{\partial r_{ij}}{\partial x_i} = -\frac{dV(r_{ij})}{dr_{ij}} \frac{2(x_i - x_j)}{2r_{ij}} = -\frac{dV(r_{ij})}{dr_{ij}} \frac{(x_i - x_j)}{r_{ij}} \quad (2.107)$$

from which

$$\mathbf{f}^{(i,j)} = -\frac{dV(r_{ij})}{dr_{ij}} \left(\frac{(x_i - x_j)}{r_{ij}}, \frac{(y_i - y_j)}{r_{ij}}, \frac{(z_i - z_j)}{r_{ij}} \right) \quad (2.108)$$

Notice that, by construction, Newton's third law is enforced, as $\mathbf{f}^{(j,i)} = -\mathbf{f}^{(i,j)}$. Given the functional form of $V(r)$, it is thus sufficient to plug the (usually simple) derivative with respect to r in Eq. (2.108) to obtain the corresponding forces.

2.4.3 Truncate and shift an interaction potential

In Molecular Dynamics, forces are the essential ingredient needed to integrate Newton's equations, not the interaction energy they derive from. Thus, any operation on the interaction potential that does not affect its derivatives is allowed. A common practice is cutting the potential energy at some distance, usually called *cutoff* distance r_c . This does come with some caveats. Unless specifically constructed, the interaction potential is defined over an infinite range, $(0, \infty)$. However, as mentioned, simulations are finite and they approximate "infinite" (bulk) systems via periodic boundaries; one has, effectively, infinite copies of the same configuration. In practice, there are three main cases:

1. the potential decays very slowly (e.g. $1/r$). These are usually called *long range* potential and are treated with special techniques (called long-range solver). We will not discuss this case in detail. One can introduce a cut-off distance; however the number of particles grows as r^2 and, thus, the contributions of particles that are very far remain relevant. As such, typically, the truncation errors are large and create artefacts.
2. the potential decays as a power law (at least faster than $1/r^2$). In this case, a cut-off distance can be introduced; however, one should be careful, as results may quantitatively depend on the cut-off distance and, in some instances, corrective terms should be included when computing quantities such as pressure or total energy, even when adopting the most conservative cut-off choice.
3. the potential decays very fast (e.g. exponentially). It is usually safe to use a sensible cut-off distance with these potentials.

The difference between cases 2 and 3 is, for the most part, merely for the sake of the presentation and, in both cases, the choice of the cut-off distance should be pragmatic. The most conservative pick is setting r_c as half of the box size, i.e. the maximum distance possible along a given axis. As said, even in this case, corrective terms should be included when computing quantities such as the pressure; however, this is relevant only when one is interested in studying the thermodynamics of the system. Effectively, such a choice for r_c removes any advantage of using optimisation techniques such as cell lists (see Sec. 2.6). In most cases, a good practice for picking a sensible cut-off is to arbitrarily set a lower value for the force (or the energy). For example, one may decide to disregard the contributions of the particles beyond a certain distance if the absolute value of the force at said distance becomes smaller than, say, 10^{-4} units of force. Such a lower limit should be reasonably small, otherwise a spurious discontinuity in the forces (and in the energy) is introduced, that can render the system less stable (from an algorithmic perspective) as well as lead to artifacts. Clearly, once the lower limit is fixed, the actual value of r_c will depend on the potential used and on the value of the parameters. In this sense, the choice should be pragmatic and try to balance accuracy with performance losses, due to an increase in the number of (potentially) irrelevant neighbours.

Shifting the potential may be useful in particular instances. First, a shift should be introduced whenever a cut-off distance is set, so that the potential is zero at r_c . However, as just discussed, this does not alleviate the issues that comes from a bad choice of r_c . Secondly, in some cases it is useful to cut the potential energy at its minimum (if such a minimum exists). In that case, the force at the cut-off distance is, by definition, zero and the cut does not introduce, per se, any artefact. By shifting the potential, we obtain again a potential energy that is continuous.

2.4.4 Lennard-Jones potential

The Lennard-Jones (LJ) potential

$$V_{LJ}(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right] \quad (2.109)$$

is a widely employed interaction term. The potential has a short range repulsive term $\sim 1/r^{12}$ and a long range attractive term $\sim 1/r^6$. The attractive term $\sim 1/r^6$ represents van der Waals interactions. The choice of a power $\sim 1/r^{12}$ at short distances stems from Pauli's exclusion principle. However, the exponent $\sim 1/r^{12}$ has no theoretical justification and was chosen exclusively because of ease of computation (it is the square of the attractive term). The Lennard-Jones potential approximates very well interaction in heavier rare gases as Ar or Kr whose interactions are dominated by van der Waals forces and electrons are in closed shells. The Lennard-Jones potential has also played a central role in the development of molecular dynamics methods and has been widely used to investigate fundamental issues as phase transitions, the effect of surfaces, small clusters etc. It is also easily generalizable

$$V_{LJ}(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^m - \left(\frac{\sigma}{r} \right)^n \right] \quad (2.110)$$

with m, n integers and $m > n$. Being power-law, the potential can be truncated and approximated with

$$\hat{V}_{LJ}(r) = \begin{cases} V_{LJ}(r) - V_{LJ}(r_c) & \text{if } r \leq r_c \\ 0 & \text{if } r > r_c \end{cases} \quad (2.111)$$

One can also cut the LJ potential at its minimum, thus obtaining the so-called cutted-and-shifted LJ potential or WCA (Weeks-Chandler-Anderson) potential

$$V_{WCA}(r) = \begin{cases} V_{LJ}(r) + \epsilon = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 + \frac{1}{4} \right] & \text{if } r \leq r_c \\ 0 & \text{if } r > r_c \end{cases} \quad (2.112)$$

as $V_{LJ}(r_c) = -\varepsilon$ if r_c is at the minimum. The position of the minimum can be computed analytically and it reads $r_{min} = \sqrt[6]{2}\sigma$. For a generalised LJ potential, the minimum is at $r_{min} = \sqrt[m-n]{m/n}\sigma$; in general, as m and n increase, the minimum shifts towards σ .

2.4.5 Harmonic Potential

The harmonic potential is the prototypical interaction potential for a permanent bond

$$V_H(r) = \frac{1}{2}k(r - r_0)^2 \quad (2.113)$$

where r_0 denotes the equilibrium value of the bond length. As usual, the harmonic potential represents a good approximation of a permanent chemical bond if we expect small fluctuations around the minimum or if we set the stiffness of the spring k to be in this limit. Two small caveats: one should always look at the bond length distribution when employing the harmonic potential, as strong forces can achieve unreasonable stretching, that could be unphysical and could lead to artifacts. On the other hand, setting a very large k may cause strong fluctuations and algorithmic instability. Notably, if cutted at the minimum, the harmonic potential can be used as a pair potential.

2.4.6 FENE potential

The FENE (finite extensible nonlinear elastic) potential is a popular non-harmonic bonding potential for simulating polymers. It reads

$$V_{FENE} = \begin{cases} -0.5KR_0^2 \ln[1 - (r/R_0)^2], & r \leq R_0 \\ \infty, & \text{else} \end{cases} \quad (2.114)$$

where R_0 and K are two parameters: R_0 set the maximum extension while K is, effectively, the “spring” constant. Notice, however, that V_{FENE} goes to zero as r goes to zero and, further, it’s a monotonic function of r ; thus, it does not select any length scale. Further, it would be unphysical to represent a monomer without an excluded volume, because of Pauli’s exclusion principle. For this reasons, it is paired with the WCA potential and the whole bonding potential is computed as $V_{bond} = V_{WCA} + V_{FENE}$; while the parameters R_0 and K can be chosen independently, the standard choice is to set σ and ε as the units of length and energy and to set $R_0 = 1.5\sigma$ and $K = 30\varepsilon$. This is usually called the Kremer-Grest model and represents an optimal compromise between algorithmic stability and a reliable (albeit simple) polymer model.

2.4.7 Bending potential

A final example involves a potential that does not depend on the mutual distance. Indeed, the bending potential usually depends on the angle θ between two consecutive bonds

$$\theta_{ijk} = \arccos\left(\frac{\mathbf{b}_{j,i} \cdot \mathbf{b}_{k,j}}{|\mathbf{b}_{j,i}| |\mathbf{b}_{k,j}|}\right) \quad (2.115)$$

where i, j and k are the three consecutive atoms and $\mathbf{b}_{j,i} = \mathbf{r}_j - \mathbf{r}_i$.

Typical functional forms for a bending potential could be harmonic

$$V(\theta) = \frac{1}{2}k_b(\theta - \theta_0)^2 \quad (2.116)$$

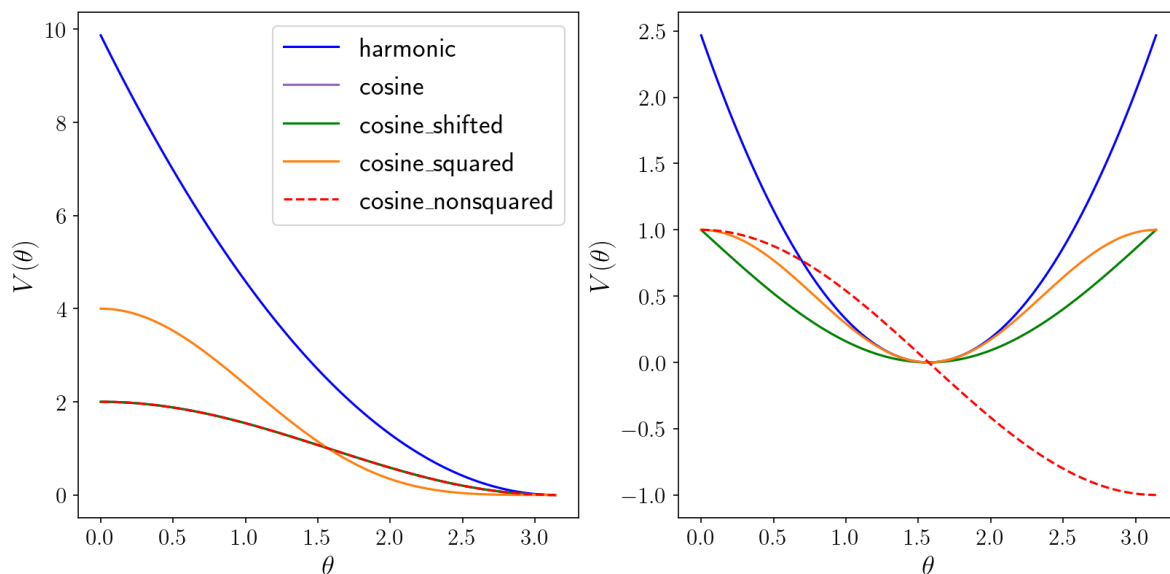


Figure 2.1: Plot of simple bending potentials Eqs (2.116)-(2.120). For all plots, $k_b = 1$ units of energy. Left panel: $\theta_0 = \pi$, right panel: $\theta_0 = \pi/2$. For the potential Eq. (2.118), the potential has been shifted for ease of visualisation by $+2k$ units of energy.

where θ_0 is the preferred value and k_b is the associated “spring” constants. Other possible choices are the “cosine”

$$V(\theta) = k_b(1 + \cos(\theta)) \quad (2.117)$$

and its shifted version

$$V(\theta) = -k_b(1 + \cos(\theta - \theta_0)) \quad (2.118)$$

or the more peculiar “cosine squared”

$$V(\theta) = k_b(\cos(\theta) - \cos(\theta_0))^2 \quad (2.119)$$

The “cosine” potential is specific for modeling the bending rigidity, i.e. the property of a macromolecule to display straight segments, whereas its shifted version can be used for an arbitrary bending angle θ_0 . The difference between the different choices is displayed in Fig. 2.1: the left panel reports the potentials for $\theta_0 = \pi$, the right panel for $\theta_0 = \pi/2$. In all cases, $k = 1$ unit of energy. As expected, they become notably different from each other far from θ_0 . One is tempted to also use the “non-squared” version of Eq. (2.119), i.e.

$$V_{bend}(\theta) = k_b(\cos(\theta) - \cos(\theta_0)) \quad (2.120)$$

because, for $\theta_0 = \pi$, it is equivalent to Eq. (2.117). However, it lacks the correct symmetry and, for $\theta_0 \neq \pi$, it would not display the correct minimum. Bending potentials can become quite complicated as they try to reproduce the complex landscapes of different orientations observed, for example, in proteins, where different minima are present. Thus, multiple cosines or even multiple Gaussians may be employed.

Forces We report here how to implement the forces for the potential Eq. (2.117). In general, the procedure is rather straightforward: (i) write down the potential, explicitly writing the dependence from the cartesian coordinates (ii) take partial derivatives. The procedure is also rather tedious, complicated and lengthy (thus prone to errors); however, thanks to symbolic calculus libraries such as Sympy or program such as Mathematica, the computation can be substantially simplified. The following results, for example, have been obtained

by using SymPy.

In what follows, i , j and k are three consecutive monomers and

$$\begin{aligned}\mathbf{r}_i &= (x_i, y_i, z_i) & \mathbf{r}_j &= (x_j, y_j, z_j) & \mathbf{r}_k &= (x_k, y_k, z_k) \\ \mathbf{r}_{ji} &= (x_j - x_i, y_j - y_i, z_j - z_i) & r_{ji} &= \sqrt{\mathbf{r}_{ji} \cdot \mathbf{r}_{ji}} \\ \mathbf{r}_{kj} &= (x_k - x_j, y_k - y_j, z_k - z_j) & r_{kj} &= \sqrt{\mathbf{r}_{kj} \cdot \mathbf{r}_{kj}} \\ \cos(\theta_{ijk}) &= \frac{\mathbf{r}_{ji} \cdot \mathbf{r}_{kj}}{r_{ji} \cdot r_{kj}}\end{aligned}$$

The force on particle i , due to the bending interaction, reads

$$\mathbf{f}_i = -\frac{\partial V(\theta_{ijk})}{\partial \mathbf{r}_i} = k_b \frac{1}{r_{ji} \cdot r_{kj}} \left(-\mathbf{r}_{ji} \frac{(\mathbf{r}_{ji} \cdot \mathbf{r}_{kj})}{r_{ji}^2} + \mathbf{r}_{kj} \right) \quad (2.121)$$

The force on particle j , due to the bending interaction, reads

$$\mathbf{f}_j = -\frac{\partial V(\theta_{ijk})}{\partial \mathbf{r}_j} = -k_b \frac{1}{r_{ji} \cdot r_{kj}} \left(-\mathbf{r}_{ji} \frac{(\mathbf{r}_{ji} \cdot \mathbf{r}_{kj})}{r_{ji}^2} + \mathbf{r}_{kj} \frac{(\mathbf{r}_{ji} \cdot \mathbf{r}_{kj})}{r_{kj}^2} + \mathbf{r}_{kj} - \mathbf{r}_{ji} \right) \quad (2.122)$$

The force on particle k , due to the bending interaction, reads

$$\mathbf{f}_k = -\frac{\partial V(\theta_{ijk})}{\partial \mathbf{r}_k} = k_b \frac{1}{r_{ji} \cdot r_{kj}} \left(-\mathbf{r}_{ji} + \mathbf{r}_{kj} \frac{(\mathbf{r}_{ji} \cdot \mathbf{r}_{kj})}{r_{kj}^2} \right) \quad (2.123)$$

2.5 Different Ensembles

2.5.1 Thermostats

So far, we discussed MD simulations in the so-called microcanonical ensemble, where the number of particles (N), the volume (V) and the total energy (E) are fixed⁴. We now deal with performing simulations in other ensembles and, in particular, we will focus on the canonical ensemble, where the system is in contact with a heat bath, whose temperature is fixed. In this case, the total energy is not conserved anymore; in equilibrium it is distributed according to the Boltzmann probability

$$p(E) = \frac{e^{-E/k_B T}}{Z} \quad (2.124)$$

where k_B is the Boltzmann's constant and Z is the partition function

In Section 2.2.5 we have introduced the temperature T_K , computed from the kinetic energy. One can show that the relative variance of T_K in the canonical ensemble is given by

$$\frac{\sigma_{T_K}^2}{\langle T_K \rangle^2} = \frac{\langle T_K^2 \rangle - \langle T_K \rangle^2}{\langle T_K \rangle^2} = \frac{2}{3N} \quad (2.125)$$

where the averages are taken according to the canonical ensemble. Thus the instantaneous temperature T_K in the fluctuates.

In the next sections, we will discuss different ways of “fixing” the temperature employing the so-called *thermostats*. Displaying the correct canonical fluctuations will be used to distinguish good and bad thermostats. Note that non-canonical thermostats may induce strange artifacts thus their use in “production runs” (i.e. when one simulates the systems and collects data to measure some observables) is highly discouraged. However, they could be used to speed up the equilibration, as they are generally less computationally expensive, provided that a suitable switch to a good thermostat is done prior to a production run.

2.5.2 Velocity rescaling

The simplest thermostat, called *velocity rescaling*, consists in a rescaling of all the velocities after each integration step. If we obtain the velocities $\vec{v}_i(t)$ from an integration scheme as the velocity Verlet, we rescale them by a factor λ as follows:

$$\vec{v}_i'(t) = \lambda \vec{v}_i(t) \quad (2.126)$$

We proceed with the integration of the equations of motion at $t + \Delta t$ using these new velocities. The factor λ is obtained from

$$\lambda = \sqrt{\frac{T_0}{T_K(t)}} \quad (2.127)$$

where $T_K(t)$ is the instantaneous temperature defined by Eq. (2.4) and T_0 is the desired temperature. The velocity rescaling fixes $T_K(t)$ to a constant value T_0 at each integration step, therefore it can't reproduce the fluctuations expected from the canonical ensemble.

2.5.3 Berendsen thermostat

The Berendsen thermostat is also based on the same idea as the velocity rescaling. However it is “softer”, as the rescaling is not (necessarily) done at each time step. The rescaling factor is given by:

$$\lambda^2 = 1 + \frac{\Delta t}{\tau} \frac{T_0 - T_K(t)}{T_K(t)} \quad (2.128)$$

⁴The microcanonical ensemble is often referred to as the NVE ensemble

where τ is a characteristic time to be fixed at the beginning of the simulation. The smallest value allowed, $\tau = \Delta t$, yields the velocity rescaling method. Instead, if $\tau \rightarrow \infty$, no rescaling is done and $\lambda = 1$. For finite and suitably chose (not too small) values of τ , the rescaling is $\lambda = 1 + \mathcal{O}(\Delta t)$. If $T_K(t) > T_0$ then $\lambda < 1$, whereas if $T_K(t) < T_0$ then $\lambda > 1$ so that the velocities are increased or decreased according to the value of the instantaneous temperature. The system reaches the desired temperature T_0 only after a characteristic time of the order τ . As for the velocity rescaling, the Berendsen thermostat does not reproduce the canonical distribution of energies.

2.5.4 Andersen thermostat

In the thermostat proposed by Andersen the coupling with the heat bath is represented by “collisions” (or stochastic impulsive forces), that act occasionally on randomly selected particles. When such collisions happen, they enforce a stochastic velocity reset on the particles; the velocity is drawn from the equilibrium Gaussian distribution. These “collisions” can be seen as Monte Carlo moves, that transport the system from one constant-energy surface to another as, in between collisions, the system evolves at constant energy. The stochastic collisions turn the MD simulation, effectively, into a Markov process that, if properly irreducible and aperiodic, generate a canonical distribution of velocities; one can also show that the canonical distribution in phase space is invariant under the application of the Andersen algorithm.

The thermostat is implemented as follows. First, a collision frequency ω is chosen, that sets the “strength” of the coupling. If successive collisions are uncorrelated, then the distribution of time intervals between them $P(t; \omega)$ is a Poissonian $P(t, \omega) = \omega e^{-\omega t}$; in particular, we interpret $P(t; \omega)dt$ as the probability that the next collision will take place in the interval $[t; t + \delta t]$. We update velocities and positions according to the desired scheme (e.g. the Velocity Verlet) for a time Δt . Then, one selects a certain number of particles to undergo a collision: the probability that a particle is selected is $\omega \Delta t$. In practice, for each particle in the system, we draw a random number uniformly distributed in $(0, 1)$: if r_i is the random number assigned to the i -th particle, then the velocity of the i -th particle is resetted and replaced with a value selected from the equilibrium distribution if $r_i < \omega \Delta t$.

2.5.5 Nosé-Hoover thermostat

The Nosé-Hoover thermostat approaches the problem of regulating the temperature of the system in yet a different way. Similarly to the Andersen thermostat, the system is brought into contact (coupled) with a “thermal bath”; however, in this case the coupling is purely deterministic and based on the use of an extended Lagrangian, i.e. a Lagrangian that contains additional, artificial coordinates and velocities (the ones of the bath). The system plus bath form a closed universe, so their total energy is conserved; however, the system exchanges energy with the bath. The net result of this exchange is that the energy of the system is distributed according to the canonical distribution.

In order to construct isothermal Molecular Dynamics, we introduce an additional coordinate s in the Lagrangian of a classical N -body system:

$$\mathcal{L}_{Nose} = \sum_{i=1}^N \frac{m_i}{2} \dot{\mathbf{r}}_i^2 - U(\mathbf{r}^N) + \frac{Q}{2} \dot{s}^2 - Lk_B T \ln(s) \quad (2.129)$$

where L is a parameter, to be fixed, Q is an effective “mass” associated to s and T is the target temperature. The momenta, conjugated to $\mathbf{r}_i$ and s follow directly

$$\mathbf{p}_i = m_i \dot{\mathbf{r}}_i \quad p_s = Q \dot{s} \quad (2.130)$$

The Hamiltonian is simply

$$\mathcal{H}_{Nose} = \sum_{i=1}^N \frac{\mathbf{p}_i^2}{2m_i} + U(\mathbf{r}^N) + \frac{p_s^2}{2Q} + Lk_B T \ln(s) \quad (2.131)$$

Thus, considering N atoms, the extended system has $6N + 2$ degrees of freedom. One can show that, if a simulation is performed in this extended ensemble, the average of a quantity $\mathcal{A}$ that depends on $\mathbf{p}' = \mathbf{p}/s$ and $\mathbf{r}$ is identical to the canonical average if $L = 3N + 1$: $\langle \mathcal{A}(\mathbf{p}', \mathbf{r}) \rangle_{Nose} = \langle \mathcal{A}(\mathbf{p}', \mathbf{r}) \rangle_{NVT}$. Interestingly, there is actually a better choice for L . To find it, it is useful to consider the role of the variable s . As the above equality between the averages suggests, the “scaled” momentum $\mathbf{p}'$ is the real momentum, while $\mathbf{p}$ is the virtual momentum. We make a similar distinction between real and virtual cases for the other variables, indicating real ones with a prime:

$$\mathbf{r}' = \mathbf{r} \quad \mathbf{p}' = \mathbf{p}/s \quad s' = s \quad p'_s = p_s/s \quad \Delta t' = \Delta t/s \quad (2.132)$$

As it stands, we can interpret s as a scaling factor for the time step: as such, the real time step fluctuates during a simulation. This entails that the sampling in the average $\langle \mathcal{A}(\mathbf{p}', \mathbf{r}) \rangle_{Nose}$ is done at (integer) multiples of the virtual time step Δt ; in real time steps, these are not constant. A different choice for L comes from sampling at equal intervals in real time. In such a case, we define the average as

$$\lim_{\tau' \rightarrow \infty} \frac{1}{\tau'} \int_0^{\tau'} dt' \mathcal{A}(\mathbf{p}'(t')/s(t'), \mathbf{r}'(t')) \quad (2.133)$$

It follows from Eq. (2.132) that the times τ' and τ are related by the relation

$$\tau' = \int_0^\tau dt \, 1/s(t) \quad (2.134)$$

Thus, the new average Eq. (2.133) is equal to the old one divided by $\langle 1/s \rangle$, where the average is in the virtual coordinates. If we impose again that the new average should be equal to $\langle \mathcal{A}(\mathbf{p}', \mathbf{r}) \rangle_{NVT}$, then we should fix $L = 3N$. Finally, the equations of motion can be written in terms of both the real and the virtual variables. As in a simulation is not convenient to work with fluctuating time intervals, we focus on the real variable

$$\frac{d\mathbf{r}'_i}{dt'} = s \frac{d\mathbf{r}_i}{dt} = s \frac{\partial \mathcal{H}_{Nose}}{\partial \mathbf{p}_i} = \frac{\mathbf{p}_i}{m_i s} = \frac{\mathbf{p}'_i}{m_i} \quad (2.135)$$

$$\frac{d\mathbf{p}'_i}{dt'} = s \frac{d\mathbf{p}_i/s}{dt} = \frac{d\mathbf{p}_i}{dt} - \frac{1}{s} \mathbf{p}_i \frac{ds}{dt} = \frac{\partial \mathcal{H}_{Nose}}{\partial \mathbf{p}_i} - \frac{1}{s} \mathbf{p}_i \frac{\partial \mathcal{H}_{Nose}}{\partial p_s} = -\frac{\partial U(\mathbf{r}^N)}{\partial \mathbf{r}'_i} - \frac{s' p'_s}{Q} \mathbf{p}'_i \quad (2.136)$$

$$\frac{1}{s} \frac{ds'}{dt'} = \frac{ds}{dt} = \frac{\partial \mathcal{H}_{Nose}}{\partial p_s} = \frac{p_s}{Q} = \frac{s' p'_s}{Q} \quad (2.137)$$

$$\frac{d(s' p'_s/Q)}{dt'} = \frac{s}{Q} \frac{dp_s}{dt} = -\frac{s}{Q} \frac{\partial \mathcal{H}_{Nose}}{\partial s} = \frac{s}{Q} \frac{1}{s} \left(\sum_i \frac{\mathbf{p}_i^2}{m_i s^2} - 3N k_B T \right) = \frac{1}{Q} \left(\sum_i \frac{\mathbf{p}'_i^2}{m_i} - 3N k_B T \right) \quad (2.138)$$

For these equations of motion, the following quantity is conserved:

$$\mathcal{H}'_{Nose} = \sum_{i=1}^N \frac{\mathbf{p}'_i^2}{2m_i} + U(\mathbf{r}^N) + \frac{s' p'_s{}^2}{2Q} + 3N k_B T \ln(s') \quad (2.139)$$

which is not an Hamiltonian. However, having a conserved quantity is still a great feature to have, as it provides a way to check the implementation. Regarding the latter, Hoover showed that the equations of motion can be simplified, as the variables s' , p'_s and Q only occur as $s' p'_s/Q$. Introducing the “friction coefficient” $\xi = s' p'_s/Q$ the equations of motion become:

$$\frac{d\mathbf{r}_i}{dt} \frac{\mathbf{p}_i}{m_i} \quad \frac{d\mathbf{p}_i}{dt} = -\frac{\partial U(\mathbf{r}^N)}{\partial \mathbf{r}_i} - \xi \mathbf{p}_i \quad (2.140)$$

$$\frac{1}{s} \frac{ds}{dt} = \xi \quad \frac{d\xi}{dt} = \frac{1}{Q} \left(\sum_i \frac{\mathbf{p}_i^2}{m_i} - 3Nk_B T \right) \quad (2.141)$$

where we dropped the prime, for convenience. The equations in Eq. (2.140) are Newton's equations with an additional friction term with a time-dependent “friction” coefficient, whose time evolution is described by Eq. (2.141). The friction coefficient ξ can be either positive or negative: for this reason, it is not a real friction coefficient. If ξ is positive, energy is taken from the system, as usual; if ξ is negative, energy is given to the system. The quantity between brackets in Eq. (2.141) is twice the total kinetic energy of the system minus $3Nk_B T$. As such, the extended dynamics works as intended: at steady state ($\xi \approx 0$), Eq. (2.141) makes it so that ξ tends to a value such that the quantity between parenthesis vanishes, so that the kinetic energy is forced to be close to $\frac{3}{2}Nk_B T$, which is the equipartition value.⁵

⁵It can further be shown more rigorously that the Nosé-Hoover thermostat reproduces the canonical fluctuation exactly.

2.6 Langevin Dynamics

The Langevin equation is a stochastic differential equation which describes a system coupled to fast degrees of freedom, where the latter are not explicitly taken into account; however, they are introduced implicitly by means of a random force. The typical example is a macromolecule immersed in a solvent (such as water), where the macromolecule has much larger mass compared the solvent molecules and therefore moves much faster. The solvent molecules are in constant motion due to thermal agitation, they collide with the large particle and exchange energy and momentum with it. If we are not interested in the motion of the solvent molecules (*fast* degrees of freedom), but only in the large particle (*slow* degrees of freedom), we may assume that the solvent molecules exert some random forces on the latter. If collisions occur at a characteristic time τ_{coll} , the Langevin description applies at times $t \gg \tau_{coll}$, when a very large number of collisions have occurred. Practically, this should be taken into account, when deciding which simulation method to use: if the Langevin description does not apply, the solvent should be considered explicitly.

We shortly review the properties of the Langevin equation: in the one dimensional case, it takes the form:

$$m\dot{v} = F - m\gamma v + f_R \quad (2.142)$$

where m is the particle mass, v its velocity, F is a deterministic force applied to the particle, γ the friction coefficient and f_R a random force. In absence of friction and random forces ($\gamma = 0$ and $f_R = 0$), Eq. (2.142) reduces to Newton's equation. The random force has the following properties:

$$\langle f_R(t) \rangle = 0 \quad \langle f_R(t)f_R(t') \rangle = 2A\delta(t - t') \quad (2.143)$$

where A is a constant to be determined. The random forces at different times are uncorrelated, which implies $t \gg \tau_{coll}$. The parameter A is related to the characteristic amplitude of the random force. In the absence of external forces, $F = 0$, the equation (2.142) can be integrated exactly:

$$v(t) = v_0 e^{-\gamma t} + \frac{1}{m} e^{-\gamma t} \int_0^t e^{\gamma s} f_R(s) ds \quad (2.144)$$

where v_0 is the initial velocity; this implies that the average velocity vanishes asymptotically as $\langle v(t) \rangle = v_0 e^{-\gamma t}$, where $\tau = 1/\gamma$ is a characteristic relaxation time. Computing the fluctuations around the average:

$$\langle (v(t) - v_0 e^{-\gamma t})^2 \rangle = \frac{A}{m^2 \gamma} (1 - e^{-2\gamma t}) \quad (2.145)$$

one finds that, at times $t \gg 1/\gamma$, the system relaxes to equilibrium and the equipartition theorem holds: which implies:

$$A = m\gamma k_B T \quad (2.146)$$

We can further integrate Eq. (2.144) to get the position $x(t) = x_0 + \int_0^t v(s) ds$.

The mean square displacement reads

$$\langle (x(t) - x_0)^2 \rangle = \left(v_0^2 - \frac{A}{m^2 \gamma} \right) \left(\frac{1 - e^{-\gamma t}}{\gamma} \right)^2 + \frac{2A}{m^2 \gamma} \left(t - \frac{1 - e^{-\gamma t}}{\gamma} \right) \quad (2.147)$$

This expression has two interesting limiting behaviors. At short times $t \ll 1/\gamma$ one finds:

$$\langle (x(t) - x_0)^2 \rangle \approx v_0^2 t^2 \quad (2.148)$$

which means that there is a *ballistic* regime, i.e. a period of time where the particle moves with a constant speed v_0 . At times, much longer than $1/\gamma$, one has

$$\langle (x(t) - x_0)^2 \rangle \approx \frac{2A}{m^2 \gamma^2} t = 2Dt \quad (2.149)$$

which describes a diffusive behavior, with the mean squared displacement increasing linearly with time. The prefactor⁶ D is the diffusion constant or diffusion coefficient, whose value is given by the Einstein relation:

$$D = \frac{k_B T}{m\gamma} \quad (2.150)$$

Notice that it is different from the Einstein-Stokes relation, reported here for a spherical object of radius R , $D = \frac{k_B T}{6\pi\eta R}$, η being the viscosity, as we are neglecting hydrodynamics effects. Physically, at very short times the random kicks generate a correlated motion; if this regime is unphysical, due to the difference in time scales between τ_{coll} and the characteristic time scale of the system, then an *overdamped* Langevin (or Brownian) dynamics should be considered. In practice, inertial effects are neglected and $\dot{v} = 0$; thus, Eq. (2.142) becomes

$$\dot{r} = \frac{F}{m\gamma} F + \frac{f_R}{m\gamma} \quad (2.151)$$

The equation Eq. (2.151) describes a purely diffusive motion.

2.6.1 Numerical integration of the Langevin equation

We show now how the Langevin equation can be discretized to implement it in Molecular Dynamics simulations. There are three possible approaches:

1. a first order scheme (sometimes called *Euler-Maruyama*, that is useful in the overdamped limit
2. a quick and easy second order scheme, which just amounts to including a Langevin force in a Velocity Verlet scheme (this will not be discussed)
3. a second order scheme that implements a stochastic correction.

We consider only the one-dimensional case; however, being the random forces completely uncorrelated, the extension to $d=2,3$ is straightforward. We write the equation in the following form

$$m\ddot{r}(t) = F(r(t)) - m\gamma\dot{r}(t) + \sqrt{2m\gamma k_B T}\eta(t) \quad (2.152)$$

with $\langle\eta(t)\eta(t')\rangle = \delta(t' - t)$. The main issue is that $\eta(t)$ is not differentiable: however, we can define integrals of these processes, and therefore, it is useful to introduce a process $w(t)$, known as a *Wiener process*, such that $\eta(t) = dw/dt$. We can derive some properties of $w(t)$ if we assume a small time interval Δt .

One of the main properties of the Wiener process is the following:

$$\langle w(s)w(s') \rangle = \int_0^s \int_0^{s'} \langle \eta(t)\eta(t') \rangle dt dt' = \int_0^s \int_0^{s'} \delta(t' - t) dt dt' = \min(s, s') \quad (2.153)$$

(to prove the last step one can consider separately the two cases $s > s'$ and $s' > s$.) Using the previous relation we can show that

$$\langle (w(t + \Delta t) - w(t))^2 \rangle = \langle w^2(t + \Delta t) + w^2(t) - 2w(t + \Delta t)w(t) \rangle = \Delta t \quad (2.154)$$

We can generate the random variable $w(t + \Delta t) - w(t)$ using, at any time step, a Gaussian distributed random number ξ with zero mean and unit variance (i.e. $\langle \xi \rangle = 0$ and $\langle \xi^2 \rangle = 1$) as follows:

$$w(t + \Delta t) - w(t) = \sqrt{\Delta t} \xi \quad (2.155)$$

⁶in general the proportionality term is $2dD$, d being the dimensionality of the system

From this relation we can easily derive a first order integrator of the Langevin equation in the overdamped limit Eq. (2.151). The first order discretization of the derivatives gives:

$$m\gamma(r(t + \Delta t) - r(t)) = F(r(t))\Delta t + \sqrt{2mk_B T} [w(t + \Delta t) - w(t)] \quad (2.156)$$

and using Eq. (2.155) we get first order integrator of the overdamped Langevin equation

$$r(t + \Delta t) = r(t) + \frac{F(r(t))}{m\gamma} \Delta t + \sqrt{\frac{2k_B T}{m\gamma}} \xi \sqrt{\Delta t} \quad (2.157)$$

In practice at any time step one needs to generate a (Gaussian) random number ξ with zero mean and unit variance. The peculiar feature of this integrator is the presence of a square root of the time step Δt . This factor is due to the property Eq. (2.155), which has a clear physical origin. The Wiener process is the integral of the random noise $w(\tau) = \int_0^\tau \eta(t)dt$. We can view $\eta(t)$ as the steps of a random walk and $w(\tau)$ is then the sum of all increments i.e. the total distance covered by the walker after a time τ . Due to the diffusive nature of random walks this distance scales as $\sqrt{\tau}$.

Having discussed the main features of the integrator in the simple overdamped case we are ready to present a more detailed integrator which takes the inertial term into account and it is accurate to order Δt^2 . We express the Langevin equation in the following differential form:

$$dr(t) = v(t)dt \quad (2.158)$$

$$dv(t) = f(r(t))dt - \gamma v(t)dt + \sigma dw(t) \quad (2.159)$$

where $\sigma = \sqrt{2k_B T \gamma / m}$ and $f(r) = F(r)/m$ and $w(t)$ is the Wiener process. We start by integrating Eqs. (2.158), (2.159) from t to $t + \Delta t$:

$$r(t + \Delta t) = r(t) + \int_t^{t+\Delta t} v(s)ds \quad (2.160)$$

$$v(t + \Delta t) = v(t) + \int_t^{t+\Delta t} f(r(s))ds - \gamma \int_t^{t+\Delta t} v(s)ds + \sigma [w(t + \Delta t) - w(t)] \quad (2.161)$$

Note that these contains integrals in v and f which produce terms of the order Δt and higher in the discretization. We start from the integral in the velocity for which Eq. (2.161) can be used to lowest order:

$$v(s) \approx v(t) + (s - t)f(r(t)) - (s - t)\gamma v(t) + \sigma[w(s) - w(t)] \quad (2.162)$$

where $t \leq s \leq t + \Delta t$. Integrating the previous result from t to $t + \Delta t$ we get:

$$\int_t^{t+\Delta t} v(s) ds = \Delta t v(t) + C(t) \quad (2.163)$$

where we have defined

$$C(t) \equiv \frac{\Delta t^2}{2} [f(r(t)) - \gamma v(t)] + \sigma \int_t^{t+\Delta t} [w(s) - w(t)]ds \quad (2.164)$$

This expression can be used as approximant of the integrals of the velocity on the Eqs. (2.160) and (2.161). To approximate the time integral of the force appearing in equation (2.161) we proceed in a similar manner. Using the relation

$$\frac{df}{dt} = \frac{\partial f}{\partial r} \dot{r} = \frac{\partial f}{\partial r} v \quad (2.165)$$

we obtain:

$$f(r(s)) = f(r(t)) + \int_t^s v(y) f'(r(y)) dy \approx f(r(t)) + (s - t) v(t) f'(r(t)) \quad (2.166)$$

Again we integrate this equation from t to $t + \Delta t$ and get

$$\int_t^{t+\Delta t} f(r(s)) ds = \Delta t f(r(t)) + \frac{\Delta t^2}{2} v(t) f'(r(t)) = \Delta t \frac{f(r(t + \Delta t)) + f(r(t))}{2} \quad (2.167)$$

where the last step is performed to eliminate the inconvenient derivative of the force using the following relation:

$$f(r(t + \Delta t)) = f(r(t)) + f'(r(t))(r(t + \Delta t) - r(t)) = f(r(t)) + f'(r(t))v(t)\Delta t \quad (2.168)$$

For the computation of the term $w(t + \Delta t) - w(t)$ in Eq. (2.161) we use again Eq. (2.155). In order to end the calculation we still need to compute $C(t)$ in Eq. (2.164), which contains integral of the difference between two Wiener functions. We note that using (2.153) one can easily prove that:

$$\left\langle (w(t + \Delta t) - w(t)) \int_t^{t+\Delta t} ds' (w(s') - w(t)) \right\rangle = \frac{\Delta t^2}{2} \quad (2.169)$$

$$\left\langle \int_t^{t+\Delta t} ds (w(s) - w(t)) \int_t^{t+\Delta t} ds' (w(s') - w(t)) \right\rangle = \frac{\Delta t^3}{3} \quad (2.170)$$

which can be viewed as higher order counterparts of Eq. (2.154). Using the same strategy as for the first order integrator we see that Eq. (2.170) can be fulfilled if:

$$\int_t^{t+\Delta t} ds (w(s) - w(t)) = \Delta t^{3/2} \eta \quad (2.171)$$

with η a random number with the properties $\langle \eta \rangle = 0$ and $\langle \eta^2 \rangle = 1/3$. Plugging in Eq. (2.171) and Eq. (2.155) in Eq. (2.169) we find $\langle \xi \eta \rangle = \frac{1}{2}$, hence the two random numbers are correlated. To solve this issue in practice we can decompose η into two uncorrelated random numbers as $\eta = a\xi + b\theta$, with $\langle \xi \theta \rangle = 0$ and $\langle \theta^2 \rangle = 1$. The coefficients a and b can be determined by imposing $\langle \xi \eta \rangle = \frac{1}{2}$ and $\langle \eta^2 \rangle = 1/3$, from which we get $a = 1/2$ and $b = 1/(2\sqrt{3})$. This implies:

$$\int_t^{t+\Delta t} ds (w(s) - w(t)) = \Delta t^{3/2} \left(\frac{\xi}{2} + \frac{\theta}{2\sqrt{3}} \right) \quad (2.172)$$

Now plugging in Eqs. (2.163), (2.167), (2.155), (2.172) in Eqs. (2.160), (2.161) we get the second order integrator of the Langevin equation

$$\begin{aligned} r(t + \Delta t) &= r(t) + \Delta t v(t) + C(t) \\ v(t + \Delta t) &= v(t) + \frac{\Delta t}{2} [f(r(t + \Delta t)) + f(r(t))] - \Delta t \gamma v(t) + \sigma \sqrt{\Delta t} \xi(t) - \gamma C(t) \\ C(t) &= \frac{\Delta t^2}{2} [f(r(t)) - \gamma v(t)] + \sigma \Delta t^{3/2} \left(\frac{1}{2} \xi(t) + \frac{1}{2\sqrt{3}} \theta(t) \right) \end{aligned} \quad (2.173)$$

Notice that the integrator reduces to the Velocity Verlet integrator if $\gamma = 0$; however, setting $T=0$ (with $\gamma \neq 0$) will not produce the same effect. Actually, in such a case the dynamics will be only dominated by friction and the system will inevitably stop moving.

5.3 Reweighting

5.3.1 Umbrella sampling

Umbrella sampling is an example of a class of Monte Carlo techniques, whose strategy is to facilitate unlikely trajectories through states that are very rare in the canonical ensemble. These trajectories are transitions needed to connect otherwise isolated relevant regions of the phase space. Thus, umbrella sampling tries to not miss the sampling of some important configurations. A typical example is that of two energy minima separated by a high saddle point of the energy function (high compared to the thermal energy scale $1/\beta$). The sampling of configurations only in one of the two minima would lead to *quasi ergodicity*.

We consider a generic change of measure $\pi \rightarrow g$,

$$\langle O \rangle_\pi = \sum_{\mathcal{C}} O(\mathcal{C}) \pi(\mathcal{C}) = \sum_{\mathcal{C}} O(\mathcal{C}) \frac{\pi(\mathcal{C})}{g(\mathcal{C})} g(\mathcal{C}) = \left\langle O \frac{\pi}{g} \right\rangle_g \quad (5.14a)$$

Umbrella sampling adopts a modified energy function $E_g(\mathcal{C}) = E(\mathcal{C}) + U(\mathcal{C})$ and

$$g(\mathcal{C}) = \frac{e^{-\beta[E(\mathcal{C})+U(\mathcal{C})]}}{\sum_{\mathcal{C}} e^{-\beta[E(\mathcal{C})+U(\mathcal{C})]}} \quad (5.15)$$

where $U(\mathcal{C})$ is an auxiliary potential energy, so that

$$\langle O \rangle_\pi = \frac{\left\langle O \frac{e^{-\beta E}}{g} \right\rangle_g}{\left\langle \frac{e^{-\beta E}}{g} \right\rangle_g} = \frac{\langle O e^{\beta U} \rangle_g}{\langle e^{\beta U} \rangle_g} \quad (5.16)$$

Then, as described for importance sampling (Sec.??), rather than inefficiently sampling $\mathcal{C}$'s with a uniform distribution and applying the weights $g(\mathcal{C})$ to compute averages $\langle \dots \rangle_g$, one uses importance sampling with

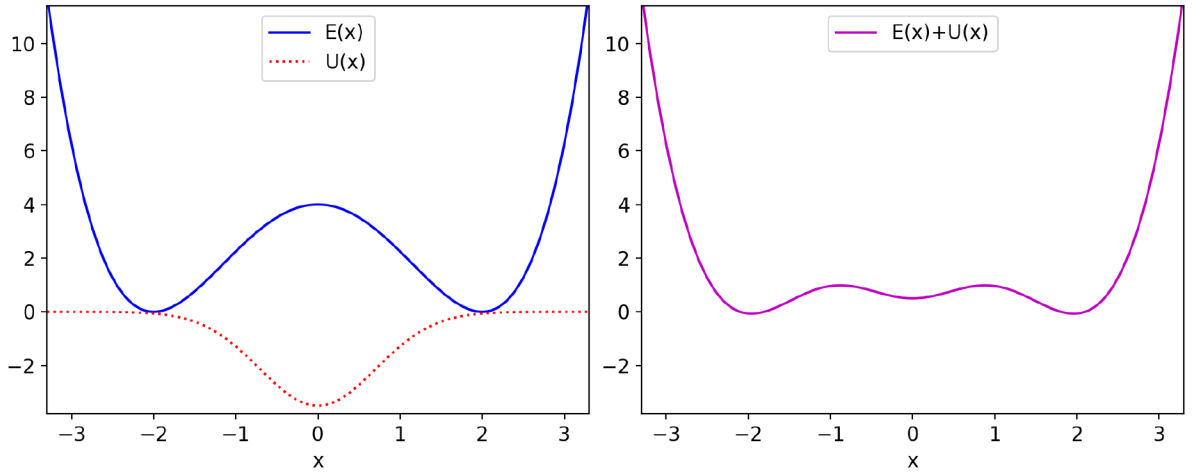


Figure 5.3: Sketch of the umbrella sampling technique. Here, x represents a reaction coordinate on a multi-dimensional system, i.e. a combination of microscopic variables that well represents the difference between configurations. In umbrella sampling, one introduces $U(x)$ so that the saddle between minima of the energy $E(x_s)$ becomes smaller for $E(x_s) + U(x_s)$ (here $x_s = 0$). Thus, for low β , sampling of configurations occurring with frequency $\propto \exp[-\beta(E(x_s) + U(x_s))]$ is possible, as opposed to that with frequency $\propto \exp[-\beta E(x_s)]$.

g in the simulations to obtain a series $\{\mathcal{C}_i\}$ of $i \leq M$ samples, from which

$$\langle O \rangle_\pi \simeq \frac{\sum_{i=1}^M O(\mathcal{C}_i) e^{\beta U(\mathcal{C}_i)}}{\sum_{i=1}^M e^{\beta U(\mathcal{C}_i)}} \quad (5.17)$$

One wants to exploit the additional freedom of choosing $U(\mathcal{C})$ to allow the system to perform transitions between rare states. For example, $U(\mathcal{C})$ may be tailored to lower some saddle point energies, e.g., by setting its minima where these saddle points are expected (see the example in Fig. 5.3). Sampling is then augmented around these saddles. This enrichment in the statistics is compensated by their lower weight $e^{\beta U}$, compared to a weight 1 of pure importance sampling.

5.3.2 The histogram method

The (single) histogram method collects data and reweights them to different temperatures. Going back to the previous section, it amounts to running a simulation (importance) sampling with $\pi \propto e^{-\beta E}$ (its averages are denoted here simply as $\langle \dots \rangle$) and then reweighting at $\beta' \neq \beta$ with $g \propto e^{-\beta' E}$:

$$\langle O \rangle = \frac{\left\langle O \frac{e^{-\beta E}}{g} \right\rangle_g}{\left\langle \frac{e^{-\beta E}}{g} \right\rangle_g} = \frac{\langle O e^{(\beta' - \beta) E} \rangle_g}{\langle e^{(\beta' - \beta) E} \rangle_g} \quad (5.18)$$

It is termed *histogram* method because a histogram of the energy

$$N(E') = \sum_{n=1}^M \delta_{E', E(\mathcal{C}_n)} \quad (5.19)$$

is sufficient for computing energy and specific heat at a different temperature. For example,

$$\left\langle E \frac{e^{-\beta E}}{g} \right\rangle_g = \frac{1}{M} \sum_E E N(E) e^{(\beta' - \beta) E} . \quad (5.20)$$

and thus

$$\langle E \rangle = \frac{\sum_E E N(E) e^{(\beta' - \beta) E}}{\sum_E N(E) e^{(\beta' - \beta) E}} . \quad (5.21)$$

However, to apply (5.18) for a generic observable O , it is convenient to record tables of samples i with their values $O_i \equiv O(\mathcal{C}_i)$, $E_i \equiv E(\mathcal{C}_i)$. Hence, when sampling with g in a Monte Carlo,

$$\left\langle O \frac{e^{-\beta E}}{g} \right\rangle_g = \frac{1}{M} \sum_{i=1}^M O_i e^{(\beta' - \beta) E_i} , \quad (5.22)$$

$$\langle O \rangle = \frac{\sum_{i=1}^M O_i e^{(\beta' - \beta) E_i}}{\sum_{i=1}^M e^{(\beta' - \beta) E_i}} . \quad (5.23)$$

In practice, one samples at β' with g and reweights the results to β . The method works well as long the energy distributions at the two (inverse) temperatures β and β' are significantly overlapped.

5.3.3 The multiple histogram method

Despite the name, the multiple histogram method (MHM¹) by Ferrenberg and Swendsen [1] is not a direct extension of the (single) histogram method but rather an optimized usage of the information gathered by different simulations, for calculating the density of states $\rho(E)$, and in turn, use ρ to estimate quantities at values of the parameters not used in Monte Carlo sampling. In the following, we report the basic steps for the derivation of the MHM. Our treatment follows closely that of Newman and Barkema's book [2] (Ch.8, which contains more details). The MHM is naturally applied to the multiple Markov chain strategy, in which K Markov chains collect data in parallel, and allows for computing observables at different temperatures in the range of the sampled ones. Indices i, j, k will refer to simulations at different inverse temperatures $\beta_i, \beta_j, \beta_k$.

When we run a Monte Carlo with importance sampling for a canonical system at inverse temperature β , the probability $\pi(E)$ of generating a sample with energy E in a step is

$$\pi(E) = \rho(E) \frac{e^{-\beta E}}{Z} \quad (5.24)$$

with partition function Z . Usually, neither the density of states ρ nor Z are known and, with M measurements, our best estimate of $\pi(E)$ derives from the histogram $N(E)$,

$$\pi(E) = \frac{N(E)}{M} \quad (5.25)$$

from which we have

$$\rho(E) = \frac{N(E)}{M} \frac{Z}{e^{-\beta E}} \quad (5.26)$$

If we run many simulations at different β_i 's, we get different estimates

$$\rho_i(E) = \frac{N_i(E)}{M_i} \frac{Z_i}{e^{-\beta_i E}} \quad (5.27)$$

that should agree with each other. The main point is obtaining an optimized weighted average of these $\rho_i(E)$ estimates, which takes into account their relevance at a given energy E . Intuitively, the larger is $N_i(E)$, the more relevant is a contribution.

In general, given measurements with standard error σ_i , a weighted average quantity (assuming Gaussian statistics) is obtained as

$$\bar{x} = \frac{\sum_i x_i / \sigma_i^2}{\sum_i 1 / \sigma_i^2} \quad (5.28)$$

i.e., weights are the inverse variances $1/\sigma_i^2$. For us, these $1/\sigma_i^2 = N_i(E)$ are equal to the number of samples. To prove this point, let us assume that the error in the number of samples for each bin derives from Poissonian statistics,

$$\Delta N_i(E) = \sqrt{N_i(E)} \quad (5.29)$$

More rigorously, this error would be computed after obtaining the average $\overline{N_i(E)}$ over several runs,

$$\Delta N_i(E) = \sqrt{\overline{N_i(E)}} \quad (5.30)$$

Distinguishing $\overline{N_i(E)}$ from $N_i(E)$ is relevant because $\overline{N_i(E)}$ is related to the exact density of states $\rho(E)$,

$$\rho(E) = \frac{\overline{N_i(E)}}{M_i} \frac{Z_i}{e^{-\beta_i E}} \quad (5.31)$$

¹Often, in the literature, it is also termed weighted histogram analysis method (WHAM).

5.3 Reweighting

With an infinite number of samples M_i in a run i , this formula would be sufficient for computing $\rho_i(E) = \rho(E)$. In practice, we have finite runs and we need to compute the error σ_i on $\rho_i(E)$,

$$\sigma_i = \frac{\Delta N_i(E)}{M_i} \frac{Z_i}{e^{-\beta_i E}} = \frac{\sqrt{N_i(E)}}{M_i} \frac{Z_i}{e^{-\beta_i E}} \quad (5.32)$$

and thus, with (5.31), we get the variance

$$\sigma_i^2 = \overline{N_i(E)} \left(\frac{1}{M_i} \frac{Z_i}{e^{-\beta_i E}} \right)^2 = \frac{1}{\overline{N_i(E)}} \rho(E)^2 \quad (5.33)$$

The inverse of these variances are the weights we need for computing the weighted averages, and, as we wanted to prove, each $1/\sigma_i^2$ is proportional to the (ideal) number of samples $\overline{N_i(E)}$. Hence, the weighted average for the density of states (5.27) is

$$\begin{aligned} \rho(E) &= \frac{\sum_i \overline{N_i(E)} \rho_i(E)}{\sum_j \overline{N_j(E)}} \\ &= \frac{\sum_i \overline{N_i(E)} [N_i(E)/M_i] [Z_i/e^{-\beta_i E}]}{\sum_j \overline{N_j(E)}} \end{aligned} \quad (5.34)$$

Note the presence of $\overline{N_i(E)}$ and $N_i(E)$ in the same equation and their different role in the process: $N_i(E)$ is going to be used in actual calculations, while the unknown $\overline{N_i(E)}$ appears only in the above derivation and disappears by using (5.31) to get rid of it:

$$\rho(E) = \frac{\sum_i N_i(E)}{\sum_j M_j Z_j^{-1} e^{-\beta_j E}} \quad (5.35)$$

Equation 5.35 still contains the unknown partition functions Z_j . Since they are

$$Z_k = \sum_E \rho(E) e^{-\beta_k E} \quad (5.36)$$

we can use our estimate for $\rho(E)$ to obtain a set of self-consistent equations for the ensemble of these partition functions,

$$Z_k = \sum_E \frac{\sum_i N_i(E)}{\sum_j M_j Z_j^{-1} e^{(\beta_k - \beta_j) E}} \quad (5.37)$$

The implementation of the MHM involves first solving iteratively (5.37), starting from some initial guesses for the Z_k 's (e.g., $Z_k = 1$ for all k), to obtain estimates of the partition functions Z_k at all sampled temperatures. Once these have converged, we use them to compute the partition function at any β ,

$$Z(\beta) = \sum_E \frac{\sum_i N_i(E)}{\sum_j M_j Z_j^{-1} e^{(\beta - \beta_j) E}} \quad (5.38)$$

Then, with this estimator for $Z(\beta)$ and (5.35) for the density of states, we finally have all the tools for computing observables that depend on the energy, such as the mean internal energy

$$\begin{aligned} U(\beta) &= \frac{1}{Z(\beta)} \sum_E E \rho(E) e^{-\beta E} \\ &= \frac{1}{Z(\beta)} \sum_E E \frac{\sum_i N_i(E)}{\sum_j M_j Z_j^{-1} e^{(\beta - \beta_j) E}} \end{aligned} \quad (5.39)$$

Implementation of the multiple histogram method

Lists rather than histograms: Modern computers allow the storage of large volumes of data. Hence, one may save to file all samples' values of the energy (E_i) and observables (O_i). This list of $n \leq M$ measurements may then be used to replace the histograms in the MHM. Eq. (5.37) becomes

$$\begin{aligned} Z_k &= \sum_E \frac{\sum_i N_i(E)}{\sum_j M_j Z_j^{-1} e^{(\beta_k - \beta_j)E}} \\ &= \sum_{i,E} \frac{N_i(E)}{\sum_j M_j Z_j^{-1} e^{(\beta_k - \beta_j)E}} \\ &= \sum_{i,n} \frac{1}{\sum_j M_j Z_j^{-1} e^{(\beta_k - \beta_j)E_{in}}} \end{aligned} \quad (5.40)$$

where the index n runs over all samples M_i of the i -th simulation, and E_{in} is the energy of such sample n . Every other observable is computed similarly:

$$Z(\beta) = \sum_{i,n} \frac{1}{\sum_j M_j Z_j^{-1} e^{(\beta - \beta_j)E_{in}}} \quad (5.41)$$

$$O(\beta) = \frac{1}{Z(\beta)} \sum_{i,n} \frac{O_{in}}{\sum_j M_j Z_j^{-1} e^{(\beta - \beta_j)E_{in}}} \quad (5.42)$$

where O_{in} is the value of the observable of sample n in the i -th simulation. Note that (5.42) allows avoiding the construction of two-dimensional histograms $N(E, O)$. Another advantage is that it works without discretization or binning in case E and O are continuous variables.

Rescaling of Z_k 's: In (5.37), the values of the partition functions Z_k might get too large or too small. This might cause over/underflow in the computer. To mitigate this problem, one may exploit the fact that (5.37) does not change if all Z 's are rescaled by a common factor A . A good choice for centering Z 's is multiplying them by the inverse of the arithmetic mean of their extrema,

$$A = \left[\min_k (Z_k) \max_k (Z_k) \right]^{-1/2}. \quad (5.43)$$

Log-sum trick: The sum of positive real numbers x_1, x_2 of much different magnitude may lead to numerical imprecision or over/underflow. One can exploit the fact that their logarithms, respectively ℓ_1 and ℓ_2 , have a much smaller range:

$$x_1 + x_2 = e^{\ell_1 + \ell_2} = e^{\ell_1} (1 + e^{\ell_2 - \ell_1}) \quad (5.44)$$

In the last line, the exponential of $\ell_2 - \ell_1$ is usually numerically more stable than the exponential of either ℓ_1 or ℓ_2 . This is generalized to the sum of many positive numbers:

$$\sum_i x_i = \sum_i e^{\ell_i} = e^{\ell_q} \sum_i e^{\ell_i - \ell_q} \quad (5.45)$$

Here, q is the index of one of the numbers, for instance, of their median value. Again, the terms $e^{\ell_i - \ell_q}$ are numerically more stable than the single e^{ℓ_i} ones. Thus, in a computer program, one may work with the logs ℓ 's and (5.45) rather than the actual values x 's. This is useful for all formulas of the single or multiple histogram methods in which one sums numbers of different magnitudes, such as the combination with the partition

5.3 Reweighting

functions in the denominator of (5.37). Defining $\ell_j = \log Z_j$, $\lambda_{kj}(E) = \log M_j - \ell_j + (\beta_k - \beta_j)E$, and $\nu(E) = \sum_i N_i(E)$, such equation is rewritten as

$$Z_k = \sum_E \frac{\nu(E)}{\sum_j e^{\lambda_{kj}(E)}} = \sum_E \frac{\nu(E)}{e^{\lambda_{kq}(E)} \sum_j e^{\lambda_{kj}(E) - \lambda_{kq}(E)}} \quad (5.46)$$

where q is chosen, for instance, as $q = 1$ or as one of the intermediate indices in the number of chains.

Convergence: In the iterative application of (5.37) to compute all Z_k , one may wonder when to stop. A criterion is setting a lower threshold to the sum of the squared relative changes at iteration t ,

$$\Delta^2(t) = \sum_k \left(\frac{Z_k^{(t)} - Z_k^{(t-1)}}{Z_k^{(t)}} \right)^2 \quad (5.47)$$

When $\Delta^2(t)$ becomes smaller than ϵ^2 (e.g., with $\epsilon = 10^{-7}$), the iterations are stopped, and the obtained Z_k are used for the computation of thermodynamic quantities.

2.6 Cell list

When the number of interacting particles N is large, the computation of pairwise interactions slows down a simulation. Indeed, the number $N(N - 1)/2 \sim N^2$ of pair interactions scales quadratically with N . In practice, the computation of all these forces is unnecessary if the interaction potential is short-ranged with cutoff σ_c . In this case, we should check the presence of other particles in a radius $\leq \sigma_c$ around each particle. The cell list is a simple procedure that enables such a check. Its advantage is CPU usage $\sim N$, which speeds up the simulation.

Implementing a cell list starts with designing a grid of cells. Typically, their edge length $\ell \geq \sigma_c$, where σ_c is the smallest among cutoffs in the simulation so that the volume composed by the cell of a reference particle i and of its neighbors in d dimensions (3^d cells, e.g., a 3×3 square if $d = 2$) contains all particles j that can interact with i ⁶. We enforce that the sides B_x, B_y , and B_z of the simulation box are integer multiples of ℓ , denoted as $I_x = B_x/\ell, I_y = B_y/\ell$, and $I_z = B_z/\ell$. For example, in Fig. 2.1, we show a case with $B_x = B_y = 4\ell$ (hence, with $N_c = I_x \times I_y = 4 \times 4 = 16$ cells, indexed by $0 \leq c \leq N_c - 1$ or $1 \leq c \leq N_c$), and $N = 19$ particles.

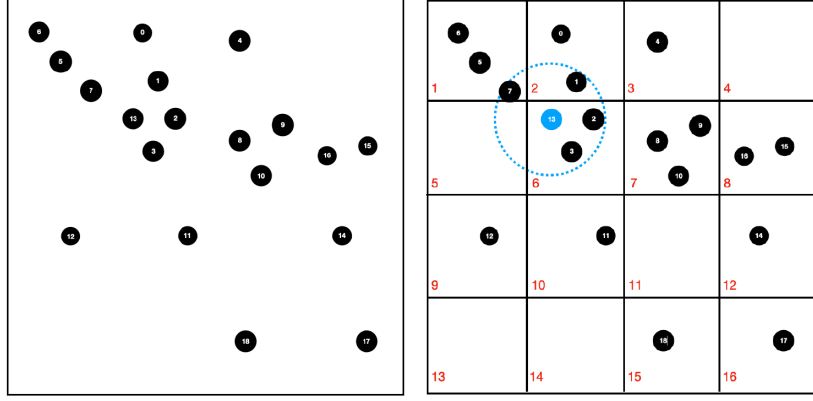


Figure 2.1: Two-dimensional example of the partitioning of the volume into cells, in this case, squares of side ℓ . The blue dotted circle shows the interaction cutoff $\sigma_c < \ell$ around a highlighted particle $i = 13$ in cell $c = 6$.

A tag list $\tau(c)$ keeps track of the occupation of cells at each stage $\mathcal{T}$ of the simulation. To fix the ideas, we choose $\mathcal{T}$ as an index increasing by one at the beginning of every molecular dynamics (MD) step. During each MD step, the value $\mathcal{T}$ is assigned to $\tau(c)$ if a particle occupies cell c . Hence, at the beginning of the step, all $\tau(c)$'s are at most $\mathcal{T} - 1$. The other structures we need are a list of *head pointers* $\phi(c)$ (of size N_c) and a list of particles $\nu(i)$ (of size N). Let us explain their meaning hereafter.

Starting from particle $i = 0$ up to particle $i = N - 1$, we create the cell list as follow:

1. For particle i with coordinates (x_i, y_i, z_i) , determine the cell c it belongs to. For example,

$$c = \text{floor}(x_i/\ell) + I_x \cdot \text{floor}(y_i/\ell) + I_x I_y \cdot \text{floor}(z_i/\ell) \quad (2.155)$$

2. If $\tau(c) \neq \mathcal{T}$, start the list of particles for cell c by assigning $\phi(c) = i$, and setting $\nu(i) = \Omega$, where for example $\Omega = -1 \notin [0, 1, \dots, N]$ denotes the end of a list. Thus, so far, particle i is the only one in c , and we mark c as occupied by setting $\tau(c) = \mathcal{T}$.
3. If rather $\tau(c) = \mathcal{T}$, some other particle $j < i$ was already added to the list, and we need to continue such list by adding i at its head: setting first $\nu(i) = \phi(c)$ links i to the last particle $j = \phi(c)$ added to c , then setting $\phi(c) = i$ redirects the head of the c -list to this new i .

⁶If $\ell < \sigma_c$, however, one can still easily draw the list of cells where interaction is possible.

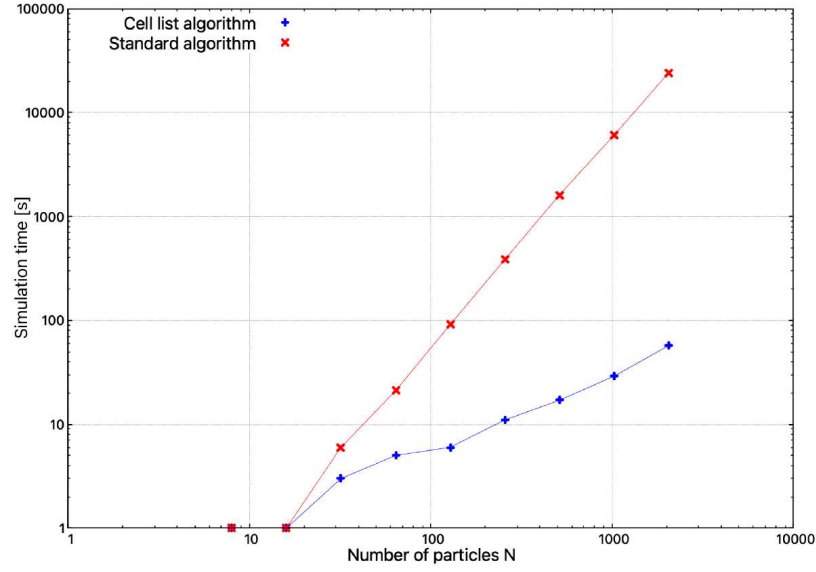


Figure 2.2: Scaling with ne number of particles of the CPU time for the cell list scheme ($\sim N$) and for the naive scheme ($\sim N^2$). The simulation is by Alberto Raiola for a model of interacting active particles.

The final result allows us to quickly scan the particles in each cell c . If $\tau(c) \neq \mathcal{T}$, the cell is empty during the current MD step $\mathcal{T}$. Otherwise, the head of the list for c in ν is found by $i = \phi(c)$, then the next element in ν is located at $i' = \nu(i)$, the next one again at $i'' = \nu(i')$, and so on till the last one i^ω is found with $\nu(i^\omega) = \Omega$. Particles in cell c are thus $i, i', i'', \dots, i^\omega$.

As anticipated, the algorithm building the cell list takes a CPU time of order N , which is better than the naive $\sim N^2$ *standard* algorithm. Fig. 2.2, by Alberto Raiola, shows numerically the two scalings with time for a simulation of active particles. The $\sim N$ scaling for the cell list seems to be there, also because the scanning of each cell list is of order one as long as cells are not overcrowded.